Modern C++ Notes

Christian Bollinger

October 21, 2026

Contents

1 Introduction **7**
1.1 Required Software . . . 7

2 Basic Language Facilities **9**
2.1 What is C++ . . . 9
2.2 First C++ Program . . . 9
2.3 C++ Compilation Process . . . 9
2.4 Primitive Types & Variables . . . 10
2.5 Basic Input/Output . . . 11
2.6 Function I . . . 11
2.7 Function II . . . 12
2.8 **TODO** Overview of Debugging . . . 13
2.9 Uniform Initilalization . . . 13
2.10 Pointers . . . 15
2.11 Assignment 1 . . . 16
2.12 References . . . 17
2.13 Assignment 2 . . . 18
2.14 References vs. Pointers . . . 19
2.15 The const Qualifier . . . 21
2.16 cons Qualifier & Compound Types . . . 22
2.17 Assignment 3 . . . 23
2.18 Automatic Type Intference (C++11) . . . 24
2.19 Range-Based For Loop 1 (C++11) . . . 25
2.20 Range-Based For Loop 2 (C++11) . . . 26
2.21 Function Overloading . . . 27
2.22 Inline Functions . . . 30
2.23 Function Pointers . . . 31
2.24 Namespace . . . 31

3 Memory Management - Part I **33**
3.1 Dynamic Memory Allocation Part I (malloc function) . . . 33
3.2 Dynamic Memory Allocation Part II (new operator) . . . 35
3.3 Dynamic Memory Allocation Part III (new[] operator) . . . 35
3.4 Dynamic Memory Allocation Part IV (2D arrays) . . . 36

4 Classes & Objects **38**
4.1 Object Oriented Programming Basics . . . 38
4.2 Class . . . 39
4.3 Constructor & Destructor . . . 41
4.4 Structure . . . 44
4.5 Non-Static Data Member initializers (C++11) . . . 45
4.6 this Poiner . . . 48
4.7 Static Class Members . . . 49
4.8 **TODO** Constant Member Functions . . . 60
4.9 **TODO** Copy Constructor - Part I . . . 60
4.10 **TODO** Copy Constructor - Part II . . . 60
4.11 **TODO** Delegating Constructors (C++11) . . . 60
4.12 **TODO** Default & Deleted Functions (C++11) . . . 60

5 Move Semantic **61**
5.1 L-values, R-values & R-value References (C++11) . . . 61
5.2 Move Semantics - Basics (C++11) . . . 63
5.3 Move Semantics - Implementation (C++11) . . . 63
5.4 Rule of Five . . . 65
5.5 **TODO** Copy Elision . . . 66

5.6 std::move Function (C++11) . . . 66

6 Operator Overloading 68
6.1 Operator Overloading - Part I-III . . . 68
6.2 Operator Overloading - Part IV (Friend Keyword) . . . 77
6.3 Operator Overloading Part V (Smart Pointer Basics) . . . 79
6.4 Operator Overloading - Part VI (Smart Pointer in C++11) . . . 80
6.5 Operator Overloading - Part VII (Rules) . . . 81
6.6 Type Conversions - Part I (Basics) . . . 81
6.7 Type Conversions - Part II (Primitive to User Type) . . . 82
6.8 Type Conversion - Part III (User to Primitive Type) . . . 83
6.9 Type Conversion - Part IV (User defined Type to User defined Type) . . . 84
6.10 Initialization vs. Assignment & Member Initialization . . . 85

7 Memory Management - Part II 88
7.1 Integer Class . . . 88
7.2 Raw Pointers . . . 90
7.3 std_unique_ptr . . . 91
7.4 Sharing Raw Pointer . . . 92
7.5 Sharing std::unique_ptr . . . 93
7.6 Sharing std::shared_ptr . . . 94
7.7 Sharing Weak Ownership . . . 95
7.8 Sharing std::weak_ptr . . . 96
7.9 Circular References . . . 97
7.10 Deleter . . . 99
7.11 Dynamic Arrays . . . 102
7.12 Make Functions . . . 103

8 More C++ Goodies 105
8.1 ENUMS- Part I (Basics) . . . 105
8.2 **TODO** ENUMS - Part II (Scoped Enums C++11) . . . 106
8.3 **TODO** Strings - Part I (Raw Strings) . . . 106
8.4 **TODO** Strings - Part II (std::strings) . . . 106
8.5 Assignment I . . . 106
8.6 Strings - Part III (String Streams) . . . 106
8.7 **TODO** User-Defined Literals . . . 106
8.8 Constant Expression - constexpr . . . 106
8.9 **TODO** std::initializer_list (C++11) . . . 106
8.10 **TODO** Dynamic Array (std::vector) . . . 106
8.11 **TODO** Assignment III . . . 106
8.12 **TODO** Union - I . . . 106
8.13 **TODO** Union - II . . . 106

9 Object Oriented Programming 107
9.1 Access Modifiers . . . 107
9.2 Project - I Introduction to Banking Application . . . 107
9.3 Project - II (Account Class) . . . 108
9.4 Project - II (Savings Class - Inheritance) . . . 110
9.5 Project - III (Checking Class - Constructor Inheritance) . . . 111
9.6 Project - IV (Transaction Class) . . . 112
9.7 Project - IV (Transaction Class - Base Class Argument) . . . 113
9.8 Project - IV (Transaction Class - Virtual Keyword) . . . 114
9.9 Project - VI (Virtual Mechanism Internals I) . . . 115
9.10 Project - VI (Virtual Mechanism Internals II) . . . 116
9.11 Project - VII (override & final specifier C++11) . . . 116
9.12 Project - VIII (Object Slicing) . . . 117
9.13 Project - IX (typed Operator) . . . 118

9.14 Project - X (dynamic_cast operator) . . . 118
9.15 Abstract Class and pure virtual Functions . . . 119
9.16 **TODO** Multiple (Dyamond) Inheritance . . . 119

10 Exception Handling **120**
10.1 Basic Exception Handling - Part I . . . 120
10.2 Basic Exception Handling - Part II (multiple Catch Blocks) . . . 121
10.3 Basic Exception Handling - Part III (Stack unwinding) . . . 122
10.4 Basic Exception Handling - Part IV (Nested Exceptions) . . . 125
10.5 Basic Exception Handling - Part V (Constructor & Destructor) . . . 126
10.6 Basic Exception Handling - Part VI (noexcept keyword) . . . 128

11 File Input & Output **129**
11.1 Raw String Literals . . . 129
11.2 Introduction to Filesytem Library . . . 129
11.3 File I/O - Part I (Basics) . . . 130
11.4 File I/O - Part II (Error Handling) . . . 132
11.5 File I/O - Part III (Copy Utility) . . . 133
11.6 File I/O - Part IV (Character IO & Seeking) . . . 134
11.7 File I/O - Part V (Binary IO) . . . 136

12 Templates **139**
12.1 Introduction to Templates . . . 139
12.2 Assignment I . . . 140
12.3 Template Argument Deduction & Instantiation . . . 142
12.4 Explicite Specialization . . . 143
12.5 Non-type Template Arguments . . . 144
12.6 Assignment III . . . 145
12.7 Integer Class . . . 147
12.8 Perfect Forwarding - Part I (C++11) . . . 150
12.9 Perfect Forwarding - Part II (C++11) . . . 152
12.10 Variadic Templates - Part I (C++11) . . . 153
12.11 Variadic Templates - Part II (C++11) . . . 154
12.12 Assignment IV . . . 156
12.13 Class Templates . . . 156
12.14 Class Template Explicit Specialization - Part I . . . 158
12.15 Class Template Explicit Specialization - Part II . . . 159
12.16 Assignment . . . 162
12.17 Class Template Partial Specializuation . . . 164
12.18 **TODO** Typedef, Type Alias & Alias Templates (C++11) . . . 166
12.19 Type Traits (C++11) . . . 166
12.20 static_assert (C++11) . . . 167

13 Lambdas **169**
13.1 Callbacks Revisted - Function Pointer . . . 169
13.2 Callbacks - Function Objects . . . 171
13.3 Lambda Expressions . . . 173
13.4 Lambda Expressions - Internals . . . 174
13.5 Lambda Expressions Capture List - Part I . . . 175
13.6 Lambda Expression Capture List - Part II . . . 176
13.7 Lambda Expression Capture List - Part III . . . 177
13.8 Generalized Capture (C++14) . . . 178
13.9 Assignement . . . 179

14 Function Rappers (C++11) **181**
14.1 std::function - I . . . 181
14.2 std::function - II . . . 181

14.3 std::bind - I . . . 183
14.4 std::bind - II . . . 183
14.5 std::bind - III . . . 185

15 Standard Template Library **188**
15.1 Introduction . . . 188
15.2 std::array . . . 189
15.3 std::vector . . . 190
15.4 std::deque . . . 190
15.5 std::list std::forward_list (C++11) . . . 190
15.6 std::set & std::multiset . . . 190
15.7 std::map & std::multimap . . . 190
15.8 Unordered Containers (C++11) - I . . . 190
15.9 Unordered Containers (C++11) - II . . . 190
15.10 std::hash (C++11) . . . 190
15.11 Big O Notation & Performance of Conatainers . . . 190
15.12 Algorithm - Part I . . . 190
15.13 Algorithm - Part II . . . 190
15.14 Container Changes in C++11 - I . . . 190
15.15 Container Changes in C++11 - II . . . 190
15.16 Container Changes in C++11 - III . . . 190
15.17 Container Changes in C++11 - IV . . . 190
15.18 Container Changes in C++11 - V . . . 190

16 C++ Concurrency **191**
16.1 Concurrency Basics . . . 191
16.2 std::thread . . . 191
16.3 std::thread Passing Arguments . . . 192
16.4 std::thread Returning Values . . . 194
16.5 std::thread Synchronization std:mutex . . . 196
16.6 std::thread Synchronization std::lock_guard . . . 197
16.7 std:Sthread Functions & std::this Namespace . . . 197
16.8 Task Based Concurrency - Part I . . . 198
16.9 Task Based Concurrency - Part II . . . 199
16.10 Launch Policies . . . 200
16.11 std::future Wait Functions . . . 200
16.12 std::promise . . . 203
16.13 Handling Exceptions . . . 205

17 C++17 Core Language Features **208**
17.1 Deprecated & Removed Features . . . 208
17.2 Attributes . . . 208
17.3 Feature Test Macros . . . 209
17.4 If & switch with Initialization . . . 210
17.5 Inline Variables . . . 211
17.6 Nested Namespaces . . . 213
17.7 noexcept . . . 215
17.8 constexpr Lambda & Capture by *this . . . 215
17.9 Structured Bindings . . . 217
17.10 Expression Evaluation Order . . . 218
17.11 Mandatory Copy Elision - I . . . 220
17.12 Mandatory Copy Elision - II . . . 221

18 C++17 Template Features **223**
18.1 Class Template Argument Deduction (CTAD) . . . 223
18.2 Class Template Argument Deduction (CTAD) . . . 223
18.3 Folding Basics . . . 224

18.4 Fold Expressions - Unary Folds . . . 224
18.5 Fold Expressions - Binary Folds . . . 225
18.6 Fold Expressions - Recap . . . 227
18.7 Type Traits Suffixes . . . 227
18.8 If constexpr - I . . . 228
18.9 If constexpr - II . . . 229

19 C++17 Standard Library Component 230
19.1 std::optional -I . . . 230
19.2 std::optional - II . . . 231
19.3 std::optional - III . . . 232
19.4 std::variant - I . . . 233
19.5 std::variant - II . . . 233
19.6 std::variant - III std::visit . . . 235
19.7 std::any . . . 240
19.8 std::string_view - I . . . 242
19.9 std::string_view - II . . . 243
19.10Filesystem - path . . . 244
19.11Filesystem - directory_entry . . . 245
19.12Filesystem - Directory Functions . . . 246
19.13Filesystem - Permissions . . . 249
19.14Parallel Algorithm - I . . . 250

20 Used Compiler Flags 254

21 Org Mode Simulate keyboard input 255

22 Document Setup & Build Improvements 258
22.1 Build Speed: Latexmk Configuration . . . 258
22.2 Reducing xelatex Passes: hyperref PageLabels . . . 258
22.3 Figure Placement: `[H]` Instead of `[htbp]` . . . 258
22.4 Source Code Formatting: Untabify . . . 259
22.5 Named Box Environments (bclogo) . . . 259

23 C++ Library Comparision 261
23.1 Overview . . . 261
23.2 ATL (Active Template Library) . . . 261
23.3 WTL (Windows Template Library) . . . 261
23.4 Boost . . . 262
23.5 POCO (Portable Components) . . . 262
23.6 ACE (Adaptive Communication Environment) . . . 263
23.7 Quick Comparison . . . 264
23.8 Recommendation for Modern Development . . . 264

24 Practical Guide 265

1 Introduction

Standard

- ISO C++ STD 98
- ISO C++ STD 2011 second major version of C++
- ISO C++ STD 2014 minor version
- ISO C++ STD 2017 major version
- ISO C++ STD 2020 major version
- ISO C++ STD 2020 current version
- ISO C++ STD 2026 The next generation of the C++ standard.

Compilers & DEvelopment Environements

- **Visual:** Studio IDE for Microsoft Windows
- **Code:** Blocks open-source cross-platform IDE
- **MinGW:** a compiler toolchain for Microsoft Windows
- **Cevelop:** Eclipse CDT extension for
- **GCC GNU:** Compiler Collection free and opensource

Source Code

- Source Code for download on Github
- Course Slides: /home/christian/Daten/01-tutorial/ModernesC++/Resources/Modern+C+++-+Distribution.pptx

1.1 Required Software

You can choose any of the following compilers & Integrated Development Environments. In this training, we'll use Visual Studio 2015/2017/2019 Community edition. The same programs can be compiled without any changes in any **C++11/14/17** compliant compiler on any platform.

1.1.1 Windows

- Visual Studio 2015/2017/2019 https://www.visualstudio.com/vs/community/ NOTE: PREFER THE LATEST VERSION OF VISUAL STUDIO
- Code::Blocks - requires a separate compiler https://www.codeblocks.org/
- Cevelop - requires a separate compiler https://www.cevelop.com/
- Eclipse CDT - requires a separate compiler (same as Cevelop) https://eclipse.org/cdt/downloads.php
- MinGW http://mingw.org/
- Visual Studio Code https://code.visualstudio.com/

1.1.2 Linux (Ubuntu, CentOS, etc)

- GNU C++ Compiler (Install from command line e.g. sudo apt-get install g++)
- Cevelop https://www.cevelop.org/
- Eclipse CDT (same as Cevelop) https://eclipse.org/cdt/downloads.php

1.1.3 MacOS

- XCode (Install from Mac Appstore)
- Cevelop https://www.cevelop.com/
- Eclipse CDT (same as Cevelop) https://eclipse.org/cdt/downloads.php

2 Basic Language Facilities

2.1 What is C++

- imperative programming
- oop programming
- generic programming

Modern C++ >= std=C++11

- Move - Smart Pointers -> now need to use raw pointers
- Automatic type infernce
- Threading
- Lambda functions
- ....

2.2 First C++ Program

Listing 1: Hello World

```cpp
#include <iostream>

int main() {
    using namespace std;
    cout << "Hello " << "C++" << endl;

    return 0;
}
```

```
Hello C++
```

- endl is a so called **manipulator**

2.3 C++ Compilation Process

- **Preprocessing**
  - statements beginning with # are processed
  - include statements get replaced with file content
  - macros are expanded
- **Compilation**
  - code is checked for correct syntax
  - converted into object code
- **Linking**
  - the object code is linked with standard libraries
- **The output of linking is the executable file**

2.4 Primitive Types & Variables

Primitive Types

- Arithmetic & void
- Arithmetic -> integral & floating point
- Integral types: bool, char, wchar_t, char16_t, char_32_t, short, int, long
- Floating point: float, double
- void is a special type used with pointers & functions

Type Modifiers

- **modifiers:** singed, unsigned, short, long
- **qualifiers**: const, volatile, static
- **memory requirement:** may change with the platform (CPU)
- **range** minimum and maximum value a type can hold
  - **<climits>** e.g. *INT_MIN* and *INT_MAX*
  - **<cfloat>** e.g. *DBL_MIN* and *DBL_MAX*

Variable Declaration

- varibale declaration: name followed by type e.g. *int i* or *float x*
- variable is also called **identifier**
- Use **initializer** to initialize a variable
- Uninitialized variables contain **chunk value**

Listing 2: Uninitialized Variable

```
using namespace std;
int main() {
    int i;
    cout << i;
}
```

32548

In function »int main()«: Warnung: »i« ist nicht initialisiert, wird aber benutzt [-Wuninitialized] 12 | cout « i; Anmerkung: »i« wurde hier deklariert 11 | int i;

Listing 3: Initializing Variables

```
using namespace std;
int main() {
    // Scalar Types
    int i=0;
    cout << i;

    float f = 1.283f;
    double d = 521.23;

    //Vector Types
    int arr[5]; // uninitialized
    int arr[5] = {1,2,3,4};

    //uniform initialization
    int myscalar {0};
    int myvector[5] {1,2,3,4};
}
```

```
0
```

2.5 Basic Input/Output

Base Classes for Input and Output

- std::basic_ostream & std::basic_istream **base class** for input and output

Type Definitions for Convenience

- std::ostream & std::istream

Objects of ostream resp istream class

- std::cout & std::cin
- std:cout for **console** output trough *operator«*
- std::cin for **keyboard** input through *operator»*
- accessible trough *<iostream>*

Listing 4: Console I/O

```cpp
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World " << 45 << 8.2f << " " << endl;
    string name;
    int age {0};
    cin >> name; // cin is read from the shell redirection: :cmdline "<<< 'Christian\n63'" of the
    cin >> age;
    cout << "Your name is: " << name << endl;
    cout << "Your age is: " << age << endl;
    return 0;
}
```

Listing 5: getline() to read

```cpp
#include <iostream>
using namespace std;
int main() {
    char buff[512] {0};
    cin.getline(buff,64,'\n'); // read until newline is entered!
    cout << "Your name is: " << buff << endl;
    return 0;
}
```

2.6 Function I

- Basic building blocks of C++
- Help avoid writing repetitive logic/code in a program
- Same code can be reused in different parts of the program
- Makes the code modular as the program is divided can be divided into cohesive modules
- Reduces overall complexity of a program

Structure of a Function

<return type> <name> (<parameters>){<body>}

The return type along with the parameters is called the signature.

Listing 6: Function declaration, definition and invocation

```
#include <iostream>

int Add(int, int);                          // Prototype or declaration

int main() {
    int a {0}, b{0}, result {0};            // Variable definition
    std::cin >> a;  // 15
    std::cin >> b;  // 37
    result = Add(a, b);                     // Function invocation
    std::cout << result << std::endl;
    return 0;
}

int Add(int x, int y)                       // Function definition
{
    return x + y;
}
```

Declaration Introduces a name to the program.

2.7 Function II

Function Prototype and its Implementation

Functions should always be put in their own source files!

Listing 7: Structure of a CPP Project

```
  func2/math
        +-- inc
        |   +-- math.hpp
        +-- src
        |   +-- math.cpp
        |   +--main.cpp
        +--Makefile
```

Listing 8: Header File

```
#ifndef MATH_HPP
#define MATH_HPP

int Add(int x, int y);

#endif
```

Listing 9: Implementation File

```
int Add(int x, int y)
{
    return x + y;
}
```

Listing 10: Main CPP

```cpp
#include "../../math/inc/math.hpp"
#include <iostream>

int main()
{
    std::cout << Add(3, 5) << std::endl;
    return 0;
}
```

Listing 11: Makefile

```
CXX=g++
CXXFLAGS=-std=c++17 -Wall -Wextra -Iinc

SRC=src/main.cpp src/math.cpp

app: $(SRC)
	$(CXX) $(CXXFLAGS) $(SRC) -o app

run: app
	./app

clean:
	rm -f app
```

Listing 12: Run

```
make run
```

2.8 TODO Overview of Debugging

- ☐ Describe emacs gnu debugger session

2.9 Uniform Initilalization

- **C++98** provided different ways to initialize types
  - Scalar types can be initialized with = or *()*
  - Array types have to be initialized with *{}*
- **C++11** introduces uniform initialization
  - Use *{}* for all the types
- **Advantages**
  - It forces initialization
  - You can use direct initialization for array types
  - It prevents narrowing conversion
  - Uniform syntax for all types

Listing 13: Initialization before C++11

```cpp
#include <string>
int main() {
// primitive types
    int ai;                             // uninitialized
    int a2 = 0;                         // copy initialization
    int a3(5);                          // direct initialization


//user defined types
    std::string s1;                      // default constructor is invoced
                                         // and initializes s1
    std::string s2("C++");               // direct initialization

// array types
    char d1[8];                          // uninitialized
    char d2[8] = {'\n'};                 // copy initialization
    char d3[8] = {'a','b','c','d','\n'}; // aggregat initialization
                                         // (but also copy initialization)
    char d4[8] = {"abcd\n"};             // copy initialization
}
```

Listing 14: Initialization with C++ >= Std11

```cpp
#include <string>
int main() {
// primitive types
    int b1{};                            // Value initialization to default value
    //  int b1();                            // most vexing parse, i.e. b1() is a function
    int b3{5};                           // direct initialization

// array types
    char e1[8]{};                        // Value initialization to default value
    char e2[8]{"Hello"};                 // direct initialization

// heap based objects
    int *p1 = new int;                   // uninitialized
    int *p2 = new int{};                 // value initialization
    char *p3 = new char[8]{};            // value initialization
    char *p4 = new char[8]{"Hello"};     // direct initialization
}
```

Summary

1. Value initialization => T obj{};
2. Direct initialization => T obj{v};
3. Copy initialization => T obj = v;

Avoid using copy initalization for user defined objects with C++ >= Std11

Listing 15: Narrowing Conversion

```cpp
#include <string>
int main() {

// compiles without warning
    float f1 = 0;
    int i1 = f1;
```

```
7
8   // thorws compile time warning
9       float f2{};
10      int i2{f1};
11
12  }
```

2.10 Pointers

2.10.1 Pointer

- Points to another type
- Holds the memory address of another variable
- Used for **indirect access** to other variables
- **Need to be initilalized during declaration**
- Declared with * operator
  - *int * ptr;*
  - *int *p1, *p2, *p3, p4;* <= p4 is not a pointer

2.10.2 Address Of Operator

- & is the address of operator
- when applied to any variable, we gat the address of that variable (the location where it is stored in the memory)
  - *int x = 10;*
  - *cout « &x; ///prints the address of x*

Listing 16: Address of Operator

```
1   #include <iostream>
2   using namespace std;
3
4   int main() {
5
6       int x = 10;
7       cout << "Address of x: " << &x << endl;
8       int *ptr = &x;
9       cout << "Address of ptr: " << ptr << endl;
10
11      return 0;
12  }
```

2.10.3 Derefernce Operator

- To access a value at the address in the pointer, use the * operator
- Also called derefernce operator
- Allows indirect read or write operation on the variable through the pointer
  - *int x = 10;*
  - *int *ptr = &x;*
  - **ptr = 5; ///Assign 5 to the address of x*
  - *int y = *ptr; ///Read a value from address of x*

Listing 17: Dereference Operator

```cpp
#include <iostream>
using namespace std;

int main() {

    int x = 10;
    cout << "Value of x: " << x << endl;
    cout << "Address of x: " << &x << endl;
    int *ptr = &x;
    cout << "Address of ptr: " << ptr << endl;

    *ptr = 5;
    cout << "New value of x: " << x << endl;


    return 0;
}
```

2.10.4 Null Operator

- *NULL* is a macro in C & pre-C++11
- It is defined as 0 in most compilers
- Used to initialize pointer types
- **C++11** introduced a new type of null called *nullptr*
- This is type safe and better then NULL macro
- Always use *nullptr* to initialize a pointer, instead of NULL macro

Listing 18: Null Operator

```cpp
#include <iostream>
using namespace std;

int main() {

    int x = 10;
    cout << "Value of x: " << x << endl;
    cout << "Address of x: " << &x << endl;
    int *ptr = nullptr;
    cout << "Address of ptr: " << ptr << endl;

    // do not read from a nullptr
    // cout << ptr << endl;

    // do not write into a nullptr
    // *ptr = 5;

    return 0;
}
```

2.11 Assignment 1

Implement the following functions using pointer arguments only

Listing 19: Assignment 1

```cpp
#include <iostream>
// Add two numbers and return the sum
int Add(int *a, int *b) {
    return *a + *b;
}

// Add two numbers and return the sum through the third pointer argument
void AddVal(int *a, int *b, int *result) {
    *result = *a + *b;
}

// Swap the value of two integers
void Swap(int *a, int *b) {
    // swap the values not pointer
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Generate the factorial of a number and return that through the second pointer argument
void Factorial(int *a, int *result) {
    *result = 1;
    for(int i=2; i<=*a; ++i) {
        *result *= i;
    }
}

using namespace std;
int main() {
    int x{5}, y{7}, *a{&x}, *b{&y};
    cout << "Add -> 5+7: " << Add(a,b) << endl;

    int res{0}, *result{&res};
    AddVal(a,b,result);
    cout << "AddVal -> 5+7: " << *result << endl;

    Swap(a,b);
    cout << "Values: a=" << *a << ", b=" << *b << endl;

    Factorial(a,result);
    cout << "Factorial -> 7!: " << *result << endl;

    return 0;
}
```

2.12 References

2.12.1 Reference Type

- Define an alternative name for a variable (alias)
- It is created within the & operator during **declaration**
- Always needs an initilaizer (called **referent**)
- The referent should be a variable
- It is **bound** to its referent
- It can be used to modify a variable indirectly (like a pointer)
- A reference is **NOT** a new variable; it is just another name

Listing 20: Reference Type

```cpp
#include <iostream>

using namespace std;
int main() {
//Referent
    int x = 10;
//Reference
    int &ref =x;
    int y = 20;
    //ref = y;

    x=6;
    cout <<"x: " << &x << endl;
    cout << "ref: " << &ref << endl;


    return 0;
}
```

Reference	Pointer
Always need an initializer	Initializer is optional
Initializer should be lvalue	Initializer need not be lvalue
Cannot be nullptr	Can be nullptr
Bound to its referent	Can point to other variables
No storage required	Has its own storage
Dererference is not required	Dereference operator is requried

2.13 Assignment 2

Implement the following functions

Listing 21: Assignment 2

```cpp
#include <iostream>

//Add two numbers and return the result through a reference parameter
void Add(int a, int b, int &result) {
    result = a+b;
}

//Find factorial of a number and return that through a reference parameter
void Factorial(int a, int &result) {
    result = 1;
    for(int i=2; i<=a; ++i) {
    result *= i;
    }
}

//Swap two numbers through reference arguments
void Swap(int &a, int &b) {
    // swap the values not pointer
    int temp = a;
    a = b;
    b = temp;
}

using namespace std;
```

```
25 int main() {
26     int x{5}, y{7}, result{0};
27
28     Add(x,y,result);
29     cout << "Add -> 5+7: " << result << endl;
30
31     Factorial(x,result);
32     cout << "Factorial -> 5!: " << result << endl;
33
34     Swap(x,y);
35     cout << "Values: x=" << x << ", y=" << y << endl;
36
37     return 0;
38 }
```

2.14 References vs. Pointers

Listing 22: Argument Passing by Value

```
1  #include <iostream>
2
3  //Swap two numbers through value arguments
4  void Swap(int x, int y) {
5      // swap the values not pointer
6      int temp = x;
7      x = y;
8      y = temp;
9  }
10
11 using namespace std;
12 int main() {
13     int a = 5;
14     int b = 7;
15
16     Swap(a,b);
17     cout << "Values: a=" << a << endl;
18     cout << "Values: b=" << b << endl;
19
20     return 0;
21 }
```

Listing 23: Argument Passing by Address

```
1  #include <iostream>
2
3  //Swap two numbers through address arguments
4  void Swap(int *x, int *y) {
5      // swap the values not pointer
6      int temp = *x;
7      *x = *y;
8      *y = temp;
9  }
10
11
12 using namespace std;
13 int main() {
14     int a = 5;
15     int b = 7;
16
17     Swap(&a,&b);
18     cout << "Values: a=" << a << endl;
```

```
19      cout << "Values: b=" << b << endl;
20
21      return 0;
22  }
```

Listing 24: Argument Passing by Reference

```
1   #include <iostream>
2
3   //Swap two numbers through reference arguments
4   void Swap(int &x, int &y) {
5       // swap the values not pointer
6       int temp = x;
7       x = y;
8       y = temp;
9   }
10
11  using namespace std;
12  int main() {
13      int a = 5;
14      int b = 7;
15
16      Swap(a,b);
17      cout << "Values: a=" << a << endl;
18      cout << "Values: b=" << b << endl;
19
20      return 0;
21  }
```

Listing 25: Pointer vs References - nullptr-

```
1   #include <iostream>
2
3   using namespace std;
4
5   //Prints a number: argument passing as address
6   void printP(int *ptr) {
7       if (nullptr != ptr)           // Mandatory: check for nullptr
8           cout << *ptr << endl;
9       else
10          cout << "nullptr passed" << endl;
11  }
12
13  //Prints a number: argument passing as Reference
14  void printR(int &ref) {
15      cout << ref << endl;
16  }
17
18  int main() {
19      int a = 5;
20      printP(&a);
21      printP(nullptr);        // runtime error if not checked
22      printR(a);
23      // printR(nullptr)      // compile time error
24
25      return 0;
26  }
```

Reference vs. Pointer

- Pointers need to be checked for nullptr
- Compiler guarantees that a reference is valid
- Code is simpler to read with references

2.15 The const Qualifier

- Creates a variable that is constant
- Value cannot be modified
- Attempt to modify will cause compilation error
- Qulified to a declaration, but always needs an initilaizer
- Replaces C macros
  - not type safe
  - not have a scope
  - lead to subtle bugs which are difficult to find
- Commonly used with references

Declaration of a Constant

const <type> <variable> {initializer} const float PI {3.141f}

Listing 26: Magic Numbers

```
#include <iostream>

using namespace std;

int main() {
    float radius = 0;
    cin >> radius;
    float area = 3.14159f * radius * radius;
    float circumference = 3.14f * 2 * radius;

    cout << "Area is: " << area << endl;
    cout << "Circumference is : " << circumference << endl;

    return 0;
}
```

Listing 27: No Magic Numbers

```
#include <iostream>

using namespace std;

int main() {
    float radius = 0;
    cin >> radius;
    const float PI = 3.141f;                // use captial letters to indicate const
```

```
9       float area = PI * radius * radius;
10      // PI = 5;                              // compile time error
11      float circumference = PI * 2 * radius;
12
13      cout << "Area is: " << area << endl;
14      cout << "Circumference is : " << circumference << endl;
15
16      return 0;
17  }
```

2.16 cons Qualifier & Compound Types

Read from right to left

const int *ptr = ...;
*ptr is a pointer to an integer of type constant.
This means *ptr is constant ptr is not.

Listing 28: const Qualfier / Compound Types

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main() {
6
7       const int CHUNK_SIZE = 512;
8       const int *ptr = &CHUNK_SIZE;        // if creating a pointer to a constant
9                                            // than the poiner should also be a constant
10      // *ptr = 1;                         // not allowed
11
12      int x = 10;
13      ptr = &x;                            // assign a new address to the pointer is allowed
14                                           // By declaring the pointer as constant the constantness
15                                           // of the original variable is preserved
16      // *ptr = 1;                         // not allowed
17
18      x = 1;                               // strangely this works
19      cout << "x=" << x <<", *ptr=" << *ptr << endl;
20
21     (void)*ptr;
22      return 0;
23  }
```

Constant Pointer

Constant pointers/references are usfull as arguments to functions where you don't want to modify the value of the variable that is passed into the function.

Listing 29: const pointer

```
1   #include <iostream>
2
3   using namespace std;
4
5   void Print1(const int *ptr) {
6       cout << "const *ptr: " << *ptr << endl;
```

```
    // *ptr = 70;                       // not allowed
}

void Print2(const int &ref) {
    cout << "const &ref: " << ref << endl;
    // *ptr = 70;                       // not allowed
}

int main() {

    int x = 10;
    Print1(&x);
    Print2(x);
    cout << "main->x=" << x << endl;

    return 0;
}
```

2.17 Assignment 3

Write the following code & do as instructed in the comments. Build and observe.

Listing 30: Assignment 3

```
#include <iostream>

using namespace std;

int main() {

    // 1.) Try to modify x1 & x2 and see the compilation output
    int x = 5 ;
    const int MAX = 12 ;
    int &ref_x1 = x ;
    const int &ref_x2 = x ;

    ref_x1 = 100;
    // ref_x2 = 200;     // compile time error

    // 2.) Try to modify the pointer (e.g. ptr1) and the pointee value (e.g. *ptr1)
    const int *ptr1 = &x ;          // const value
    int *const ptr2 = &x ;          // const pointer
    const int * const ptr3 = &x ;   // const value and const pointer

    int y = 100;
    ptr1 = &y;       // modify pointer
    // *ptr1 = y;    // modify pointee, compile time error

    // ptr2 = &y;    // modify pointer, compile time error
    *ptr2 = y;    // modify pointee

    // ptr3 = &y;    // modify pointer, compile time error
    // *ptr3 = y;    // modify pointee, compile time error

    // 3.) Find which declarations are valid. If invalid, correct the declaration
    const int *ptr4 = &MAX ;
    // int *ptr5 = &MAX ; // zuweisung const value
    const int *ptr5 = &MAX ;

    const int &r1 = ref_x1 ;
    // int &r2 = ref_x2 ;
    const int &r2 = ref_x2 ;
```

```
39
40      const int *&p_ref1 = ptr1 ;
41      int * const &p_ref2 = ptr2 ;
42
43      //avoid warning
44      cout << *ptr1   << " " << *ptr2   << " "
45           << *ptr3   << " " << *ptr4   << " "
46           << *ptr5   << " " << r1      << " "
47           << r2      << " " << *p_ref1 << " "
48           << *p_ref2 << endl;
49      return 0;
50  }
```

2.18 Automatic Type Intference (C++11)

- Indicates storage class of a variable in C & pre-C++11
- Remnant of B language (which largely influenced C)
- Meaning has been changed in C++11
- Allows the compiler to automatically infer the type from the initializer
- The initializer is important
  - *auto <identifier> = <initializer>*
- <initializer could be an expression that returns some value e.g. a literal, mathematical expression or a function call that returns a value

Listing 31: auto Keyword

```
1   #include <iostream>
2
3   using namespace std;
4
5   int Sum(int x, int y) {
6       return x + y;
7   }
8
9   int main() {
10
11      // usage of auto keyword
12      auto i = 10;
13      auto j = 5;
14      auto sum = i + 4.3f;
15      auto result = Sum(i, j);
16
17      // auto and static keyworkd
18      static auto y = 2;
19
20      // auto and const keyworkd
21      const int x = 10;
22      auto var1 = x;          // var1 not a const variable
23      const auto var2 = x;    // var2 is a const variable
24
25      // auto and reference
26      auto &var3 = x;         // var3 is a constant reference
27
28      // auto and pointers
29      auto ptr = &x;          // ptr is a pointer to const int (const int*)
30
```

```
31      // avoids warning
32      cout << sum    << ", " << result << ", "
33           << y      << ", " << var1   << ", "
34           << var2   << ", " << var3   << ", "
35           << *ptr   << ", "
36           << endl;
37
38      return 0;
39
40  }
```

auto Keyword and Qualifiers

When using auto Keyworkd, the qualifiers are discarded for variables but not for pointers and references.

Do not use the * for pointer declaration with auto Keyword.

Listing 32: auto Keyword and initializer list

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main() {
6
7       // Initializer list note = is required here!
8       auto list = { 1,2,3,4};  // type is initializer list
9
10      // print initializer list
11      for (auto i : list)
12          cout << i << ", ";
13      cout << endl;
14
15      return 0;
16
17  }
```

2.19 Range-Based For Loop 1 (C++11)

2.19.1 Standard For-Loop

Listing 33: standard for Loop

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main() {
6
7       int arr[]{1,2,3,4,5};
8
9       for(int i=0; i<5; i++)
10          cout << arr[i] << ", ";
11
12      return 0;
13  }
```

2.19.2 Range-based For-Loop

- Allows iteration over arrays and containers

- Doesn't need index variable
- Each iteration returns an element
- Can be used with any object that behaves like a **range**

Syntax

for(variable declaration : range) {
statement;
}

Listing 34: Range-Based For Loop

```cpp
#include <iostream>

using namespace std;

int main() {

    int arr[]{1,2,3,4,5};

    for(const auto& element : arr) // use const to avoid modifying arr[]
        cout << element << ", ";

    return 0;
}
```

2.19.3 For vs Range-Based For

For	Range-Based For
Index base iteration	Does not use index to iterate
Requires end condition	End condition is provided by the range
Index variabel needs to be in-, decremented	No need to in-, decrement
Error prone e.g. wrong end conditicion, over-, underflow, incorrect iteration	Lesses chances of errors
More control over iteration	No control over iteration
Use for finer control	Use with ranges

2.20 Range-Based For Loop 2 (C++11)

Listing 35: Internal Organization of Range-Based for loop

```cpp
#include <iostream>

using namespace std;

int main() {

    int arr[]{1,2,3,4,5};

    int *beg = &arr[0]; // points to the beginning of arr
    int *end = &arr[5]; // points to one past the end of arr

    while(beg != end) {
        cout << *beg << ", ";
```

```
        beg++;
    }

    return 0;
}
```

Listing 36: C++11: Internal Organization of Range-Based for loop

```
#include <iostream>

using namespace std;

int main() {

    int arr[]{1,2,3,4,5};

    auto &&range = arr;
    auto itrBeg = begin(range);
    auto itrEnd = end(range);

    for(;itrBeg != itrEnd; ++itrBeg)
    {
        auto v = *itrBeg;
        cout << v << ", ";
    }

    return 0;
}
```

2.21 Function Overloading

Listing 37: Function Overloading

```
#include <iostream>

using namespace std;

int    Add(int a, int b)       {return a+b;}
double Add(double a, double b) {return a+b;}

int main() {

    int result1 = Add(5,3);          // compiler checks the arguments
    cout << result1 << endl;

    double result2 = Add(3.1,6.2);  // compiler checks the arguments
    cout << result2 << endl;

    return 0;
}
```

2.21.1 Function Overloading

- Two or more functions declared with the same name
  - Arguments should differ in type and/or number
  - For pointers & references arguments, qualifiers participate in overlaod
  - Return type is ignored
  - For member functions, qualifiers participate in overload

- Different implementation of the same behaviour
- The correct implementation is chosen based on arguments
- This is resolved at compile-time
  - **static polymorphism**
- Convenience for the caller

2.21.2 Name Mangeling

- Unique names generated by the compiler for functions
- Allows linker to link the call with the correct overloaded function
- Name Mangeling algorithm varies from compiler to compiler
- Depends on the type & number of function arguments
- Consequently, C++ functions are not callable from C code

2.21.3 Compiler directive extern "C"

- Compiler directive applied on global functions and variables
- Suppresses name mangeling of the type on which it is applied
- Can be applied only to one function in a set of overloaded functions
- Allows C++ functions to be called from C or other languages

Listing 38: extern "C"

```
// Apply on function declartion & definition

// in .h file
extern "c" <function declaration>

// in .cpp file
extern "c" <function definition>{
}
```

2.21.4 Map File to inspect Name Mangeling

Listing 39: Generate Map File with Linke Option: Wl,-Map=fomap.map

```
#include <iostream>

using namespace std;

int    Add(int a, int b)       {return a+b;}
double Add(double a, double b) {return a+b;}

extern "C" void print(int x)    {cout << x << endl;}
void print(double x) {cout << x << endl;}

int main() {

    int result1 = Add(5,3);          // compiler checks the arguments
    cout << result1 << endl;

    double result2 = Add(3.1,6.2);  // compiler checks the arguments
```

```
17      cout << result2 << endl;
18
19      return 0;
20  }
```

```
8
9.3
```

Usage of extern "C"

extern "C" must be applied at the function declaration as well as at the function definition (link error)

if the header file is included into the definition file, extern "C" can be omitted within the definition.

In the declaration as well as in the definition file extern "C" can be used with braces:
extern "C" { func1() func2() }

- Allows some or all function arguments to have a default value
- It becomes optional to pass values to those arguments
  - Compiler automatically assigns default value if no explicit value is provided
  - Explicit value is preferred over default value
  - Default arguments should begin from the right side in the list of function arguments
  - Simplifies the invocation for the caller

Listing 40: Default Arguments

```
1   #include <iostream>
2
3   using namespace std;
4
5   void CreateWindow(const char* title, int x, int y, int width, int height);
6
7   void CreateWindow(
8       const char* title,
9       int x,
10      int y = -1,
11      int width = - 1,
12      int height = -1) {
13
14      cout << "Title: " << title << endl;
15      cout << "x-coordinate: " << x << endl;
16      cout << "y-coordinate: " << y << endl;
17      cout << "width: " << width << endl;
18      cout << "height: " << height << endl;
19
20  }
21
22  int main() {
23
24      CreateWindow("No Default", 100, 200, 1024, 720);
25      cout << endl;
26      CreateWindow("Default",1000);
27
28      return 0;
29  }
```



Pass the values starting from the left side, no gaps allowd between default argument list.

2.22 Inline Functions

- A function that is marked with *inline* keyword
- Such functions are defined in a header file
- Requests the compiler to replace the call with the function body
- The overhead of the function call is avoided
  - Stach memory for arguments not required
  - No need to save the return address
- May improve the performance of the code

Listing 41: Inline Functions

```
#include <iostream>

using namespace std;

// inline int Square(int x) {return x*x;}
#define Square(x) x*x

int main() {

    int val = 5;
    int result = Square(val + 1);

    cout << result << endl;
    return 0;
}
```



To Remember

- Only a **request** to the compiler
- Certain functions may not be inlined
  - large functions
  - functions having too many conditional statements
  - recursive functions
  - functionss invoked through pointers
  - etc
- Different compilers have different rules
- Modern compilers may automatically inline even no-inline functions
- Excessive inlining may increase binary size

2.22.1 Macros vs. Inline Functions

Macro	Inline Function
Works through text substition	The call is replaced with body
Error prone due to substition nature	Safe to use as it has function semantics
Does not have an address	Has an address
Difficult to use with multiple lines of code	Can have multiple lines of code
Cannot be used for member functions	Class member functions can be inline

2.23 Function Pointers

- Pointer that holds the address of the function
- The type is same as the **signature** of the function (return type & arguments)
- Can be used to inderectly invoke the function even if the function name is not known
- Used by algorithm and classes for customization



Syntax

<ret> (*fnptr)(args) = &Function; int (*PtrAdd)(int,int) = &Add;

Listing 42: Function Pointer

```
#include <iostream>

using namespace std;

void Print(int count, char ch) {

    for(int i = 0; i < count; ++i) {
^^Icout << ch << ' ';
    }
    cout << endl;
}

void EndMessage() {
    cout << "Enf of program" << endl;
}

int main() {

    atexit(EndMessage);

    void(*pfn) (int, char) = Print;
    (*pfn)(8, '#');
    pfn(16,'#');


    return 0;
}
```

2.24 Namespace

- Named declarative region used for declaring types
- The types are not visible outside the namespace

- Standard library is in *std* namespace
- Prevents name clashes
- Helps modularize code
  - *namespce <name> { (namespace, class, structure, function, variable, etc) }*

2.24.1 Namespace Access

- Types inside a namespace have a scope
- Cannot be accessed outside the namespace
- Either open the namespace or the type
  - use the global using declarative and open the entire namespace
    - *using namespace std;*
  - use the using declerative and open a specific type
    - *using std::cout*
  - using the full qualified name
    - *std::cout « "C++" « std::endl;*

Listing 43: Namespace

```
#include <iostream>

namespace Avg {
    float Calculate(float x, float y) { return (x*y)/2;}
}

namespace Basic {
    float Calculate(float x, float y) { return (x*y);}
}

int main() {

    Avg::Calculate(3.2f, 7.8f);
    Basic::Calculate(3.2f, 7.8f);
    return 0;
}
```

3 Memory Management - Part I

3.1 Dynamic Memory Allocation Part I (malloc function)

- C/C++ programs are provided with different types of memory areas
  - **stack** - allocated automatically for local variables
  - **data section** - allocated for global and static data
  - **heap** - allocated at runtime
- All the memory is taken from the process address space
- C/C++ programs provide support for memory allocation at runtime (also called dynmic memory)
- Allocations on the heap have to be managed by the **programmer**
- Stack and data section allocations are managed by the **runtime**

3.1.1 Dynamic Memory Allocation in C

- C provides various functions for allocating memory from the heap
  - *malloc*: allocate raw memory on the heap
  - *calloc*: allocates memory on the heap and initializes it to zero
  - *realloc*: allocates larger chunk of memory for an existing allocation
  - *free*: deallocates/releases the memory allocated through the above functions

Listing 44: malloc/calloc with gcc

```
#include <stdio.h>
#include <stdlib.h>  // declaration of memory allocation function

int main() {

    // single element not initialzied memory
    int *p1 = malloc(sizeof(int));      // single element
    if(p1 == NULL) {
        printf("Failed to allocate memory\n");
        return -1;
    }
    *p1 = 5;
    printf("*p1=%d\n", *p1);
    free(p1);       // no memory leak
    p1 = NULL;      // no dangling pointer

    // array of 5 ints not initilized
    int *a1 = malloc(5*sizeof(int));  // 5 elements
    if(a1 == NULL) {
        printf("Failed to allocate memory\n");
        return -1;
    }
    a1[0] = 10;
    a1[1] = 20;
    printf("a1[0]=%d, a1[1]=%d, a1[2]\n", a1[0], a1[1]);
    free(a1);     // no memory leak
    a1 = NULL;    // no dangling pointer


    // array of 5 ints initialized to 0
    int *a2 = calloc(5, sizeof(int));  // 5 elements
```

```
32      if(a2 == NULL) {
33          printf("Failed to allocate memory\n");
34          return -1;
35      }
36      a2[0] = 10;
37      a2[1] = 20;
38      printf("a2[0]=%d, a2[1]=%d, a2[3]=%d, a2[3]=%d, a2[4]=%d\n", \
39             a2[0], a2[1], a2[2], a2[3], a2[4]);
40      free(a2);      // no memory leak
41      a2 = NULL;     // no dangling pointer
42
43
44      return 0;
45  }
```

```
*p1: 5
a1[0]: 10, a1[1]: 20, a1[2]: 0, a1[3]: 0, a1[4]: 0
a2[0]: 10, a2[1]: 20, a2[3]: 0, a2[3]: 0, a2[4]: 0
```

Important

When allocation memory with *malloc*, *calloc* or *realloc* then:

- check the return type is not NULL
- free the memory when not used anymore - avoid **memory leak**
- set pointer to NULL after freeing - otherwise we have a **dangling** pointer

Listing 45: malloc with g++

```
1   #include <stdio.h>
2   #include <stdlib.h>  // declaration of memory allocation function
3
4   int main() {
5
6       int *p = (int*) malloc(sizeof(int));
7       if(p == NULL) {
8           printf("Failed to allocate memory\n");
9           return -1;
10      }
11
12      *p = 5;
13      printf("%d", *p);
14      free(p);      // no dangling pointer
15      p = NULL;     // no dangling pointer
16
17      return 0;
18  }
```

gcc vs c++

malloc returns a void pointer and if compiling this code with a c++ compiler you have to explicitly perform a cast between void pointer to the target pointer type.

3.2 Dynamic Memory Allocation Part II (new operator)

- C++ provides two operators for dynamic memory allocation
- *new*: allocated memory on the heap
- *delete*: deallocates memory
- Memory that is allocated through *new* has to be released with *delete*

Listing 46: new operator

```
#include <iostream>

using namespace std;

int main() {

    int *p = new int(7);
    *p = 5;
    cout << "address=" << p << " value="<< *p << endl;
    delete p;
    p = nullptr;   // no dangling pointer

    return 0;
}
```



Syntax

<type> *<variable> = new <type>(optiona args); delete <variable>;
int *p = new int(value); delete p;

malloc	new
Function	Operator
Requires size during allocation	Size is ascertained from the type
Cannot initialize memory	Can initialize memory
Cannot call constructors	Can call constructors
Retruns void pointer that needs to be type casted	Return correct type of pointer
Cannot be customized	Can be customized through overloading
malloc, calloc, realloc	has different formss
Return NULL on failure	Throws exception on failure

3.3 Dynamic Memory Allocation Part III (new[] operator)

3.3.1 new For Arrays

- Another form of new
- Used for allocating dynamic arrays
- Written as new[]
- Susbequentyl, use delete[] for releasing the memory



Syntax

*<type> *<variable> = new <type>[size]; delete [] <variable>;*
*int *p = new int[5]; //* Allocate memory for 5 integers *delete []p; //*note the usage of

Listing 47: new for Arrays

```
#include <string.h>
#include <iostream>

void NewArrays() {

    using namespace std;

    int *p = new int[5]{0,1,2,3,4} ;
    for(int i=0; i<5; i++) {
        cout << p[i] << ", ";
    }
    cout << endl;
    delete []p;
}

void Strings() {
    using namespace std;

    char *p = new char[4]; // allocate 1 charachter for null terminate char.
    strcpy(p, "C++");
    cout << p << endl;
    delete []p;
}

int main() {
    NewArrays();
    Strings();

    return 0;
}
```

3.4 Dynamic Memory Allocation Part IV (2D arrays)

```
#include <string.h>
#include <iostream>

void TwoDArray() {
    using namespace std;

    int *p1 = new int[3] {1,2,3};  // 1. row
    int *p2 = new int[3] {4,5,6};  // 2. row

    int **pData = new int *[2];    // array of pointers

    pData[0] = p1;
    pData[1] = p2;

    for(int i=0; i<2; i++) {
        for(int j=0; j<3; j++) {

            cout << pData[i][j] << ", ";
        }
        cout << endl; // end of columns
```

```
21      }
22      cout << endl;  // end of rows
23
24      // delete the memory poiners in the same sequence as the have been allocated:
25      // 1. delete the row pointers
26      // 2. delete the memory of the array of pointers
27      delete[]p1; // delete []pdata[0]
28      delete[]p2; // delete []pdata[1]
29
30      delete [] pData;
31  }
32
33
34  int main() {
35      using namespace std;
36      // first index represents the row
37      // second index represents the column
38      int data[2][3] = {
39          1,2,3,   // first row
40          4,5,6    // second row
41      }; //1,2,3,4,5,6
42
43      for(int i=0; i<2; i++) {
44          for(int j=0; j<3; j++) {
45
46              cout << data[i][j] << ", ";
47          }
48          cout << endl; // end of columns
49      }
50      cout << endl;  // end of rows
51
52      TwoDArray();
53
54      return 0;
55  }
```



Remember

The number of delete calls should match the numbers of new calls!

4 Classes & Objects

4.1 Object Oriented Programming Basics

- Uses objects as fundamental building blocks, rahter algorithms
- Program is created as a collection of objects
- Every object is an instance of some class
- Classes are united via inheritance relationship
- Simulate interactions of objects in real-world systems
- Allows representation of objects in problem domain as software objects
- Decreases complexity of software systems
- Make code reusable, extensible and maintanable

4.1.1 Object Model

- Basic principles that help us write OO programs
  - Abstraction
  - Encapsulation
  - Inheritance
  - Polymorphism

4.1.2 Abstraction (OOSC: Modularity/Decomposability)

- Abstraction focuses on important & necessary details
- Unwanted features are left out
- e.g. name of a person without other details (age, weight, etc)
- Helps focus on important charachteristics
- Used to represent real-life objects in software, but without the association complexity
- Represented through a **class, *struct**, **interface**, **union** or **enum**

4.1.3 Encapsulation (OOSC: Information Hiding)

- Next step after abstraction
- Hides the implementation details of a class
- The class provides behaviour without revealing the implementation
- Objects of such classes are easy to use
- The internal implementation can be changed without the users knowlege
- implementation through access modifiers in OO langunage
- **Encapsulation** is the mechanism of bundling data and methods into a single unit (like a class).
- **Information hiding** is the design goal of concealing the internal implementation details of an object so that clients can only interact with it through a controlled interfac

4.1.4 Inheritance

- Represnts a hirachy of classes
- The classes are relatate through **"is-" relationship**
- The relation is due to same behaviour of classes
- **e.g. a dog is an animal**
- The behaviour % its implementation is inherited by the children from the parent
- The child classes may then reuse the behaviour with the same implementation or provide a different implementation
- Promotes reuse & extensibility

4.1.5 Composition

- Signifies relationship between objects
- Represented as **"has-a or "part-of" relationship**
- Promotes reuse of objects
- **e.g. car has en engine**

4.1.6 Polymorphism

- Means different forms
- Represents common behaviour with different implementations
- Response will be different for each object, either based on its class or the arguments
- e.g. car, cycle, person, etc. can move differently
- Implemented through function overloading, templates & virtual functions
- Used in conjunction with inheritance & composition
- Promotes reuse, scalability & extensibility

4.2 Class

- Blueprint/template/recipe
- Represents an abstraction
- Every object is instantiated
- Instance of a class is an object
- Can have multiple instances
- Objects are independent



Syntax

class <name> { //Members are private by default
<modifiers>: <member variables> <member functions>
};

Listing 48: Sequence Diagram with Relevant Errors

Class diagram: projects/section-04/class/diagram/car-hirachie.png

Listing 49: Car Class Declaration car1

```

class Car {

private:
    float fuel;
    float speed;
    int passengers;

public:
    void FillFuel(float amount);
    void Accelerate();
    void Brake();
    void AddPassengers(int count);
    void Dashbord();
};
```

Listing 50: Car Class Definition car1

```
#include "car.h"
#include "iostream"

void Car::FillFuel(float amount) {
    fuel = amount;
}

void Car::Accelerate() {
    speed++;
    fuel -= 0.5f;
}

void Car::Brake() {
    speed = 0;
}

void Car::AddPassengers(int count) {
    passengers = count;
}

void Car::Dashbord() {

    std::cout << "Fuel: " << fuel << std::endl;
    std::cout << "Speed: " << speed << std::endl;
    std::cout << "Passengers: " << passengers << std::endl;
}
```

Listing 51: Main CPP - car1 Client of Car Class

```
#include "car.h"

int main() {

    Car car;
    car.FillFuel(6);

```

```
8       for (int i=0; i<6; i++)
9       car.Accelerate();
10
11      car.Dashbord();
12
13      return 0;
14
15  }
16
```

Listing 52: Makefile car1

```
1   CXX=g++
2   CXXFLAGS=-std=c++17 -Wall -Wextra -Iinc
3
4   SRC=src/main.cpp src/car.cpp
5
6   app: $(SRC)
7           $(CXX) $(CXXFLAGS) $(SRC) -o app
8
9   run: app
10          ./app
11
12  clean:
13          rm -f app
```

Listing 53: Run car1

```
1   make run
```

```
g++ -std=c++17 -Wall -Wextra -Iinc src/main.cpp src/car.cpp -o app
./app
Fuel: 3
Speed: 6
Passengers: 998156647
```

Uninitalized Variables

Passengers shows an unpredictable value because it was not initialized.

4.3 Constructor & Destructor

- Invoked automatically during instantiation
- Used for initialization
- Doe's not have any return type
- Can be overloaded
  - Default
  - Paramterized
  - Copy
  - Delegation
  - Inheriting

4.3.1 Default Constructor

- Constructor with no arguments
  - *Car c;* invokes default constructor
- Automatically sythsized by the compiler
  - if no other user-defined constructor exist

4.3.2 Parameterized Constructor

- Constructor that accepts one or more arguments
- Used to initialize the object with user-defined values
- Never synthsized by the compiler
- Blocks auto generation of default constructor
-

4.3.3 Destructor

- Function that is invoked automatically when an object is destroyed
- Used for releasing resources that may have been allocated in the constructor
- Cannot be overloaded
- No arguments
- Compiler will sythesize one

4.3.4 Example

Listing 54: Car Class Declaration car2

```
1
2  class Car {
3
4  private:
5      float fuel;
6      float speed;
7      int passengers;
8
9  public:
10     Car();       // default constructor
11     Car(float);  // Paramterized constructor
12     ~Car();      // destructor
13     void FillFuel(float amount);
14     void Accelerate();
15     void Brake();
16     void AddPassengers(int count);
17     void Dashbord();
18 };
```

Listing 55: Car Class Definition car2

```
1  #include "car.h"
2  #include "iostream"
3
4  // default constructor
5  Car::Car() {
```

```
    fuel = 0;
    speed = 0;
    passengers = 0;

    std::cout << "Car()" << std::endl;
}

// Parameterized constructor
Car::Car(float amount) {
    fuel = amount;
    speed = 0;
    passengers = 0;

    std::cout << "Car(" << amount << ")" << std::endl;
}

// destructor
Car::~Car() {
    std::cout << "~Car()" << std::endl;
}
```

Listing 56: Car Class Definition car2

```
void Car::FillFuel(float amount) {
    fuel = amount;
}

void Car::Accelerate() {
    speed++;
    fuel -= 0.5f;
}

void Car::Brake() {
    speed = 0;
}

void Car::AddPassengers(int count) {
    passengers = count;
}

void Car::Dashbord() {

    std::cout << "Fuel: " << fuel << std::endl;
    std::cout << "Speed: " << speed << std::endl;
    std::cout << "Passenger: " << passengers << std::endl;
}
```

Listing 57: Main CPP - car2 Client of Car Class

```
#include "car.h"

int main() {

    Car c;
    c.FillFuel(6);

    Car b(5);

    for (int i=0; i<6; i++) {
        c.Accelerate();
```

```
12          b.Accelerate();
13      }
14
15      c.Dashbord();
16      b.Dashbord();
17
18      return 0;
19
20  }
```

Listing 58: Makefile car2

```
1   CXX=g++
2   CXXFLAGS=-std=c++17 -Wall -Wextra -Iinc
3
4   SRC=src/main.cpp src/car.cpp
5
6   app: $(SRC)
7           $(CXX) $(CXXFLAGS) $(SRC) -o app
8
9   run: app
10          ./app
11
12  clean:
13          rm -f app
```

Listing 59: Run car2

```
1   make run
```

```
g++ -std=c++17 -Wall -Wextra -Iinc src/main.cpp src/car.cpp -o app
./app
Car()
Car(5)
Fuel: 3
Speed: 6
Passenger: 0
Fuel: 2
Speed: 6
Passenger: 0
~Car()
~Car()
```

4.4 Structure

- Create user-defined type through keyword *struct*
- Similar to a class
- Default acces is *public*
- Frequently used
  - to repesent simple abstract types such as point, vector3D, etc.
  - for implementing **function Objects**, used as *callbacs"k in std template library

Listing 60: Struct Point

```cpp
#include <iostream>

struct Point {
    int x;
    int y;
};

void DrawLine(Point start, Point end) {

    using namespace std;

    cout << __func__ <<"()"<< endl;
    cout << " - Start Point:" << " X=" << start.x << " Y=" << start.x << endl;
    cout << " - End Point:  " << " X=" << end.x   << " Y=" << end.x << endl;
}

int main() {

    Point start;
    start.x = 2;
    start.y = 3;

    Point end;
    end.x = 2;
    end.y = 3;

    DrawLine(start, end);

    return 0;
}
```

4.5 Non-Static Data Member initializers (C++11)

- Convenient way of initializing class members with values during declaration

Syntax

class <name> { //Members are private by default
<type1> <var1> {<initializer>}; <type2> <var2> {<initializer>};
};

- Ensures the members are initialized with valid values
- Can be used to initialize any type
- Compiler automatically generates initialization code
- keyword auto is not allowed

Listing 61: Car Class Declaration car3

```cpp

class Car {

private:
    float fuel     {0}; // non-static data member initialization
    float speed    {0}; // non-static data member initialization
    int passengers {0}; // non-static data member initialization
```

```
8
9  public:
10     Car();       // default constructor
11     Car(float);  // Paramterized constructor
12     ~Car();      // destructor
13     void FillFuel(float amount);
14     void Accelerate();
15     void Brake();
16     void AddPassengers(int count);
17     void Dashbord();
18 };
```

Listing 62: Car Class Definition car3 - Constructor

```
1  #include "car.h"
2  #include "iostream"
3
4  // default constructor
5  Car::Car() {
6
7      std::cout << "Car()" << std::endl;
8  }
9
10 // Parameterized constructor
11 Car::Car(float amount) {
12     fuel = amount;
13
14     std::cout << "Car(" << amount << ")" << std::endl;
15 }
16
17 // destructor
18 Car::~Car() {
19     std::cout << "~Car()" << std::endl;
20 }
```

Listing 63: Car Class Definition car3 - Services

```
1  void Car::FillFuel(float amount) {
2      fuel = amount;
3  }
4
5  void Car::Accelerate() {
6      speed++;
7      fuel -= 0.5f;
8  }
9
10 void Car::Brake() {
11     speed = 0;
12 }
13
14 void Car::AddPassengers(int count) {
15     passengers = count;
16 }
17
18 void Car::Dashbord() {
19
20     std::cout << "Fuel: " << fuel << std::endl;
21     std::cout << "Speed: " << speed << std::endl;
22     std::cout << "Passenger: " << passengers << std::endl;
23 }
```

Listing 64: Main CPP - car3 Client of Car Class

```cpp
#include "car.h"
#include <iostream>

int main() {
    using namespace std;

    cout << "car c" << endl;
    Car c;
    c.Dashbord();

    cout << "\ncar b" << endl;
    Car b(5);
    b.Dashbord();

    return 0;

}

```

Listing 65: Makefile car3

```
CXX=g++
CXXFLAGS=-std=c++17 -Wall -Wextra -Iinc

SRC=src/main.cpp src/car.cpp

app: $(SRC)
	$(CXX) $(CXXFLAGS) $(SRC) -o app

run: app
	./app

clean:
	rm -f app
```

Listing 66: Run car3

```
make run
```

```
g++ -std=c++17 -Wall -Wextra -Iinc src/main.cpp src/car.cpp -o app
./app
car c
Car()
Fuel: 0
Speed: 0
Passenger: 0

car b
Car(5)
Fuel: 5
Speed: 0
Passenger: 0
~Car()
~Car()
```

4.6 this Poiner

- A hidden pointer passed to member function
- **Points to the object that invoked the member function**
- Provided as a keyword that is meaningful only in member functions
- Can be used to access members inside the member functions

Listing 67: Car Class Declaration car4

```
1
2  class Car {
3
4  private:
5      float fuel     {0}; // non-static data member initialization
6      float speed    {0}; // non-static data member initialization
7      int passengers {0}; // non-static data member initialization
8
9  public:
10     Car();       // default constructor
11     Car(float);  // Paramterized constructor
12     ~Car();      // destructor
13     void FillFuel(float amount);
14     void Accelerate();
15     void Brake();
16     void AddPassengers(int count);
17     void Dashbord();
18 };
```

Listing 68: Examples of this pointer usage

```
1      #include "car.h"
2      #include "iostream"
3
4      // default constructor
5      Car::Car() { std::cout << "Car()" << std::endl; }
6      // Parameterized constructor
7      Car::Car(float amount) {
8          fuel = amount;
9          std::cout << "Car(" << amount << ")" << std::endl;
10     }
11     // destructor
12     Car::~Car() { std::cout << "~Car()" << std::endl;}
13   void Car::FillFuel(float amount) {fuel = amount;}
14
15     void Car::Accelerate() {
16         this->speed++;            // member acces with this pointer
17         this->fuel -= 0.5f;       // member acces with this pointer
18     }
19
20     void Car::Brake() { speed = 0;}
21
22     void Foo(const Car &car) {}
23
24   // help the compiler to distinguishes local and member variables
25   // invoke Foo() function from a member function
26   // invoke Foo() function and pass Car object as an argument
27   // derefence this pointer give back an object
28     void Car::AddPassengers(int passengers) {
29         this->passengers = passengers;
```

```
30          Foo(*this);
31      }
32
33      void Car::Dashbord() {
34          std::cout << "Fuel: " << fuel << std::endl;
35          std::cout << "Speed: " << speed << std::endl;
36          std::cout << "Passenger: " << passengers << std::endl;
37      }
```

Listing 69: Main CPP - car4 Client of Car Class

```
1  #include "car.h"
2  #include <iostream>
3
4  int main() {
5      using namespace std;
6
7      cout << "car c" << endl;
8      Car c;
9      c.Dashbord();
10
11     cout << "\ncar b" << endl;
12     Car b(5);
13     b.Dashbord();
14
15     return 0;
16
17 }
18
```

Listing 70: Makefile car4

```
1  CXX=g++
2  CXXFLAGS=-std=c++17 -Wall -Wextra -Iinc
3
4  SRC=src/main.cpp src/car.cpp
5
6  app: $(SRC)
7          $(CXX) $(CXXFLAGS) $(SRC) -o app
8
9  run: app
10         ./app
11
12 clean:
13         rm -f app
```

Listing 71: Run car4

```
1  make run
```

4.7 Static Class Members

Listing 72: Car Class Declaration car5

```
1
2    class Car {
3
4    private:
5        float fuel     {0}; // non-static data member initialization
```

```
    float speed    {0}; // non-static data member initialization
    int passengers {0}; // non-static data member initialization

    static int count;   // static member function

public:
    Car();       // default constructor
    Car(float);  // Paramterized constructor
    ~Car();      // destructor
    void FillFuel(float amount);
    void Accelerate();
    void Brake();
    void AddPassengers(int count);
    void Dashbord();

    static void ShowCount();
};
```

Listing 73: Examples of Static Class Members

```
#include "car.h"
#include "iostream"

// initialization of static member variables
int Car::count = 0;

// default constructor
Car::Car() {

    ++count;  //increment static member variable
    std::cout << "Car()" << std::endl;
}
// Parameterized constructor
Car::Car(float amount) {
    ++count;              //increment static member variable
    fuel = amount;
    std::cout << "Car(" << amount << ")" << std::endl;
}
// destructor
Car::~Car() {
    --count;              //decrement static member variable
    std::cout << "~Car()" << std::endl;
}
```

Listing 74: Examples of Static Class Members

```
void Car::FillFuel(float amount) {
    fuel = amount;
}
void Car::Accelerate() {
    this->speed++;            // member acces with this pointer
    this->fuel -= 0.5f;       // member acces with this pointer
}
void Car::Brake() {
    speed = 0;
}
void Foo(const Car &car) {

}
// help the compiler to distinguishes local and member variables
```

```
  // invoke Foo() function from a member function
  // invoke Foo() function and pass Car object as an argument
  // derefence this pointer give back an object
  void Car::AddPassengers(int passengers) {
      this->passengers = passengers;
      Foo(*this);
  }
  void Car::Dashbord() {
      std::cout << "Fuel: " << fuel << std::endl;
      std::cout << "Speed: " << speed << std::endl;
      std::cout << "Passenger: " << passengers << std::endl;
  }
  void Car::ShowCount() {
      std::cout << "Number of Cars: " << count << std::endl;
  }
```

Listing 75: Main CPP - car5 Client of Car Class

```
  #include "car.h"
  #include <iostream>

  int main() {
      using namespace std;

      Car::ShowCount();

      Car car(4);
      // car.FillFuel(6);
      for(int i=0; i<4; i++) {
    car.Accelerate();
      }

      car.Dashbord();

      Car c1, c2;

      Car::ShowCount();

      return 0;
  }
```

Listing 76: Makefile car5

```
CXX=g++
CXXFLAGS=-std=c++17 -Wall -Wextra -Iinc

SRC=src/main.cpp src/car.cpp

app: $(SRC)
        $(CXX) $(CXXFLAGS) $(SRC) -o app

run: app
        ./app

clean:
        rm -f app
```

Listing 77: Run car5

```
make run
```

```
./app
Number of Cars: 0
Car(4)
Fuel: 2
Speed: 4
Passenger: 0
Car()
Car()
Number of Cars: 3
~Car()
~Car()
~Car()
```

4.7.1 Constant Member Functions

- Member functions qualified with const keyword
- Both declaration and definition is qualified
- Such functions **cannot change value** of any member variables
- Usful for creating **read-only** functions
- Constant objects can invoke only constant member functions

Listing 78: Car Class Declaration car6

```
1
2  class Car {
3
4  private:
5      float fuel     {0}; // non-static data member initialization
6      float speed    {0}; // non-static data member initialization
7      int passengers {0}; // non-static data member initialization
8
9      static int count;   // static member function
10
11 public:
12     Car();       // default constructor
13     Car(float);  // Paramterized constructor
14     ~Car();      // destructor
15     void FillFuel(float amount);
16     void Accelerate();
17     void Brake();
18     void AddPassengers(int count);
19
20     void Dashbord () const;
21     static void ShowCount ();
22 };
```

Listing 79: Examples of Constant Member Functions

```
1  #include "car.h"
2  #include "iostream"
3
4
5  // initialization of static member variables
6  int Car::count = 0;
7
8  // default constructor
```

```
Car::Car() {
    ++count;  //increment static member variable
    std::cout << "Car()" << std::endl;
}

// Parameterized constructor
Car::Car(float amount) {
    ++count;              //increment static member variable
    fuel = amount;
    std::cout << "Car(" << amount << ")" << std::endl;
}

// destructor
Car::~Car() {
    --count;              //decrement static member variable
    std::cout << "~Car()" << std::endl;
}
```

Listing 80: Examples of Constant Member Functions

```
void Car::FillFuel(float amount) { fuel = amount;}

void Car::Accelerate() {
    this->speed++;            // member acces with this pointer
    this->fuel -= 0.5f;       // member acces with this pointer
}

void Car::Brake() { speed = 0; }

void Foo(const Car &car) {}
// help the compiler to distinguishes local and member variables
// invoke Foo() function from a member function
// invoke Foo() function and pass Car object as an argument
// derefence this pointer give back an object
void Car::AddPassengers(int passengers) {
    this->passengers = passengers;
    Foo(*this);
}

void Car::Dashbord() const {
    std::cout << "Fuel: " << fuel << std::endl;
    std::cout << "Speed: " << speed << std::endl;
    std::cout << "Passenger: " << passengers << std::endl;
    // Not allowed in const member function!
    // this->fuel++;
}

void Car::ShowCount() {std::cout << "Number of Cars: " << count << std::endl;}
```

Listing 81: Main CPP - car6 Client of Car Class

```
#include "car.h"
#include <iostream>

int main() {
    using namespace std;

    Car::ShowCount();

    Car car(4);
```

```
10      car.Dashbord();
11
12      Car c1, c2;
13      Car::ShowCount();
14
15      return 0;
16  }
```

Listing 82: Makefile car6

```
1  CXX=g++
2  CXXFLAGS=-std=c++17 -Wall -Wextra -Iinc
3
4  SRC=src/main.cpp src/car.cpp
5
6  app: $(SRC)
7          $(CXX) $(CXXFLAGS) $(SRC) -o app
8
9  run: app
10         ./app
11
12 clean:
13         rm -f app
```

Listing 83: Run car6

```
1  make run
```

```
./app
Number of Cars: 0
Car(4)
Fuel: 4
Speed: 0
Passenger: 0
Car()
Car()
Number of Cars: 3
~Car()
~Car()
~Car()
```

4.7.2 Copy Constructor - Part I

- Creates copy of the object's state in another object
- Synthesized automatically
- Default implementation copies values
- User-defined implementation required for pointer members

Listing 84: Integer Class -

```
1  #include <iostream>
2
3  class Integer {
4      int *m_pInt;
5
```

```
public:
    Integer();
    Integer(int value);
    int GetValue() const;
    void SetValue(int value);

    ~Integer();
};

Integer::Integer() {
    m_pInt = new int(0);
}

Integer::Integer(int value) {
    m_pInt = new int(value);
}

Integer::~Integer() {
    delete m_pInt;
}

int Integer::GetValue() const {
    return *m_pInt;
}

void Integer::SetValue(int value) {
    *m_pInt = value;
}

using namespace std;

int main() {

    Integer i(5);
    Integer i2(i);      // !!!!   risky operation   !!!!
    cout << i.GetValue() << endl;
    cout << i2.GetValue() << endl;

    return 0;
}
```

4.7.3 Copy Constructor - Part II

- Creates copy of the object's state in another object
- Synthesized automatically
- Default implementation copies values
- User-defined implementation required for pointer members

Listing 85: Integer Class -

```
#include <iostream>

//---------------------------------------------------------------------
// Class Declaration
//---------------------------------------------------------------------
class Integer {
    int *m_pInt;

public:
```

```
    Integer();
    Integer(int value);
    Integer(const Integer& obj);  // Copy Constructor, copy by value
    int GetValue() const;
    void SetValue(int value);

    ~Integer();
};

//-----------------------------------------------------------------------
// Class Definition
//-----------------------------------------------------------------------
Integer::Integer() {  m_pInt = new int(0);}
Integer::Integer(int value) {m_pInt = new int(value);}
Integer::~Integer() {delete m_pInt;}
//copy constructor - Deep Copy
//Reference avoids loop of shallow copy
Integer::Integer(const Integer& obj) {m_pInt = new int(*obj.m_pInt);}
int Integer::GetValue() const {return *m_pInt;}
void Integer::SetValue(int value) {*m_pInt = value;}


//-----------------------------------------------------------------------
// Test Program
//-----------------------------------------------------------------------
using namespace std;

int main() {

    int *p1 = new int(5);
    // Shallow Copy - Copies only the pointer address
    int *p2 = p1;
    // Deep Copy - Copies the value at the address of the pointer
    int *p3 = new int(*p1);

    // mark the pointers as used!
    (void)p1;
    (void)p2;
    (void)p3;

    Integer i  {5};
    Integer i2 {i};      // call copy constructor
    cout << i.GetValue() << endl;
    cout << i2.GetValue() << endl;

    return 0;
}
```



Rule Of 3

- All should be defined if a user implements any of them
  - Destructor
  - Copy Constructor
  - Copy assignment operator
- This will be due to allocation of some resources in a constructor
  - Destructor will free the resource
  - Copy constructor will perform a **deep copy**

  - Copy assignment operation will also perform a **deep copy**
- Not Implementing user defined operations can lead to memory leak or shallow copy!

4.7.4 Delegation Constructors (C++11)

- Allows a constructor to invoke another constructor
- Replacment for common initialization
- Reduces duplicate initialization code in multiple constructors



Syntax

class Class {
Class():Class(val1, val2) { ///Default Constructor delegates to Parmeterized Constructor Class(arg1, arg2) { //Parameterized Constructor ///Initialization code
};

Listing 86: Car Class Declaration car7

```
1
2  class Car {
3
4  private:
5      float fuel     {0}; // non-static data member initialization
6      float speed    {0}; // non-static data member initialization
7      int passengers {0}; // non-static data member initialization
8
9      static int count;   // static member function
10
11 public:
12     Car();              // default constructor
13     Car(float amunt);   // Paramterized constructor
14     Car(float amount, int passengers);
15     ~Car();       // destructor
16     void FillFuel(float amount);
17     void Accelerate();
18     void Brake();
19     void AddPassengers(int count);
20
21     void Dashbord () const;
22     static void ShowCount ();
23 };
```

Listing 87: Examples of Delegation Constructor

```
1  #include "car.h"
2  #include "iostream"
3
4
5  // initialization of static member variables
6  int Car::count = 0;
7
8  // default constructor
9  Car::Car():Car(0) {
10     std::cout << "Car()" << std::endl;
11 }
```

```
12
13  // Parameterized constructor
14  Car::Car(float amount):Car(amount,0) {
15      std::cout << "Car(float), " << amount << std::endl;
16  }
17
18    Car::Car(float amount, int passengers) {
19      ++count;              //increment static member variable
20      fuel = amount;
21      this->passengers = passengers;
22      std::cout << "Car(float, int), " << amount << ", " << passengers << std::endl;
23  }
24
25  // destructor
26  Car::~Car() {
27      --count;              //decrement static member variable
28      std::cout << "~Car()" << std::endl;
29  }
30
```

Listing 88: Examples of Delegation Constructor

```
1
2   void Car::FillFuel(float amount) { fuel = amount;}
3
4   void Car::Accelerate() {
5       this->speed++;            // member acces with this pointer
6       this->fuel -= 0.5f;       // member acces with this pointer
7   }
8
9   void Car::Brake() { speed = 0; }
10
11  void Foo(const Car &car) { (void)car; }
12  // help the compiler to distinguishes local and member variables
13  // invoke Foo() function from a member function
14  // invoke Foo() function and pass Car object as an argument
15  // derefence this pointer give back an object
16  void Car::AddPassengers(int passengers) {
17      this->passengers = passengers;
18      Foo(*this);
19  }
20
21  void Car::Dashbord() const {
22      std::cout << "Fuel: " << fuel << std::endl;
23      std::cout << "Speed: " << speed << std::endl;
24      std::cout << "Passenger: " << passengers << std::endl;
25      // Not allowed in const member function!
26      // this->fuel++;
27  }
28
29  void Car::ShowCount() {std::cout << "Number of Cars: " << count << std::endl;}
```

Listing 89: Main CPP - car7 Client of Car Class

```
1   #include "car.h"
2   #include <iostream>
3
4   int main() {
5       using namespace std;
6
```

```
7       Car car;
8
9       return 0;
10  }
```

Listing 90: Makefile car7

```
1   CXX=g++
2   CXXFLAGS=-std=c++17 -Wall -Wextra -Iinc
3
4   SRC=src/main.cpp src/car.cpp
5
6   app: $(SRC)
7           $(CXX) $(CXXFLAGS) $(SRC) -o app
8
9   run: app
10          ./app
11
12  clean:
13          rm -f app
```

Listing 91: Run car7

```
1   make run
```

```
g++ -std=c++17 -Wall -Wextra -Iinc src/main.cpp src/car.cpp -o app
./app
Car(float, int), 0, 0
Car(float), 0
Car()
~Car()
```

4.7.5 Default & Deleted Functions (C++11)

Listing 92: Integer Class - default keyword

```
1   #include <iostream>
2
3   class Integer {
4       int m_Value {0};
5
6   public:
7       Integer() = default;
8       Integer(int value) {m_Value = value;}
9       Integer(const int&) = default;
10  };
11
12  using namespace std;
13
14  int main() {
15
16      Integer i1;
17      Integer i2(3);
18
19      return 0;
20  }
```

Listing 93: Integer Class - delete keyword

```cpp
#include <iostream>

class Integer {
    int m_Value {0};

public:
    Integer() = default;
    Integer(int value) {m_Value = value;}
    Integer(const int&) = delete;
    void SetValue(int value) {m_Value = value;}
    void SetValue(float) = delete;
};

using namespace std;

int main() {

    Integer i1;
    // Integer i2(3);              // not possible anymoe
    i1.SetValue(100);
    // i1.SetValue(3.7f);          // not possible anymore

    return 0;
}
```

4.8 TODO Constant Member Functions

4.9 TODO Copy Constructor - Part I

4.10 TODO Copy Constructor - Part II

4.11 TODO Delegating Constructors (C++11)

4.12 TODO Default & Deleted Functions (C++11)

5 Move Semantic

5.1 L-values, R-values & R-value References (C++11)

L-Value	R-Value
Has a name	Does not have a name
All variables are L-Values	R-value is a temporary value
Can be assigend values	Cannot be assigned values
Some expressions return l-values	Some expressions return r-value
L-value persists beyond the expression	Does not persist beyond the expression
Functions that return by reference return l-value	Functions that return by value return r-value
Reference to l-value (called l-value reference)	R-value reference to r-value (called r-value reference)

Listing 94: L-Value and R-Value

```
#include <iostream>

int Add(int x, int y)   { return x+ y;}
int & Transform(int &x) { x *=x; return x;}

using namespace std;

int main() {
    //x,y & z are l-values & 5, 10 & 8 are r-values

    int x = 5;
    int y = 10;
    int z = 8;

    //Expression returns r-value
    int result = (x+y) * z;

    //Expression returns l-value
    ++x = 6;

    return 0;
}
```

R-Value References (C++11)

- A reference created to a temporary
- Represents a temporary
- Created with && operator (R-value reference operator)
- Cannot point to l-values
- R-value references always bind to temporaries
- L-value references always bind to l-values

R Value References

int &&r1 = 10; //R-value reference
int &&r2 = Add(5,8); //Add returns by value (temporary)
int &&r3 = 7+2; //Expression return a temporary

Listing 95: R-Value References, binds to const reference

```cpp
#include <iostream>

using namespace std;

//Returns r-value
int Add(int x, int y)   { return x+ y;}
//Returns l-value
int & Transform(int &x) { x *=x; return x;}

void Print(int &x)       { cout << "Print(int &x)" << endl; (void)x;}
void Print(const int &x) { cout << "Print(const int &x)" << endl; (void)x;}
//void Print(int &&x)      { cout << "Print(int &&x)" << endl; }

int main() {
    int x = 10;
    Print(x);
    Print(3);

    return 0;
}
```

```
Print(int &x)
Print(const int &x)
```

Listing 96: R-Value References, binds to R-value Reference

```cpp
#include <iostream>

using namespace std;

//Returns r-value
int Add(int x, int y)   { return x+ y;}
//Returns l-value
int & Transform(int &x) { x *=x; return x;}

void Print(int &x)       { cout << "Print(int &x)" << endl; (void)x;}
void Print(const int &x) { cout << "Print(const int &x)" << endl; (void)x;}
void Print(int &&x)      { cout << "Print(int &&x)" << endl; (void)x;}

int main() {
    int x = 10;
    Print(x);
    Print(3);

    return 0;
}
```

```
Print(int &x)
Print(int &&x)
```

To Remember

The temporary is always bind to R-value references, but they can also bind to const references. In overloading they always bind to R-Value references.
This can be used to detect temporaries in expressions that use user defined objects.

5.2 Move Semantics - Basics (C++11)

- Copy is implemented through copy constructor
- Copy of the object state is created
- Wasteful in case copy is created from a temporary
- Instead, the state can be moved from the source object
- Implemented through move semantics

5.3 Move Semantics - Implementation (C++11)

NRVO

NRVO: Named Return Value Optimization.
It's a specific compiler optimization in C++ where a named local variable returned from a function is constructed directly in the caller's return slot, avoiding a copy or move.
OFF: C++ :flags "-std=c++11 -fno-elide-constructors"

Listing 97: Integer Class - Copy Semantics

```
1   #include <iostream>
2
3   using namespace std;
4
5   class Integer {
6       int *m_pInt;
7
8   public:
9       Integer() {                                          // default constructor
10      cout << "Integer()" << endl;
11      m_pInt = new int(0);
12      }
13      Integer(int value) {                                 // parameterized constructor
14      cout << "Integer(int)" << endl;
15      m_pInt = new int(value);
16      }
17      Integer(const Integer &obj) {                        // copy constructor
18      cout << "Integer(const Integer& obj)" << endl;
19      m_pInt = new int(*obj.m_pInt);                      // allocate new memory and copy the data
20      }
21      ~Integer() {                                         // destructor
22        cout << "~Integer()" << endl;
23        delete m_pInt;
24      }
25      int GetValue() const {return *m_pInt;}
26      void SetValue(int value) {*m_pInt = value;}
27  };
28
29
30  Integer Add(const Integer &a, const Integer &b)
31  {
32      Integer temp;
33      temp.SetValue(a.GetValue() + b.GetValue());
```

```
34      return temp;                                        // calls copy constructor when
        ↪ returning by value
35  }
36
37    int main() {
38        Integer a(1), b(3);
39        a.SetValue(Add(a,b).GetValue());
40        return 0;
41    }
```

```
Integer(int)
Integer(int)
Integer()
Integer(const Integer& obj)
~Integer()
~Integer()
~Integer()
~Integer()
```

Listing 98: Integer Class - Move Semantics

```
1   #include <iostream>
2
3   using namespace std;
4
5   class Integer {
6       int *m_pInt;
7
8   public:
9       Integer() {                                         // default constructor
10          cout << "Integer()" << endl;
11          m_pInt = new int(0);
12      }
13      Integer(int value) {                                // parameterized constructor
14          cout << "Integer(int)" << endl;
15          m_pInt = new int(value);
16      }
17      Integer(const Integer & obj) {                      // copy constructor
18          cout << "Integer(const Integer& obj)" << endl;
19          m_pInt = new int(*obj.m_pInt);
20      }
21      Integer(Integer && obj) {                           // MOVE CONSTRUCTOR
22      cout << "Integer(Integer && obj)" << endl;
23      m_pInt = obj.m_pInt;                              // steeling the resources from obj (shallow
        ↪ copy)
24      obj.m_pInt = nullptr;                             // avoids crashing desctructor when obj is
        ↪ deleted
25      }
26      ~Integer() {
27      cout << "~Integer()" << endl;
28      delete m_pInt;
29      }
30      int GetValue() const {return *m_pInt;}
31      void SetValue(int value) {*m_pInt = value;}
32  };
33
34
35  Integer Add(const Integer &a, const Integer &b)
36  {
37      Integer temp;
```

```
38      temp.SetValue(a.GetValue() + b.GetValue());
39      return temp;
40  }
41
42  int main() {
43      Integer a(1), b(3);
44      a.SetValue(Add(a,b).GetValue());
45      return 0;
46  }
```

```
Integer(int)
Integer(int)
Integer()
Integer(Integer && obj)
~Integer()
~Integer()
~Integer()
~Integer()
```

5.4 Rule of Five

5.4.1 Resource Management

- Some classes own a resource
- This resource may be a poiner, file handle, socket etc.
- Such resources may be aquired in the constructor
- Subsenquently, you have to decide the action in case objects of such classes are copied, moved or destroyed
- e.g. on destruction, the resource must be released to avoid *leaks#
- Similarly, on copy or move the resource must be handled accordingly

5.4.2 Rule of Five

- Consequently, you should implement the rule of 5 for such classes
- If a class has ownership semantics, then you must provide a user-defined
  - Destructor
  - Copy constructor
  - Copy assignment operator
  - Move constructor
  - Move assignment operator
- This is to ensure proper handling of the underlying resource

5.4.3 Rule of Yero

- If a class does not have ownership semantics, then do not provide any user defined function from the rule of 5.
- This way the compiler will automatically synthesize the necessary functions
- Providing user-defined implementation of some functions will prevent the compiler from synthesizing others

5.5 TODO Copy Elision

5.6 std::move Function (C++11)

Listing 99: Integer Class - std::move

```
#include <iostream>

using namespace std;

class Integer {
    int *m_pInt;
public:
    Integer() {                                          // default constructor
        cout << "Integer()" << endl;
        m_pInt = new int(0);
    }
    Integer(int value) {                                 // parameterized constructor
        cout << "Integer(int)" << endl;
        m_pInt = new int(value);
    }
    Integer(const Integer & obj) {                       // copy constructor
        cout << "Integer(const Integer& obj)" << endl;
        m_pInt = new int(*obj.m_pInt);
    }
    Integer(Integer && obj) {                            // MOVE CONSTRUCTOR
        cout << "Integer(Integer && obj)" << endl;
        m_pInt = obj.m_pInt;                             // steeling the resources from obj
        ↪  (shallow copy)
        obj.m_pInt = nullptr;                            // avoids crashing desctructor when obj
        ↪  is deleted
    }
    ~Integer() {
        cout << "~Integer()" << endl;
        delete m_pInt;
    }
    int GetValue() const {return *m_pInt;}
    void SetValue(int value) {*m_pInt = value;}
};

Integer Add(const Integer &a, const Integer &b)
{
    Integer temp;
    temp.SetValue(a.GetValue() + b.GetValue());
    return temp;
}

int main() {
    Integer a(1);
    cout << "copy constructor with auto b(a);" << endl;
    auto b(a);
    cout << "move constructor with auto c(static_cast<Integer>(&a);" << endl;
    auto c(static_cast<Integer&&>(a));

    // std::move() shows the intention more clearly than a simple cast operation.
    cout << "move constructor with auto d(std::move(a));" << endl;
    auto d(move(a));
    return 0;
}
```

```
Integer(int)
```

```
copy constructor with auto b(a);
Integer(const Integer& obj)
move constructor with auto c(static_cast<Integer>(&a);
Integer(Integer && obj)
move constructor with auto d(std::move(a));
Integer(Integer && obj)
~Integer()
~Integer()
~Integer()
~Integer()
```

6 Operator Overloading

6.1 Operator Overloading - Part I-III

- Custom implementation for primitive operators
- Allow usage of user-defined objects in mathematical expression
- Overloaded as functions but provide a convenient notation
- Implemented as member or global functions
- Require usage of the operator keyword
  <ret> operator <#> (<arguments>)
- As global functions, require same no. of arguments as the operands
- As member functions, one operand is passed as argument through this pointer
  - binary operator will require only one explicit argument
  - unary operator will not require any explicit argument
    Integer operator+(const Integer &, const Integer &); Integer Integer::operator+(const Integer &);

Listing 100: Integer Class Declaration

```
#include <iostream>
class Integer {
    int *m_pInt;
public:
    Integer();                                  // Default constructor
    Integer(int value);                         // Parameterized constructor
    Integer(const Integer &obj);                // Copy constructor
    Integer(Integer &&obj);                     // Move constructor
    int GetValue()const;
    void SetValue(int value);
    ~Integer();
    Integer & operator ++();                    // pre-increment operator
    Integer operator ++(int);                   // post-increment operator
    bool operator ==(const Integer &a)const;    // comparison
    Integer & operator =(const Integer &a);     // Copy assignment
    Integer & operator =(Integer &&a);          // Move assignment
    Integer operator +(const Integer & a)const; // + operator

    void operator ()();                         // Function Call Operator
    explicit operator int();                    // Type Conversion Operator

    // operator >> is decalred as friend of Integer Class
    friend std::istream & operator >> (std::istream &input, Integer &a);
};
Integer operator +(int x, const Integer &y);
std::ostream & operator <<(std::ostream & out, const Integer &a);
std::istream & operator >> (std::istream &input, Integer &a);
```

Listing 101: Integer Class Definition: Constructors

```
#include "integer.h"
#include <iostream>

using namespace std;

```

```
Integer::Integer() {                            // standard constructor
    cout << "Integer()" << endl;
    m_pInt = new int(0);
}

Integer::Integer(int value) {                   // paramterizied constructor
    cout << "Integer(int)" << endl;
    m_pInt = new int(value);
}

Integer::Integer(const Integer & obj) {         // copy constructor
    cout << "Integer(const Integer&)" << endl;
    m_pInt = new int(*obj.m_pInt);
}

Integer::Integer(Integer && obj) {              // move constructor
    cout << "Integer(int&&)" << endl;
    m_pInt = obj.m_pInt;
    obj.m_pInt = nullptr;
}

Integer::~Integer() {                          // destructor
    cout << "~Integer()" << endl;
    delete m_pInt;
}
```

Listing 102: Integer Class Definition: Setter and Getter

```

  int Integer::GetValue() const {
    return *m_pInt;
  }

  void Integer::SetValue(int value) {
    *m_pInt = value;
  }
```

Listing 103: Integer Class Definition: Operator Overlaoding member functiosn

```
Integer & Integer::operator++() { // ++a means a.operator++()
    cout << "operator++()" << endl;
    ++(*m_pInt);
    return *this;  // the value a the address of this
}

Integer Integer::operator++(int) { // a++ means a.operator++(int)
    cout << "operator++(int)" << endl;
    Integer temp(*this);
    ++(*m_pInt);
    return temp;
}

bool Integer::operator==(const Integer & a) const { // c==a means c.operator==(a)
    cout << "operator==(const Integer &) const" << endl;
    return *m_pInt == *a.m_pInt;
}

Integer & Integer::operator=(const Integer & a) { // c=a means c.operator=(a)
    cout << "operator=(const Integer &) const" << endl;
    if (this != &a) {            // check for self assignemnt
```

```
    delete m_pInt;
    m_pInt = new int(*a.m_pInt);
    }
    return *this;
}

Integer & Integer::operator=(Integer && a) { // c=&&a means c.operator=(&&a)
    cout << "operator=(Integer &&) const" << endl;
    if (this != &a) {
    delete m_pInt;
    m_pInt = a.m_pInt;
    a.m_pInt = nullptr;
    }
    return *this;
}

Integer Integer::operator+(const Integer & a) const { // c+a means c.operator+(a)
    cout << "operator+(const Integer &) const" << endl;
    Integer temp;
    *temp.m_pInt = *m_pInt + *a.m_pInt; // *this->m_pInt
    return temp;
}

void Integer::operator()() {
    // cout << "operator()()" << endl;
    cout << *m_pInt << endl;
}
```

Listing 104: Integer Class Definition: Global Operator overloading

```
Integer operator +(int x, const Integer &y) {
    // cout << "operator+(int, const Integer)" << endl;
    Integer temp;
    temp.SetValue(x + y.GetValue());
    return temp;
}
ostream & operator <<(ostream & out, const Integer &a) {
    // cout << "operator <<(ostream &, const Integer &)" << endl;
    out << a.GetValue();
    return out;
}
istream & operator >> (istream &input, Integer &a) {
    // cout << "operator >>(istream &, Integer &)" << endl;
    int x;
    input >> x;
    // a.SetValue(x);  // SetValue() allows copy by value, no friend required
    *a.m_pInt = x;     // access to private member of Integer Class.
    return input;
}
```

6.1.1 Integer-01: Add operator+()

Listing 105: Main CPP - Integer Client

```
  #include <integer.h>
  #include <iostream>
  #include <sstream>

    void Process(Integer val) {

        (void)val;
```

```
    }

    int main() {
        using namespace std;
        istringstream cin_sim("63");

// 1) Overloading operator+
        Integer a(1), b(2);
        Integer sum = a + b;
        cout << sum.GetValue() << endl;

// 2) Overloading operator++     Pre-Increment
        ++sum;
        cout << sum.GetValue() << endl;

// 3) Overloading operator++     Post-Increment
        sum++;
        cout << sum.GetValue() << endl;
// 4) Overloading operator==       Comparison
        cout << (a == b) << endl;

// 5) Overloading operator=          Assignment
        Integer c;
        c = a;
        cout << c.GetValue() << endl;
// 6) Overloading operator+ with int
        Integer sum1 = a + 5;  // a.operator+(5)
        Integer sum2 = 5 + a;  // 1. operand is of primitive type -> overlaod as global function
        cout << sum1.GetValue() << ", " << sum2.GetValue()<< endl;
// 7) Overloading operator<<
        cout << sum1 << ", " << sum2 << endl; // operator <<(std::cout, sum1).operator(std::endl)
// 8) Overloading operator>>
        cin >> c;
        cout << c << endl;;
// 9) Overloading operator() Functin Call Operator
        b();
        return 0;
    }
```

Listing 106: Makefile Integer

```
CXX=g++
CXXFLAGS=-std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc

SRC=src/main.cpp ../src/integer.cpp

app: $(SRC)
        $(CXX) $(CXXFLAGS) $(SRC) -o app

run: app
        ./app

clean:
        rm -f app
```

```
./app
Integer(int)
Integer(int)
Integer()
Integer(int&&)
```

```
~Integer()
3
4
Integer(const Integer&)
Integer(int&&)
~Integer()
~Integer()
5
0
Integer()
1
Integer(int)
Integer()
Integer(int&&)
~Integer()
~Integer()
Integer()
Integer(int&&)
~Integer()
6, 6
6, 6
32765
2
~Integer()
~Integer()
~Integer()
~Integer()
~Integer()
~Integer()
```

6.1.2 Integer-02: Pre Increment operator++()

Listing 107: Main CPP - Integer Client

```
#include <integer.h>
#include <iostream>

int main() {
        using namespace std;

// 1) Overloading operator+
        Integer a(1), b(2);
        Integer sum = a + b;
        cout << sum.GetValue() << endl;

// 2) Overloading operator++     Pre-Increment
        ++sum;
        cout << sum.GetValue() << endl;

        return 0;
   }
```

```
./app
Integer(int)
Integer(int)
Integer()
Integer(int&&)
~Integer()
```

```
3
4
~Integer()
~Integer()
~Integer()
```

6.1.3 Integer-03: Post Increment operator ++(int)

Listing 108: Main CPP - Integer Client

```cpp
  #include <integer.h>
  #include <iostream>

int main() {
        using namespace std;

 // 1) Overloading operator+
        Integer a(1), b(2);
        Integer sum = a + b;
        cout << sum.GetValue() << endl;

 // 3) Overloading operator++     Post-Increment
        sum++;
        cout << sum.GetValue() << endl;

        return 0;
    }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(int)
Integer()
Integer(int&&)
~Integer()
3
Integer(const Integer&)
Integer(int&&)
~Integer()
~Integer()
4
~Integer()
~Integer()
~Integer()
```

6.1.4 Integer-04: Comparison operator==()

Listing 109: Main CPP - Integer Client

```cpp
  #include <integer.h>
  #include <iostream>

int main() {
        using namespace std;

 // 1) Overloading operator+
        Integer a(1), b(2);
        Integer sum = a + b;

```

```
11  // 4) Overloading operator==       Comparison
12        cout << (a == b) << endl;
13
14        return 0;
15     }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(int)
Integer()
Integer(int&&)
~Integer()
0
~Integer()
~Integer()
~Integer()
```

6.1.5 Integer-05: Assignment operator=()

Listing 110: Main CPP - Integer Client

```
1   #include <integer.h>
2   #include <iostream>
3
4  int main() {
5         using namespace std;
6
7  // 1) Overloading operator+
8         Integer a(1), b(2);
9         Integer sum = a + b;
10
11  // 5) Overloading operator=          Assignment
12        Integer c;
13        c = a;
14        cout << c.GetValue() << endl;
15        return 0;
16     }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(int)
Integer()
Integer(int&&)
~Integer()
Integer()
1
~Integer()
~Integer()
~Integer()
~Integer()
```

6.1.6 Integer-06: Global Add operator+(int x, const Integer &y)

Listing 111: Main CPP - Integer Client

```
1   #include <integer.h>
```

```
  #include <iostream>

int main() {
        using namespace std;

// 1) Overloading operator+
        Integer a(1), b(2);
        Integer sum = a + b;

 // 6) Overloading operator+ with int
        Integer sum1 = a + 5;  // a.operator+(5)
        Integer sum2 = 5 + a;  // 1. operand is of primitive type -> overlaod as global function
        cout << sum1.GetValue() << ", " << sum2.GetValue()<< endl;

        return 0;
    }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(int)
Integer()
Integer(int&&)
~Integer()
Integer(int)
Integer()
Integer(int&&)
~Integer()
~Integer()
~Integer()
Integer()
Integer(int&&)
~Integer()
6, 6
~Integer()
~Integer()
~Integer()
~Integer()
~Integer()
```

6.1.7 Integer-07: Global Output Operator «

Listing 112: Main CPP - Integer Client

```
  #include <integer.h>
  #include <iostream>

int main() {
        using namespace std;

// 1) Overloading operator+
        Integer a(1), b(2);
        Integer sum = a + b;

 // 6) Overloading operator+ with int
        Integer sum1 = a + 5;  // a.operator+(5)
        Integer sum2 = 5 + a;  // 1. operand is of primitive type -> overlaod as global function

 // 7) Overloading operator<<
        cout << sum1 << ", " << sum2 << endl; // operator <<(std::cout, sum1).operator(std::endl)
```

```
17          b();
18          return 0;
19      }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(int)
Integer()
Integer(int&&)
~Integer()
Integer(int)
Integer()
Integer(int&&)
~Integer()
~Integer()
Integer()
Integer(int&&)
~Integer()
6, 6
2
~Integer()
~Integer()
~Integer()
~Integer()
~Integer()
```

6.1.8 Integer-08: Gloabl Input Operator »

Listing 113: Main CPP - Integer Client

```
1   #include <integer.h>
2   #include <iostream>
3   #include <sstream>
4
5   int main() {
6           using namespace std;
7           istringstream cin_sim("63");
8
9   // 1) Overloading operator+
10          Integer a(1), b(2);
11          Integer sum = a + b;
12
13  // 5) Overloading operator=          Assignment
14          Integer c;
15          c = a;
16          cout << c.GetValue() << endl;
17
18  // 8) Overloading operator>>
19          cin >> c;
20          cout << c << endl;;
21
22      return 0;
23      }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
```

```
Integer(int)
Integer()
Integer(int&&)
~Integer()
Integer()
1
32767
~Integer()
~Integer()
~Integer()
~Integer()
```

6.1.9 Integer-09: Function Call operator()()

Listing 114: Main CPP - Integer Client

```
  #include <integer.h>
  #include <iostream>

int main() {
        using namespace std;

// 1) Overloading operator+
        Integer a(1), b(2);
        Integer sum = a + b;


 // 9) Overloading operator() Functin Call Operator
        b();
        return 0;
    }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(int)
Integer()
Integer(int&&)
~Integer()
2
~Integer()
~Integer()
~Integer()
```

6.2 Operator Overloading - Part IV (Friend Keyword)

Friends

Friends of a Class are defined in the declaration of a class

Listing 115: Friend Class Demonstration

```
  class A_Class {
      int a_private_int {5};
      void a_private_function() {;}

  public:
```

```
6       void a_public_function() {;}
7     };
8
9     class B_Class {
10        int b_private_int {5};
11        void b_private_function() {;}
12
13    public:
14        void b_public_function() {;}
15        friend class A_Class;           // give B_Class access to all private's of A_Class
16    };
17
```

Listing 116: Friend Class Usage in Integer Class

```
1   // ------------------------------------------------------------------------
2   // Integer Class Declaration
3   // ------------------------------------------------------------------------
4   class Integer {
5       int *m_pInt;
6   public:
7       Integer();                                  // Default constructor
8       Integer(int value);                         // Parameterized constructor
9       Integer(const Integer &obj);                // Copy constructor
10      Integer(Integer &&obj);                     // Move constructor
11      int GetValue()const;
12      void SetValue(int value);
13      ~Integer();
14      Integer & operator ++();                    // pre-increment operator
15      Integer operator ++(int);                   // post-increment operator
16      bool operator ==(const Integer &a)const;    // comparison
17      Integer & operator =(const Integer &a);     // Copy assignment
18      Integer & operator =(Integer &&a);          // Move assignment
19      Integer operator +(const Integer & a)const; // + operator
20
21      void operator ()();                         // Function Call Operator
22
23      // operator >> is friend of Class Integer
24      friend std::istream & operator >> (std::istream &input, Integer &a);
25  }
26
27  // ------------------------------------------------------------------------
28  // Global Function Declarations
29  // ------------------------------------------------------------------------
30  Integer operator +(int x, const Integer &y);
31  std::ostream & operator <<(std::ostream & out, const Integer &a);
32  std::istream & operator >> (std::istream &input, Integer &a);
33
34  // ------------------------------------------------------------------------
35  // Global Function Definition (operator>>() overloading)
36  // ------------------------------------------------------------------------
37  std::istream & operator >> (std::istream &input, Integer &a) {
38      int x;
39      input >> x;
40      // a.SetValue(x);  // SetValue() allows copy by value, no friend required
41      *a.m_pInt = x;     // access to private member of Integer Class.
42      return input;
43  }
44  // ------------------------------------------------------------------------
```

6.3 Operator Overloading Part V (Smart Pointer Basics)



RAII

Resource Aquisition Is Initialization: The lifetime of a resource is bound to a local object.

Listing 117: Main CPP - Smart Pointer for Integer Class

```
  #include <integer.h>
  #include <iostream>

// Smart Pointer for Integer Class
class IntPtr {
    Integer *m_p;
public:
    IntPtr(Integer *p) :m_p(p) {

    }
    ~IntPtr() {
    delete m_p;
    }
    Integer *operator ->() {
    return m_p;
    }
    Integer & operator *() {
    return *m_p;
    }
};

// Raw Pointer
// void CreateInteger() {
//     Integer *p = new Integer;
//     p->SetValue(3);
//     std::cout << p->GetValue() << std::endl;
//     delete p;  // risk of memory leak if not delete is not called!!
// }

// Smart Pointer for Integer Class
void CreateInteger() {
    IntPtr p = new Integer;
    p->SetValue(3);
    std::cout << p->GetValue() << std::endl;
}

int main() {
        using namespace std;

    CreateInteger();

// 10) Smart Pointer

        return 0;
    }
```

Listing 118: Run App

```
make run
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
```

Smart Pointer

A smart pointer behaves like a pointer but it automatically frees the memory.

6.4 Operator Overloading - Part VI (Smart Pointer in C++11)

Listing 119: Main CPP - Smart Pointer for Integer Class

```
#include <integer.h>
#include <iostream>
#include <memory>   // for smart pointer

using namespace std;

void Process(unique_ptr<Integer> ptr) {
    (void)ptr;
}

void Process(shared_ptr<Integer> ptr) {  // copy by value
                                         // increments shared-ptr reference counter
    (void)ptr;
}


// Smart Pointer C++11
void CreateInteger() {
    unique_ptr<Integer> p1(new Integer);
    p1->SetValue(3);
    cout << p1->GetValue() << endl;
    cout << "move: ";
    Process(move(p1));                  // move is allowed
                                        // pointer is deleted when Process terminates

    // auto px(p);                      // error use of deleted function
                                        // unique_ptr(const unique_ptr&) = delete;

    // cout << p1->GetValue() << endl;  // after move p1 is not valid anymore

    shared_ptr<Integer> p2(new Integer);
    Process(p2);                        // Copy by Value
    p2->SetValue(5);
    cout << p2->GetValue() << endl;     // P2 is deleted when shared-ptr reference counter is 0
}                                       // this is the case when CreateInteger exits.

int main() {
    CreateInteger();

    return 0;
}
```

Listing 120: Makefile Integer

```
CXX=g++
CXXFLAGS=-std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc

SRC=src/main.cpp ../src/integer.cpp

app: $(SRC)
        $(CXX) $(CXXFLAGS) $(SRC) -o app
```

```
8
9  run: app
10         ./app
11
12 clean:
13         rm -f app
```

Listing 121: Run App

```
1  make run
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer()
3
move: ~Integer()
Integer()
5
~Integer()
```



Unique Pointer

Is used when you do not want to share the underlying resource. (

6.5 Operator Overloading - Part VII (Rules)

- Associativity, precedence & arity (operand count) does not change
- Operator functions should be **non-static**
  - except for new & delete
- One argument shold be **user defined** type
- **Global** overload if first operand is primitive type
- Not all operators can be overloaded
  - . ?: .* sizeof
- Cannot define new operators
- Overload for conventional behaviour only

6.6 Type Conversions - Part I (Basics)

- Conversions between types
- Performed through casts
- Ordered by compiler **(implicit)** or user **(explicit)**
- Conversion between
  - basic & basic
  - basic & user defined
  - user defined & basic
  - user defined & user defined

Listing 122: Type conversion examples

```
#include <iostream>

using namespace std;

int main() {
    int a = 5, b = 2;
    float f1 = a/b;                     // result is 2 because
                                        // division of 2 integer gives also an integer
    cout << "f1= " << f1 << endl;

    float f2 = (float) a/b;             // c-style cast - no check for validness of cast
    cout << "f2= " << f2 << endl;

    float f3 = static_cast<float>(a)/b; // static cast
    cout << "f3= " << f3 << endl;

    char *p1 = (char *) &a;                    // syntaxs ok
    // char *p2 = static_cast<char *>(&a);     // syntaxs not ok - not compilable
    char *p3 = reinterpret_cast<char *>(&a);   // syntaxs ok
    (void)p1;                                  // avoids warning
    (void)p3;
    return 0;
}
```

```
f1= 2
f2= 2.5
f3= 2.5
```

C-Sytle Cast

float f2 = (float) a/b;

- no check for validness of cast
- discards the qualifiers, e.g. const, ...

6.7 Type Conversions - Part II (Primitive to User Type)

Primitie Type

This concersion is performed by means of Parameterized Constructor.

Listing 123: Main CPP - Type Conversion Primitive to User Defined Type

```
#include <integer.h>
#include <iostream>

using namespace std;

void Print(Integer a) {(void)a,}

int main() {

```

```
10      Integer a1{5};   // explicit type cast with constructor
11
12      // Initialization
13      Integer a2 = 5;  // implicit type conversion
14
15      Print(5);        // implicit type conversion
16
17      // Assignment
18      a1 = 7;          // implicti type conversion
19      return 0;
20  }
```

Listing 124: Run App

```
1  make run
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(int)
Integer(int)
~Integer()
~Integer()
~Integer()
```



Keyword: Explicite

In some cases that may be undesirable, we do not want the compiler to automatically use our constructor for implicit conversion.
Therefore we can mark the constructor with the explicit keyword in the class declaration.

So in most cased the single argument constructors of your class should be marked with **explicit** keyword.

6.8 Type Conversion - Part III (User to Primitive Type)

Listing 125: Main CPP - Type Conversion Primitive to User Defined Type

```
1   #include <integer.h>
2   #include <iostream>
3
4   using namespace std;
5
6   void Print(Integer a) {(void)a; }
7
8   Integer::operator int() {
9       return *m_pInt;
10  }
11
12  int main() {
13
14      Integer a1{5};   // explicit type cast with constructor
15
16      int x = static_cast<int>(a1);    // use conversion operator
17      return 0;
18  }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(int)
Integer(int)
~Integer()
Integer(int)
~Integer()
~Integer()
~Integer()
```



Type Conversion Operator

operator <type>()
No Arguments No Return Type

6.9 Type Conversion - Part IV (User defined Type to User defined Type)

Listing 126: Main CPP - User defined Type to User defined Type

```
#include <integer.h>
#include <iostream>

using namespace std;

class Product {
    Integer m_Id;

public:
    Product(const Integer &id) : m_Id{id} {
    cout << "Product(const Integer &)" << endl;
    }
    ~Product() {
    cout << "~Product" << endl;
    }
    const Integer& GetInteger() const {
    return m_Id;
    }
    operator Integer() {   // syntactic sugar
    return m_Id;
    }
};


void Print(Integer a) {(void)a; }

Integer::operator int() {
    return *m_pInt;
}

int main() {

    Product p {5};
    Integer id_1 = p.GetInteger();
    Integer id_2 = p;                 // id_2 = p.operator Integer();

    Integer id_3 {5};

```

```
    if(id_1 == p) {
    cout << "Id_1 matches with the product\n" << endl;
    }

    return 0;
}
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(const Integer&)
Product(const Integer &)
~Integer()
Integer(const Integer&)
Integer(const Integer&)
Integer(int)
Integer(const Integer&)
~Integer()
Id_1 matches with the product

~Integer()
~Integer()
~Integer()
~Product
~Integer()
```

6.10 Initialization vs. Assignment & Member Initialization

Listing 127: Main CPP - Assignment

```
#include <integer.h>
#include <iostream>

using namespace std;

class Product {
    Integer m_Id;

public:
    Product(const Integer &id)  {
    cout << "Product(const Integer &)" << endl;
    m_Id = id;                          // use of assignment
    }
    ~Product() {
    cout << "~Product()" << endl;
    }
};

int main() {

// 01 - Assignment Integer Classe
    // Integer a1;
    // a1 = 3;   // a1 = Integer(8);
    // cout << a1 << endl;

// 02 - Assignment Product Class
    Product p {5};

    return 0;
```

```
}
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer()
Product(const Integer &)
operator=(const Integer &) const
~Integer()
~Product()
~Integer()
```

Listing 128: Main CPP - Member Initializer List

```
#include <integer.h>
#include <iostream>

using namespace std;

class Product {
    Integer m_Id;

public:
    Product(const Integer &id) :m_Id (id) {         // use of member initializizer list
    cout << "Product(const Integer &)" << endl;
    }
    ~Product() {
    cout << "~Product()" << endl;
    }
};


int main() {

// 01 - Initialization Integer Classw
    // Integer a1 {3};
    // cout << a1 << endl;

// 02 - Initialization Product Class
    Product p {5};


    return 0;
}
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(const Integer&)
Product(const Integer &)
~Integer()
~Product()
~Integer()
```



Prioritize

Assignement need more function calls as assignemnt - therefore initialization shall be preferd over assignment.

7 Memory Management - Part II

7.1 Integer Class

Listing 129: Integer Class Declaration

```
#include <iostream>
class Integer {
    int *m_pInt;
public:
    Integer();                                  // Default constructor
    Integer(int value);                         // Parameterized constructor
    Integer(const Integer &obj);                // Copy constructor
    Integer(Integer &&obj);                     // Move constructor
    int GetValue()const;
    void SetValue(int value);
    ~Integer();
    Integer & operator ++();                    // pre-increment operator
    Integer operator ++(int);                   // post-increment operator
    bool operator ==(const Integer &a)const;    // comparison
    Integer & operator =(const Integer &a);     // Copy assignment
    Integer & operator =(Integer &&a);          // Move assignment
    Integer operator +(const Integer & a)const; // + operator

    void operator ()();                         // Function Call Operator
    explicit operator int();                    // Type Conversion Operator

    // operator >> is decalred as friend of Integer Class
    friend std::istream & operator >> (std::istream &input, Integer &a);
};
Integer operator +(int x, const Integer &y);
std::ostream & operator <<(std::ostream & out, const Integer &a);
std::istream & operator >> (std::istream &input, Integer &a);
```

Listing 130: Integer Class Definition: Constructors

```
#include "integer.h"
#include <iostream>

using namespace std;

Integer::Integer() {                            // standard constructor
    cout << "Integer()" << endl;
    m_pInt = new int(0);
}

Integer::Integer(int value) {                   // paramterizied constructor
    cout << "Integer(int)" << endl;
    m_pInt = new int(value);
}

Integer::Integer(const Integer & obj) {         // copy constructor
    cout << "Integer(const Integer&)" << endl;
    m_pInt = new int(*obj.m_pInt);
}

Integer::Integer(Integer && obj) {              // move constructor
    cout << "Integer(int&&)" << endl;
    m_pInt = obj.m_pInt;
    obj.m_pInt = nullptr;
}
```

```
26
27 Integer::~Integer() {                          // destructor
28     cout << "~Integer()" << endl;
29     delete m_pInt;
30 }
```

Listing 131: Integer Class Definition: Setter and Getter

```
1
2   int Integer::GetValue() const {
3     return *m_pInt;
4   }
5
6   void Integer::SetValue(int value) {
7     *m_pInt = value;
8   }
```

Listing 132: Integer Class Definition: Operator Overlaoding member functiosn

```
1  Integer & Integer::operator++() { // ++a means a.operator++()
2      cout << "operator++()" << endl;
3      ++(*m_pInt);
4      return *this;  // the value a the address of this
5  }
6
7  Integer Integer::operator++(int) { // a++ means a.operator++(int)
8      cout << "operator++(int)" << endl;
9      Integer temp(*this);
10     ++(*m_pInt);
11     return temp;
12 }
13
14 bool Integer::operator==(const Integer & a) const { // c==a means c.operator==(a)
15     cout << "operator==(const Integer &) const" << endl;
16     return *m_pInt == *a.m_pInt;
17 }
18
19 Integer & Integer::operator=(const Integer & a) { // c=a means c.operator=(a)
20     cout << "operator=(const Integer &) const" << endl;
21     if (this != &a) {            // check for self assignemnt
22     delete m_pInt;
23     m_pInt = new int(*a.m_pInt);
24     }
25     return *this;
26 }
27
28 Integer & Integer::operator=(Integer && a) { // c=&&a means c.operator=(&&a)
29     cout << "operator=(Integer &&) const" << endl;
30     if (this != &a) {
31     delete m_pInt;
32     m_pInt = a.m_pInt;
33     a.m_pInt = nullptr;
34     }
35     return *this;
36 }
37
38 Integer Integer::operator+(const Integer & a) const { // c+a means c.operator+(a)
39     cout << "operator+(const Integer &) const" << endl;
40     Integer temp;
41     *temp.m_pInt = *m_pInt + *a.m_pInt; // *this->m_pInt
```

```
42      return temp;
43  }
44
45  void Integer::operator()() {
46      // cout << "operator()()" << endl;
47      cout << *m_pInt << endl;
48  }
```

Listing 133: Integer Class Definition: Global Operator overloading

```
1   Integer operator +(int x, const Integer &y) {
2       // cout << "operator+(int, const Integer)" << endl;
3       Integer temp;
4       temp.SetValue(x + y.GetValue());
5       return temp;
6   }
7   ostream & operator <<(ostream & out, const Integer &a) {
8       // cout << "operator <<(ostream &, const Integer &)" << endl;
9       out << a.GetValue();
10      return out;
11  }
12  istream & operator >> (istream &input, Integer &a) {
13      // cout << "operator >>(istream &, Integer &)" << endl;
14      int x;
15      input >> x;
16      // a.SetValue(x);  // SetValue() allows copy by value, no friend required
17      *a.m_pInt = x;     // access to private member of Integer Class.
18      return input;
19  }
```

7.2 Raw Pointers

- hard to read
- hard to remember when to call delete and assign nullptr.

Listing 134: Main CPP - Raw Pointers

```
1   #include <integer.h>
2   #include <iostream>
3
4   using namespace std;
5
6   void DisplayValue(Integer *p) {
7       if(!p)
8       {
9       return;
10      }
11      cout << p->GetValue() << endl;
12  }
13
14  Integer * GetPointer(int value) {           // Factory function - create an instance of a class
15      Integer *p = new Integer {value};
16      return p;
17  }
18
19  void Operate(int value) {
20
21      Integer *p = GetPointer(value);
22
```

```
23      if(p == nullptr) {
24      p = new Integer {value};
25      }
26      p->SetValue(100);
27      DisplayValue(p);
28      delete p;
29      p = nullptr;
30      p = new Integer {};
31      *p = __LINE__;
32      DisplayValue(p);
33      delete p;
34
35  }
36
37  int main() {
38
39      Operate(5);
40
41      return 0;
42  }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
100
~Integer()
Integer()
Integer(int)
operator=(Integer &&) const
~Integer()
31
~Integer()
```

7.3 std_unique_ptr

- unique pointer has to be used when the underlying pointer does not have to be shared with other parts of the code

Listing 135: Main CPP - Smart Pointers

```
1   #include <integer.h>
2   #include <iostream>
3   #include <memory>
4
5   using namespace std;
6
7   void DisplayValue(Integer *p) {
8       if(!p)
9         return;
10      cout << p->GetValue() << endl;
11  }
12
13  Integer * GetPointer(int value) {           // Factory function - create an instance of a class
14      Integer *p = new Integer {value};
15      return p;
16  }
17
18  void Store(unique_ptr<Integer> &p) {
19      cout << "Storing data into a file: " << p->GetValue() << endl;
```

```
}

void Operate(int value) {

    unique_ptr<Integer> p { GetPointer(value) } ; //unique pointer has explicit constructors
    if(p == nullptr) {
      // p = new Integer {value};
      p.reset(new Integer {value} );
    }
    p->SetValue(100);
    DisplayValue(p.get());   // get the underlying pointer
    p.reset(new Integer {});
    *p = __LINE__;
    DisplayValue(p.get());
    // Store(move(p));          // no copy of unique_ptr
    Store(p);                   // passing by reference is ok.
    *p = 200;
}

int main() {

    Operate(5);
    return 0;
}
```

```
./app
Integer(int)
100
Integer()
~Integer()
Integer(int)
operator=(Integer &&) const
~Integer()
35
Storing data into a file: 35
Integer(int)
operator=(Integer &&) const
~Integer()
~Integer()
```

7.4 Sharing Raw Pointer

Listing 136: Shared raw pointer

```
#include <iostream>
#include <memory>
#include <string>

using namespace std;

class Project{
    string m_Name;
public:
    void SetName(const string &name) {
  m_Name = name;
    }
    void ShowProjectDetails() const {
  cout << "[Project Name] " << m_Name << endl;
    }
```

```
};
class Employee {
    Project *m_pProject {};
public:
    void SetProject(Project *prj) {
  m_pProject = prj;
    }
    const Project * GetProject() const {
  return m_pProject;
    }

};
void ShowInfo(Employee *emp) {
    cout << "Employee project details: ";;;
    emp->GetProject()->ShowProjectDetails();
}
int main() {
    Project *prj = new Project {};
    prj->SetName("Video Decoder");
    Employee *e1 = new Employee {};
    e1->SetProject(prj);
    Employee *e2 = new Employee {};
    e2->SetProject(prj);
    Employee *e3 = new Employee {};
    e3->SetProject(prj);
    prj->ShowProjectDetails();
    ShowInfo(e1); ShowInfo(e2); ShowInfo(e3);
    delete prj; delete e1; delete e2; delete e3;

    return 0;
}
```

```
[Project Name] Video Decoder
Employee project details: [Project Name] Video Decoder
Employee project details: [Project Name] Video Decoder
Employee project details: [Project Name] Video Decoder
```

7.5 Sharing std::unique_ptr

The Requirement here is the Project pointer has to be shared with other projects.

Listing 137

```
#include <iostream>
#include <memory>
#include <string>

using namespace std;

class Project{
    string m_Name;
public:
    void SetName(const string &name) {
  m_Name = name;
    }
    void ShowProjectDetails() const {
  cout << "[Project Name] " << m_Name << endl;
    }
};
class Employee {
```

```
    unique_ptr<Project> m_pProject {};
public:
    void SetProject(unique_ptr<Project> &prj) {
  m_pProject = move(prj);                    // after move the underlaying pointer is not
  ↪  available anay more
    }
    const unique_ptr<Project>& GetProject() const {
  return m_pProject;
    }
};
void ShowInfo(const unique_ptr<Employee> &emp) {
    cout << "Employee project details: ";;;
    emp->GetProject()->ShowProjectDetails();
}
int main() {
    unique_ptr<Project> prj { new Project };
    prj->SetName("Video Decoder");
    unique_ptr<Employee> e1 { new Employee };
    e1->SetProject(prj);                        // the status of prj is lost, moved to m_pProject
    unique_ptr<Employee> e2 { new Employee };
    e2->SetProject(prj);
    unique_ptr<Employee> e3 { new Employee };
    e3->SetProject(prj);

    prj->ShowProjectDetails();
    ShowInfo(e1); ShowInfo(e2); ShowInfo(e3);

    return 0;
}
```

7.6 Sharing std::shared_ptr

The Requirement here is the Project pointer has to be shared with other projects.

Listing 138: Sharing std::shared-ptr

```
#include <iostream>
#include <memory>
#include <string>

using namespace std;

class Project{
    string m_Name;
public:
    void SetName(const string &name) {
  m_Name = name;
    }
    void ShowProjectDetails() const {
  cout << "[Project Name] " << m_Name << endl;
    }
};
class Employee {
    shared_ptr<Project> m_pProject {};
public:
    void SetProject(const shared_ptr<Project> &prj) {
  m_pProject = prj;
    }
    const shared_ptr<Project>& GetProject() const {
  return m_pProject;
    }
};
```

```
27  void ShowInfo(const shared_ptr<Employee> &emp) {
28      cout << "Employee project details: ";;;
29      emp->GetProject()->ShowProjectDetails();
30  }
31  int main() {
32      shared_ptr<Project> prj { new Project };
33      prj->SetName("Video Decoder");
34      shared_ptr<Employee> e1 { new Employee };
35      e1->SetProject(prj);                          // the status of prj is lost, moved to m_pProject
36      shared_ptr<Employee> e2 { new Employee };
37      e2->SetProject(prj);
38      shared_ptr<Employee> e3 { new Employee };
39      e3->SetProject(prj);
40
41      cout << "Reference count: " << prj.use_count() << endl;
42      e3.reset();
43      cout << "Reference count: " << prj.use_count() << endl;
44      e3.reset(new Employee {});
45      e3->SetProject(prj);
46      cout << "Reference count: " << prj.use_count() << endl;
47
48      ShowInfo(e1); ShowInfo(e2); ShowInfo(e3);
49      prj->ShowProjectDetails();
50
51      return 0;
52  }
```

```
Reference count: 4
Reference count: 3
Reference count: 4
Employee project details: [Project Name] Video Decoder
Employee project details: [Project Name] Video Decoder
Employee project details: [Project Name] Video Decoder
[Project Name] Video Decoder
```

7.7 Sharing Weak Ownership

Listing 139: Weak Onwernship

```
1   #include <iostream>
2   #include <memory>
3   #include <string>
4
5   using namespace std;
6
7   class Printer {
8
9       shared_ptr<int> m_pValue {};
10
11  public:
12      void SetValue(shared_ptr<int> p) {
13    m_pValue = p;
14      }
15      void Print() {
16    cout << "Ref count : " << m_pValue.use_count() << endl;
17    cout << "Value is: " << *m_pValue << endl;    // how to detetec if rosource is available?
18      }
19  };
20
21  int main() {
```

```
22
23      Printer prn;
24
25      int num {};
26      cin >> num;
27      shared_ptr<int> p {new int{num}};
28      prn.SetValue(p);
29
30      if(*p > 10) {
31    p = nullptr;      // delete the underlying resource
32      }
33
34      prn.Print();
35
36      return 0;
37  }
```

```
Ref Count : 1
Value is: 11
```

7.8 Sharing std::weak_ptr

Weak Pointer

Weak Pointer helps us pointing weakly to a resource that may or may not be available.

Listing 140: std::weak-ptr

```
1   #include <iostream>
2   #include <memory>
3   #include <string>
4
5   using namespace std;
6
7   class Printer {
8
9       weak_ptr<int> m_pValue {};
10
11  public:
12      void SetValue(weak_ptr<int> p) {
13    m_pValue = p;
14      }
15      void Print() {
16    if(m_pValue.expired()) {   // detect if resource is available or not!!!
17        cout << "Resource is no longer available" << endl;
18        return;
19    }
20    auto sp = m_pValue.lock();  // returns a shared pointer + inr. ref counter
21    cout << "Ref count : " << sp.use_count() << endl;
22    cout << "Value is: " << *sp << endl;
23      }
24  };
25
26  int main() {
27      Printer prn;
28      int num {};
29      cin >> num;
30      shared_ptr<int> p {new int{num}};
31      prn.SetValue(p);
```

```
32
33      if(*p > 10) {
34    p = nullptr;     // delete the underlying resource
35      }
36
37      prn.Print();
38      return 0;
39  }
```

```
Ref count : 2
Value is: 2
```

7.9 Circular References

Listing 141: Circular References with Raw Pointers

```
1   #include <iostream>
2   #include <memory>
3   #include <string>
4
5   using namespace std;
6
7   class Employee;
8
9   class Project{
10
11  public:
12      Employee * m_emp;
13      Project() {
14      cout << "Project()" << endl;
15      }
16      ~Project() {
17      cout << "~Project()" << endl;
18      }
19  };
20
21  class Employee {
22
23  public:
24      Project * m_prj;
25      Employee() {
26      cout << "Employee()" << endl;
27      }
28      ~Employee() {
29      cout << "~Employee()" << endl;
30      }
31  };
32
33  int main() {
34        Project  *prj = new Project {};
35        Employee *emp = new Employee {};
36
37        emp->m_prj = prj;
38        prj->m_emp = emp;
39
40  delete emp;
41  delete prj;
42
43      return 0;
44  }
```

```
Project()
Employee()
~Employee()
~Project()
```

Listing 142: Circular References with Shared Pointers

```
#include <iostream>
#include <memory>
#include <string>

using namespace std;

class Employee;

class Project{

public:
    shared_ptr<Employee> m_emp;
    Project() {
    cout << "Project()" << endl;
    }
    ~Project() {
    cout << "~Project()" << endl;
    }
};

class Employee {

public:
    shared_ptr<Project> m_prj;
    Employee() {
    cout << "Employee()" << endl;
    }
    ~Employee() {
    cout << "~Employee()" << endl;
    }
};

int main() {
      shared_ptr<Project>  prj {new Project {}};
      shared_ptr<Employee> emp {new Employee {}};

      emp->m_prj = prj;
      prj->m_emp = emp;

    return 0;
}
```

```
Project()
Employee()
```

Circualr References

Leads to memory leaks, since boths shared pointers point to each other when functions goes out of scope (Reference cout 2).

Listing 143: Circular References with Weak Pointer

```
#include <iostream>
#include <memory>
#include <string>

using namespace std;

class Employee;

class Project{

public:
    shared_ptr<Employee> m_emp;
    Project() {
  cout << "Project()" << endl;
    }
    ~Project() {
  cout << "~Project()" << endl;
    }
};

class Employee {

public:
    weak_ptr<Project> m_prj;
    Employee() {
  cout << "Employee()" << endl;
    }
    ~Employee() {
  cout << "~Employee()" << endl;
    }
};

int main() {
      shared_ptr<Project>  prj {new Project {}};
      shared_ptr<Employee> emp {new Employee {}};

      emp->m_prj = prj;
      prj->m_emp = emp;

    return 0;
}
```

```
Project()
Employee()
~Project()
~Employee()
```

7.10 Deleter



Deleter

Allowing to manage resources that may not have been acquired using **new**.

Listing 144: legacy code

```cpp
#include <iostream>
#include <memory>
#include <string>

using namespace std;

int main() {
    int * p = (int *) malloc(4);
    *p = 100;
    cout << *p << endl;
    free(p);

    return 0;
}
```

```
100
```

Listing 145: smart poiner

```cpp
#include <iostream>
#include <memory>
#include <string>

using namespace std;

int main() {
    unique_ptr<int> p {(int*) malloc(4)};
    *p = 100;
    cout << *p << endl;

    return 0;
}
```

```
100
```



Undefined Behaviour

To release the memory malloc requires **free** but a smart pointer calls by default **delete**. As deleter callback any kind of callable could be used: functions, function objects or delegates. The prefered usage are function objects since function increase the size of the smart pointer object.

Listing 146: unique-ptr deleter callbak: deleter callbak

```cpp
#include <iostream>
#include <memory>
#include <string>

using namespace std;

// defining a function object
struct Free {

    // required to define function call operator
    // argument is the type of pointer to be released
    void operator()(int *p) {
  free(p);
  cout << "operator()(int *p)" << endl;
    }
```

```
};
// argument is the type of pointer to be released
void MallocFree(int *p) {
    free(p);
    cout << "MallocFree(int *p)" << endl;
}
int main() {
    // unique_ptr<int, Free> p {(int*) malloc(4), Free{}};              // function object
    unique_ptr<int, void(*)(int *)> p {(int*) malloc(4), MallocFree};   // functions
    *p = 100;
    cout << *p << endl;

    return 0;
}
```

```
100
MallocFree(int *p)
```

Listing 147: shared-ptr deleter callbak: deleter callbak

```
#include <iostream>
#include <memory>
#include <string>

using namespace std;

// defining a function object
struct Free {
    // required to define function call operator
    // argument is the type of pointer to be released
    void operator()(int *p) {
  free(p);
  cout << "operator()(int *p)" << endl;
    }
};
// argument is the type of pointer to be released
void MallocFree(int *p) {
    free(p);
    cout << "MallocFree(int *p)" << endl;
}
void UniquePtrDeleters() {
    // unique_ptr<int, Free> p {(int*) malloc(4), Free{}};              // function object
    unique_ptr<int, void(*)(int *)> p {(int*) malloc(4), MallocFree};   // functions
    *p = 100;
    cout << *p << endl;
    // free(p);
}
void SharedPtrDeleters() {
    shared_ptr<int> p {(int*) malloc(4), Free{}};              // function object
    // shared_ptr<int> p {(int*) malloc(4), MallocFree};   // functions
    *p = 100;
    cout << *p << endl;
    // free(p);
}
int main() {
    SharedPtrDeleters();

    return 0;
}
```

```
100
operator()(int *p)
```

7.11 Dynamic Arrays

Listing 148: Dynamic Array Problem

```
#include <iostream>
#include <cstdlib>
#include <memory>
#include <string>

using namespace std;


int main() {

// Problem 1: default deleter calls delete and not delete[]
    unique_ptr<int> p {new int [5] {1,2,3,4,5}};

// Problem 2: array syntax not available
    // p[0] = 10;                // not valid
    p.get()[0] = 10;             // access is a bit unconvenient

// Problem 3: do not use dynamic array with shared_ptr

    return 0;
}
```

Listing 149: Dynamic Array - Smart Pointer Type Specialisation

```
#include <iostream>
#include <cstdlib>
#include <memory>
#include <string>

using namespace std;


int main() {

    // unique_ptr<int[]> p {new int [5] {1,2,3,4,5}};
    shared_ptr<int[]> p {new int [5] {1,2,3,4,5}};   // use int[] as type
    p[0] = 10;

    return 0;
}
```

7.12 Make Functions

Modern C++ discourages manual memory management. We should avoid using new and delete operators directly in our code, but if we want to create a smart pointer we have to use new to allocate memory on the heap.

Listing 150: Make Functions Shared Pointer

```
#include <iostream>
#include <cstdlib>
#include <memory>
#include <string>

using namespace std;

class Point {
public:
    Point(int x, int y) {
      cout << "Point(" << x << "," << y << ")" << endl;
    }
};
int main() { // without using now to allocate resources for underlaying pointer
    shared_ptr<int> p1{new int{5}};

    // creates the integer object on the heap and initializes it with 5.
    auto p2 = make_shared<int>(5);
    // samrt pointer to custom type
    auto pt = make_shared<Point>(7,200); // pass the arguments to the constructor
    // Dynamic Array - REQUIRES c++20
    auto pArr = make_shared<int[]>(5);
    pArr[0] = 100;
    return 0;
}
```

Listing 151: Make Functions Unique Pointer

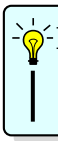
```
#include <iostream>
#include <cstdlib>
#include <memory>
#include <string>

using namespace std;

class Point {
public:
    Point(int x, int y) {
      cout << "Point(" << x << "," << y << ")" << endl;
    }
};
int main() { // without using now to allocate resources for underlaying pointer
    unique_ptr<int> p1{new int{5}};

    // creates the integer object on the heap and initializes it with 5.
    auto p2 = make_unique<int>(5);
    // samrt pointer to custom type
    auto pt = make_unique<Point>(7,200); // pass the arguments to the constructor
    // Dynamic Array
    auto pArr = make_unique<int[]>(5);
    pArr[0] = 100;

    return 0;
}
```



Macker Shared

make_shared saves muiltiple calls to **new** and **delete**. Uses only one call to allocate memory for the resource itself as well as for the control block.

8 More C++ Goodies

8.1 ENUMS- Part I (Basics)

Listing 152: Using Macros or Const Integers

```
#include <iostream>

#define RED 0
const int GREEN = 1;

  using namespace std;

void FillColor(int color) {
    // Fill background with some color
    if (color == 0) {
  // paint with red
    }
    else if(color == 1) {
  // paint with green
    }
}
int main() {

    FillColor(RED);
    FillColor(GREEN);
    FillColor(6); //!!!

    return 0;
}
```

Enumerated Type

- Type created throuhg *enum* keyword
- Created with restricted range of values, called symbolic constants or enumertors
- Enumerators are internally represented as undefined integral types
- Can implicitly convert to an integer, but not the other way round
- Default value start from 0, but users can assign any value
- Enumerators are visible in the scope in which they're defined

  enum Color(Red, Green, Blue); Color c = Red; c = 1; \//Compiler error, use static_cast to convert c = static_cast<Color>(1); int x = Green; \//x will contain 1

8.2 TODO ENUMS - Part II (Scoped Enums C++11)

8.3 TODO Strings - Part I (Raw Strings)

8.4 TODO Strings - Part II (std::strings)

8.5 Assignment I

8.6 Strings - Part III (String Streams)

8.7 TODO User-Defined Literals

8.8 Constant Expression - constexpr

8.9 TODO std::initializer_list (C++11)

8.10 TODO Dynamic Array (std::vector)

8.11 TODO Assignment III

8.12 TODO Union - I

8.13 TODO Union - II

9 Object Oriented Programming

9.1 Access Modifiers

modifier	Meaning
private	Inaccessible outside the class
public	Accesible outside the class
protected	Accessible only to class and its child classes

Class Base	Class Child : public Base	
private: MemberA	private: MemberA	not accessible in the Child
public: MemberB	public: MemberB	accessilbe
protected: MemberC	protected: MemberC	accessible

Class Base	Class Child : private Base	
private: MemberA	private: MemberA	not accessible in the Child
public: MemberB	private: MemberB	accessible
protected: MemberC	private: MemberC	accessible

Class Base	Class Child : protected Base	
private: MemberA	private: MemberA	not accessible in the Child
public: MemberB	protected: MemberB	accessible
protected: MemberC	protected: MemberC	accessible

If no modifier is used for inheritance than the default modifier is used:

- *public* if the child is structure
- *private* if the child is a class

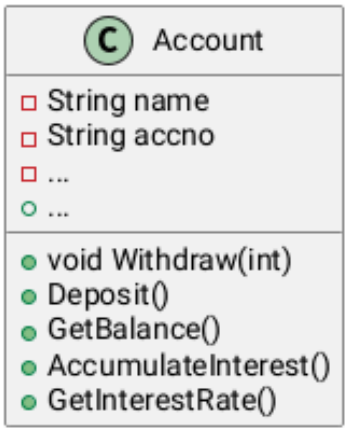
independant wheather the base is a struct or a class.
Object Construction

- Constructors execute from base to child
- Destructors execute from child to base
- Base data members will be part of child object

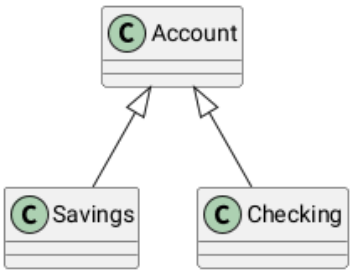
9.2 Project - I Introduction to Banking Application

- Manage accounts
- Customers can perform common operations
- Bank can perfomr administrative tasks
- Represent account through Account class
- Common implementation for all account types

Listing 153: Account Class



Listing 154: Banking Application



9.3 Project - II (Account Class)

9.3.1 Class Declaration

Listing 155: Class Account

```
#include <string>

class Account {
private:
    std::string m_Name;
    float m_Balance;
    int m_AccNo;
    static int s_ANGenerator; // account number generator
public:
    Account(const std::string &name, float balance);
    ~Account();
    const std::string GetName()const;
    float GetBalance() const;
    int GetAccountNo() const;

    void AccumulateInterest();
    void Withdraw(float amount);
    void Deposit(float amount);
    float GetInterestRate() const;
};
```

Listing 156: Class Checking

```
#include "Account.h"
```

```
class Checking : public Account {
private:
    float m_MinimumBalance {50};
public:
    // Constructor Inheritance - inherits all constructor of the base
    using Account::Account;
    // Constructor with 2 Parameters is obsolete due to the constructor inheritance
    // Checking(const std::string &name, float balance);
    Checking(const std::string &name, float balance, float minbalance);
    ~Checking();
    void Withdraw(float amount) override;
    float GetMinimumBalance()const;
};
```

Listing 157: Transaction

```
#pragma once
#include "Account.h"
void Transact(Account *pAccount);
```

9.3.2 Class Definition

Listing 158: Class Account

```
#include "Account.h"
#include <iostream>

 int Account::s_ANGenerator = 1000;

Account::Account(const std::string &name, float balance) :
    m_Name(name), m_Balance(balance){
    m_AccNo = ++s_ANGenerator;
    //std::cout << "Account(const std::string &, float)" << std::endl;
}
Account::~Account() {
    //std::cout << "~Account()" << std::endl;
}
const std::string Account::GetName() const {
    return m_Name;
}
float Account::GetBalance() const {
    return m_Balance;
}
int Account::GetAccountNo() const {
    return m_AccNo;
}
void Account::AccumulateInterest() {
}
void Account::Withdraw(float amount) {
    /*
      Balance should be greater than 0 & the amount
      to withdraw should be less than balance
    */
    if (amount < m_Balance && m_Balance > 0) {
        m_Balance -= amount;
    }
    else {
        std::cout << "Insufficient balance" << std::endl;
    }
}
void Account::Deposit(float amount) {
```

```
38      m_Balance += amount;
39  }
40  float Account::GetInterestRate() const {
41      return 0.0f;
42  }
```

9.3.3 Main

Listing 159: Main CPP - Banking Application

```
1   #include <iostream>
2   #include "Account.h"
3
4   using namespace std;
5   int main()
6   {
7       Account acc("Bob", 1000);
8       cout << "Initial balance: " << acc.GetBalance() << endl;
9       acc.Deposit(200);
10      acc.Withdraw(380);
11      cout << "Balance: " << acc.GetBalance() << endl;
12
13      return 0;
14  }
```

9.3.4 Make / Run

```
./app
Initial balance: 1000
Balance: 820
```

9.4 Project - II (Savings Class - Inheritance)

The account class itself is not of much use to us unless we inherit this class into specific Account classes. Let's go ahead and add an Accout class called Savings.

Listing 160: Savings.h

```
1   #include "Account.h"
2   class Savings : public Account {
3   private:
4       float m_Rate; // interest rate accumlate the interest into the Savings account
5   public:
6       Savings(const std::string &name, float balance, float rate);
7       ~Savings();
8       float GetInterestRate() const ;
9       void AccumulateInterest() ;
10  };
```

Listing 161: Savings.cpp

```
1   #include "Savings.h"
2   #include <iostream>
3
4   Savings::Savings(const std::string & name, float balance, float rate)
5       : Account(name, balance), m_Rate(rate) {
6   // initializer list calling Constructor of Base Class
7   }
8   Savings::~Savings() {
9       //std::cout << "~Savings()" << std::endl;
```

```
}
float Savings::GetInterestRate() const {
    return m_Rate;
}
void Savings::AccumulateInterest() {
    m_Balance += (m_Balance * m_Rate);
}
```

Listing 162: Main CPP - Banking Application

```
#include <iostream>
#include "Savings.h"

using namespace std;
int main()
{
    Savings acc("Bob", 1000, 0.5f);
    cout << "Initial balance: " << acc.GetBalance() << endl;
    acc.Deposit(200);
    acc.AccumulateInterest();
    acc.Withdraw(380);
    cout << "Balance: " << acc.GetBalance() << endl;

    return 0;
}
```

```
./app
Initial balance: 1000
Balance: 1420
```

9.5 Project - III (Checking Class - Constructor Inheritance)

Checking class asures no withdraw when balance is below a minimum. Default is 50.
In Checking class we are going to use **Constructor Inheritance**, thus all constructors are inherited from base class.

Listing 163: Checking.h

```
#include "Account.h"

class Checking : public Account {

private:
    float m_MinimumBalance {50};
public:
    // Constructor Inheritance
    using Account::Account;

    Checking(const std::string &name, float balance, float minbalance);
    ~Checking();
    void Withdraw(float amount);
    float GetMinimumBalance()const;
};
```

Listing 164: Checking.cpp

```
#include "Checking.h"
#include <iostream>

```

```
Checking::~Checking() {
}

Checking::Checking(const std::string &name, float balance, float minbalance)
    : Account(name, balance), m_MinimumBalance(minbalance) {
}
void Checking::Withdraw(float amount) {
    if ((m_Balance - amount) > m_MinimumBalance) {
        Account::Withdraw(amount);          // call base class member function
    }
    else {
        std::cout << "Invalid amount" << std::endl;
    }
}
float Checking::GetMinimumBalance() const {
    return m_MinimumBalance;
}
```

Listing 165: Main CPP - Banking Application

```
#include <iostream>
#include "Checking.h"

using namespace std;
int main()
{
    Checking acc("Bob", 1000);  // invokes the inherited base class constructor
    cout << "Initial balance: " << acc.GetBalance() << endl;
    acc.Withdraw(980);
    cout << "Balance: " << acc.GetBalance() << endl;

    return 0;
}
```

```
./app
Initial balance: 1000
Invalid amount
Balance: 1000
```

Constructor Inheritance

All the constructors of the base can be inherited with the using directive e.g. *using Account::Account;* in the class delcalration of the child class. The according constructor is therefore invoked automatically.

9.6 Project - IV (Transaction Class)

Listing 166: Transaction.h

```
#include "Checking.h"
void Transact(Checking *pAccount);
```

Listing 167: Transaction.cpp

```
#include <iostream>
#include "Transaction.h"

```

```
void Transact(Checking * pAccount) {
    std::cout << "Transaction started" << std::endl;
    std::cout << "Initial balance:" << pAccount->GetBalance() << std::endl;
    pAccount->Deposit(100);
    pAccount->AccumulateInterest();
    pAccount->Withdraw(170);

    std::cout << "Interest rate: " << pAccount->GetInterestRate() << std::endl; // polymorphical call
    std::cout << "Final balance: " << pAccount->GetBalance() << std::endl;
}
```

Listing 168: Main CPP - Banking Application

```
#include <iostream>
#include "Transaction.h"

using namespace std;
int main()
{
    Checking acc("Bob", 100);  // invokes the inherited base class constructor
    Transact(&acc);

    return 0;
}
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I./inc ./src/main.cpp ./src/Account.cpp ./src/Sav
./app
Transaction started
Initial balance:100
Invalid amount
Interest rate: 0
Final balance: 200
```

Problem

Transaction class is hardcoded to perform transactions only on Checking account because that's specified as argument in Transaction().

9.7 Project - IV (Transaction Class - Base Class Argument)

Listing 169: Transaction.h

```
#include "Account.h"
void Transact(Account *pAccount);
```

Listing 170: Transaction.cpp

```
#include <iostream>
#include "Transaction.h"

void Transact(Account * pAccount) {
    std::cout << "Transaction started" << std::endl;
    std::cout << "Initial balance:" << pAccount->GetBalance() << std::endl;
    pAccount->Deposit(100);
    pAccount->AccumulateInterest();
    pAccount->Withdraw(170);
```

```
10
11      std::cout << "Interest rate: " << pAccount->GetInterestRate() << std::endl; // polymorphical call
12      std::cout << "Final balance: " << pAccount->GetBalance() << std::endl;
13  }
```

Listing 171: Main CPP - Banking Application

```
1   #include <iostream>
2   #include "Transaction.h"
3   #include "Savings.h"
4
5   using namespace std;
6   int main()
7   {
8       Savings acc("Bob", 100, 0.5f);  // invokes the inherited base class constructor
9       Transact(&acc);
10
11      return 0;
12  }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I./inc ./src/main.cpp ./src/Account.cpp ./src/Sav
./app
Transaction started
Initial balance:100
Interest rate: 0
Final balance: 30
```

Problem

AccumulateInterest not executed and Interest rate still 0.

9.8 Project - IV (Transaction Class - Virtual Keyword)

Polymorphism

Polymorphism is enabled through the keyword virtual. This allows the function evaluation for late binding.

Listing 172: Account.h

```
1   #ifndef ACCOUNT_H
2   #define ACCOUNT_H
3
4   #include <string>
5
6   class Account {
7   private:
8       std::string m_Name;
9       int m_AccNo;
10      static int s_ANGenerator; // account number generator
11
12  protected:
13      float m_Balance; // protected members can be accessed by child classes
14
15  public:
```

```
16      Account(const std::string &name, float balance);
17      ~Account();
18
19      const std::string GetName() const;
20      float GetBalance() const;
21      int GetAccountNo() const;
22
23      virtual void AccumulateInterest();
24      virtual void Withdraw(float amount);
25      void Deposit(float amount);
26      virtual float GetInterestRate() const;
27  };
28
29  #endif // ACCOUNT_H
```

Listing 173: Main CPP - Banking Application

```
1   #include <iostream>
2   #include "Transaction.h"
3   #include "Savings.h"
4
5   using namespace std;
6   int main()
7   {
8       Savings acc("Bob", 100, 0.05f);  // invokes the inherited base class constructor
9       Transact(&acc);
10
11      return 0;
12  }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I./inc ./src/main.cpp ./src/Account.cpp ./src/Sav
./app
Transaction started
Initial balance:100
Interest rate: 0.05
Final balance: 40
```

Polymorphism

Polymorphism is enabled through the keyword virtual. This allows the function evaluation for late binding.

9.9 Project - VI (Virtual Mechanism Internals I)

Polymorphism

- Different forms of the function are provided
- The call is resloved at compile time or runtime
- *Runtime polymorphism* or *dynamic binding*
- Implemented through virtual mechanism
- Compiler inserts code to invoke the correct function at **runtime**
- Automatically generated through the **virtual keyword**
- Such functions are called **polymorphic functions**

- Should be invoked only through a pointer or a reference

Vtable

- Array of function pointers called as **virtual table**
- contains addresses of the virtual functions

Vptr

- Virtual poiner member of the class (automatically generated)
- Points to the Vtable of the corresponding class
- The **Vptr** is inherited and points to the **Vtable** of the inherited class

9.10 Project - VI (Virtual Mechanism Internals II)



Destructor

The base class should always have a **virtual destructor**.

9.11 Project - VII (override & final specifier C++11)



Final

If a class is not meant to be inherited, then the class can be marked as final.
class NoInheritClas final {};



Final

If a class is not meant to be inherited, then the class can be marked as final.
class NoInheritClas final {};



Override

Use override to indicate you are going to override a virtual function which has therfore to mach in name and signature of the function.
void Serialize(float version) override;
This gives an compiler error if

- no virtual function in base class exists
- function name is different
- signature of the function is different



Override Final

Override Final indicates child classes can not override this function anymore
void Serialize(float version) override final;
This function can not be overriden in a child class.

9.12 Project - VIII (Object Slicing)

Listing 174: Main CPP - Upcast

```cpp
#include <iostream>
#include "Savings.h"
#include "Checking.h"
#include "Transaction.h"

using namespace std;
int main()
{
    Checking ch("Bob", 100);
    Account *pAccount = &ch;  // Upcasting: Upcast a child class object ot a base class object
                              // Through the base class object, can the memeber functions of
                              // the child class invoked.
                              // works only for poiners resp. references

    Account myAccount = ch;   // Object Slicing!!

    return 0;
}
```

Object Slicing

- A Situation in which the compiler deliberately removes some part of an object
- Occurs when a child class object is assigned to a concrete base class object
- The size of a child class object may be more than its base class
- While assigning it to a base object, some part of the memory may get overwritten
- This will lead to **memory corruption**
- To **prevent** this, the **compiler slices** the child object, effectively removing the memory that conatains it attributes
- The object is then copied into the base object.



Access Child Class Member Function

Ensure the base class object is either a pointer or a reference.

Listing 175: Main CPP - Down Cast

```cpp
#include <iostream>
#include "Savings.h"
#include "Checking.h"
#include "Transaction.h"

using namespace std;
int main()
{
    Checking ch("Bob", 100);
    Account *pAccount = &ch;

    // Checking *pChecking = pAccount;   // gives an error
    Checking *pChecking = static_cast<Account*>(pAccount);   // gives an error

    return 0;
}
```

9.13 Project - IX (typed Operator)

Listing 176: Main CPP - typeid Function

```
#include <iostream>
#include "Savings.h"
#include "Checking.h"
#include "Transaction.h"
#include <typeinfo>

using namespace std;
int main()
{
    Savings ch("Bob", 100, 60);
    Account *p = &ch;
    // Transact(&ch);

    const type_info &ti = typeid(*p);

    if(ti == typeid(Savings)) {
    cout << "p points to savings object" << endl;
    }
    else {
    cout << "Not pointing to savings object" << endl;
    }
    return 0;
}
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/Account.cpp ../src/Sa
./app
p points to savings object
```

typeid

typeid will work at runtime on polymorphic types anf for non polymorpic types, it determines the type at compile type.

9.14 Project - X (dynamic_cast operator)

Listing 177: Main CPP - Dynamic Cast Operator

```
#include <iostream>
#include "Savings.h"
#include "Checking.h"
#include "Transaction.h"
#include <typeinfo>

using namespace std;
int main()
{
    Savings ch("Bob", 100, 60);
    // Checking ch("Bob", 100, 60);
    Transact(&ch);

    return 0;
}
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/Account.cpp ../src/Sa
./app
Transaction started
Initial balance:100
Minimum balance of Checking:60
Invalid amount
Interest rate: 0
Final balance: 200
```



RTTI

During compile time, the compiler creates the type information of the object as a type_info object, and that is stored with the class vtable at runtime.
When we use type id or dynamic cast, these operators will query that information. So RTTI will impose overhead on the program, especially dynamic_cast.
Avoid RTTI instead rely on polymorphism so that the corect behaviour is invoked polymorphic without having to know the type of the object.

9.15 Abstract Class and pure virtual Functions

pure virtual Function

- *virtual void Serialize(float version) = 0*
- *= 0* is know as the **pure specifier**
- does not have an implementation (optional)
- cannot be invoked (except by derived classes if defined)
- no entry in the **vtable**
- must be overriden by the derived classes
- if it is not overriden the child class becomes also a pure virtual function

Abstract Class

- has at least **one pure virtual function**
- can contain other members
- can **not be instantiated**, but used through a pointer or reference
- Establishes a contract with clients
- Used for creating **interfaces**

9.16 TODO Multiple (Dyamond) Inheritance

10 Exception Handling

10.1 Basic Exception Handling - Part I

Exception Handling

- Mechanism to handle errors in programs that occur at runtime
- These errors are called exceptions
- Exist outside the normal functioning of the program
- Require immediate handling by the program
- If not handled, the program crashes
- Cannot be ignored, unlike C error handling

Listing 178: C-Error Handling

```
#include <iostream>
#include <limits>

using namespace std;

int ProcessRecords(int count) {

    int *pArray = (int*)malloc(count * sizeof(int));
    if(pArray == nullptr) {
        cout << "Failed to allocate memmory" << endl;
        return -1;
    }
    for(int i = 0; i < count; i++) {
        pArray[i] = i;;
    }
    free(pArray);
    return 0;
}

int main() {

    ProcessRecords(numeric_limits<int>::max());  // return type can be igonerd
    return 0;
}
```

Mechanism

- **try**
  - creates a scope/block & exception causing code appears here
  - can contain other try-catch statements
- **throw**
  - throws an exception from the try block
  - exception is object that is constructed in throw statements
- **catch**
  - handler that catches the exception object
  - should appear just after the try block
  - multiple catch blocks can exist

Listing 179: Exception Handler

```
#include <iostream>
#include <limits>

using namespace std;

void ProcessRecords(int count) {

    int *pArray = (int*)malloc(count * sizeof(int));
    if(pArray == nullptr) {
        throw runtime_error("Failed to allocate memmory");
    }
    for(int i = 0; i < count; i++) {
        pArray[i] = i;;
    }
    free(pArray);
}

int main() {

    try {
        ProcessRecords(numeric_limits<int>::max());
    }
    catch (runtime_error &ex) {
        cout << ex.what() << endl;

    }
    return 0;
}
```

10.2 Basic Exception Handling - Part II (multiple Catch Blocks)

Listing 180: Exception Handler

```
#include <iostream>
#include <limits>

using namespace std;

void ProcessRecords(int count) {

    if (count < 10) {
        throw out_of_range("Count should be greater than 10");
    }
    int *p = new int[count]; // throws an exception of type bad_alloc
    (void)p;
    int *pArray = (int*)malloc(count * sizeof(int));
    if(pArray == nullptr) {
        throw runtime_error("Failed to allocate memmory");
    }
    for(int i = 0; i < count; i++) {
        pArray[i] = i;;
    }
    free(pArray);
}

int main() {

    try {
```

```
26          // ProcessRecords(numeric_limits<int>::max());
27          ProcessRecords(9);
28      }
29      // catch (runtime_error &ex) {
30      //  cout << ex.what() << endl;
31      // }
32      // catch(out_of_range &ex) {
33      //  cout << ex.what() << endl;
34      // }
35      // catch(bad_alloc &ex) {
36      //  cout << ex.what() << endl;
37      // }
38      catch(exception &ex) {
39          cout << ex.what() << endl;
40      }
41      catch(...) {                            // catche also undocumented exceiption
42          cout << "Exception" << endl;
43      }
44
45      return 0;
46  }
```

```
Exception
```

10.3 Basic Exception Handling - Part III (Stack unwinding)

Listing 181: Exception Handler - objects on Stack

```
1   #include <iostream>
2   #include <limits>
3
4   using namespace std;
5
6   class Test {
7   public:
8       Test() {cout << "Test(): Acquire resources" << endl;};
9       ~Test() {cout << "~Test(): Release resources" << endl;};
10  };
11
12  void ProcessRecords(int count) {
13      Test t;
14      if (count < 10) {
15          throw out_of_range("Count should be greater than 10");
16      }
17      int *p = new int[count]; // throws an exception of type bad_alloc
18      (void)p;
19      int *pArray = (int*)malloc(count * sizeof(int));
20      if(pArray == nullptr) {
21          throw runtime_error("Failed to allocate memmory");
22      }
23      for(int i = 0; i < count; i++) {
24          pArray[i] = i;;
25      }
26      free(pArray);
27      delete[] p;
28  }
29
30  int main() {
31
32      try {
33          // ProcessRecords(numeric_limits<int>::max());
```

```
34          ProcessRecords(9);
35      }
36      catch(exception &ex) {
37          cout << ex.what() << endl;
38      }
39      catch(...) {                            // catche also undocumented exceiption
40          cout << "Exception" << endl;
41      }
42
43      return 0;
44  }
```

```
Test(): Acquire resources
~Test(): Release resources
Count should be greater than 10
```

Listing 182: Exception Handler - objects on Heap

```
1   #include <iostream>
2   #include <limits>
3
4   using namespace std;
5
6   class Test {
7   public:
8       Test() {cout << "Test(): Acquire resources" << endl;};
9       ~Test() {cout << "~Test(): Release resources" << endl;};
10  };
11
12  void ProcessRecords(int count) {
13      Test *t = new Test {};
14      if (count < 10) {
15          throw out_of_range("Count should be greater than 10");
16      }
17      int *p = new int[count]; // throws an exception of type bad_alloc
18      (void)p;
19      int *pArray = (int*)malloc(count * sizeof(int));
20      if(pArray == nullptr) {
21          throw runtime_error("Failed to allocate memmory");
22      }
23      for(int i = 0; i < count; i++) {
24          pArray[i] = i;;
25      }
26      free(pArray);
27      delete[] p;
28      delete t;     // after an exception the memory is not going to be freed!!!!
29  }
30
31  int main() {
32
33      try {
34          // ProcessRecords(numeric_limits<int>::max());
35          ProcessRecords(9);
36      }
37      catch(exception &ex) {
38          cout << ex.what() << endl;
39      }
40      catch(...) {                            // catche also undocumented exceiption
41          cout << "Exception" << endl;
42      }
```

```
43
44      return 0;
45  }
```

```
Test(): Acquire resources
Count should be greater than 10
```

Destructor of ~Test() is not called here!



Stack Unwinding

Stack unwinding will destroy only local objects. If any objects have been created on the heap, they will not be destroyed and infact the memory can be leaked.

Listing 183: Exception Handler - objects on Heap with Smart Pointer (RAII)

```
1   #include <iostream>
2   #include <limits>
3   #include <memory>
4   #include <vector>
5
6   using namespace std;
7
8   class Test {
9   public:
10      Test() {cout << "Test(): Acquire resources" << endl;};
11      ~Test() {cout << "~Test(): Release resources" << endl;};
12  };
13
14  void ProcessRecords(int count) {
15      unique_ptr<Test> t{new Test{}};
16      if (count < 10) {
17          throw out_of_range("Count should be greater than 10");
18      }
19      vector<int> p;
20      p.reserve(count);     // throws an exception of type bad_alloc
21
22      vector<int> pArray;
23      p.reserve(count);     // throws an exception of type bad_alloc
24
25      for(int i=0; i<count; ++i) {
26          pArray.push_back(i);
27          cout << pArray[i] <<", ";
28      }
29      cout << endl;
30  }
31
32  int main() {
33
34      try {
35          // ProcessRecords(numeric_limits<int>::max());
36          ProcessRecords(9);
37      }
38      catch(exception &ex) {
39          cout << ex.what() << endl;
40      }
41      catch(...) {                          // catche also undocumented exceiption
42          cout << "Exception" << endl;
43      }
44
```

```
45     return 0;
46 }
```

```
Test(): Acquire resources
~Test(): Release resources
Count should be greater than 10
```

Avoid Manual Resource Management

Use Smart Pointers and standard containers instead.

10.4 Basic Exception Handling - Part IV (Nested Exceptions)

Listing 184: Exception Handler - Nested Exceptions

```
1  #include <iostream>
2  #include <limits>
3  #include <memory>
4  #include <vector>
5  #include <random>
6
7  using namespace std;
8
9  class Test {
10 public:
11     Test() {cout << "Test(): Acquire resources" << endl;};
12     ~Test() {cout << "~Test(): Release resources" << endl;};
13 };
14
15 void ProcessRecords(int count) {
16     vector<int> pArray;
17     p.reserve(count);    // throws an exception of type bad_alloc
18     for(int i=0; i<count; ++i) {
19         pArray.push_back(i);
20     }
21     default_random_engine eng;
22     bernoulli_distribution dist;
23     int errors {};
24     for(const auto & element : pArray) {
25         try {
26             cout << "Processing record #: "<< element;
27             if(!dist(eng)) {
28                 ++errors;
29                 throw runtime_error("Could not process the record");
30             }
31             cout << endl;
32         }
33         catch (runtime_error &ex) {
34             cout << " [ERROR " << ex.what() << "]" << endl;
35             if(errors > 3) {
36                 runtime_error err("Too many errors. Abandoning operation");
37                 ex = err;
38                 throw; // rethrow exception
39             }
40         }
41     }
42     cout << endl;
43 }
```

```
44
45 int main() {
46
47     try {
48         // ProcessRecords(numeric_limits<int>::max());
49         ProcessRecords(19);
50     }
51     catch(exception &ex) {
52         cout << ex.what() << endl;
53     }
54     return 0;
55 }
```

```
Test(): Acquire resources
Processing record #: 0
Processing record #: 1
Processing record #: 2
Processing record #: 3 [ERROR Could not process the record]
Processing record #: 4 [ERROR Could not process the record]
Processing record #: 5 [ERROR Could not process the record]
Processing record #: 6
Processing record #: 7 [ERROR Could not process the record]
~Test(): Release resources
Too many errors. Abandoning operation
```



Nested Exception Handling

Gives us more control and how the exception should be handled.

10.5 Basic Exception Handling - Part V (Constructor & Destructor)

Listing 185: Exception Handler - Constructor & Destructor

```
1  #include <iostream>
2  #include <limits>
3  #include <memory>
4  #include <vector>
5  #include <random>
6
7  using namespace std;
8
9  class A{
10 public:
11     A() {cout << "A()" << endl;}
12     ~A() {cout << "~A()" << endl;}
13 };
14 class B{
15 public:
16     B() {cout << "B()" << endl;}
17     ~B() {cout << "~B()" << endl;}
18 };
19
20
21 class Test {
22     A *pA{};
23     B b{};      // calls destructor when exception
24     int *pInt{};
```

```
25      char *pStr{};
26      int *pArr{};
27  public:
28      Test() {
29      cout << "Test(): Acquire resources" << endl;
30      pA = new A;
31      pInt = new int;
32      throw runtime_error("Failed to initialize");
33      pStr = new char[1000];
34      pArr = new int[50000];
35      };
36      ~Test() {
37      cout << "~Test(): Release resources" << endl;
38      delete pA;
39      delete pInt;
40      delete [] pStr;
41      delete [] pArr;
42      };
43  };
44
45  int main() {
46      try {
47      Test t;
48      }
49      catch (runtime_error &ex) {
50      cout << ex.what();
51      }
52      return 0;
53  }
```

```
B()
Test(): Acquire resources
A()
~B()
Failed to initialize
```

Listing 186: Exception Handler - Constructor & Destructor with smart pointer

```
1   #include <iostream>
2   #include <limits>
3   #include <memory>
4   #include <vector>
5   #include <random>
6
7   using namespace std;
8
9   class A{
10  public:
11      A() {cout << "A()" << endl;}
12      ~A() {cout << "~A()" << endl;}
13  };
14  class B{
15  public:
16      B() {cout << "B()" << endl;}
17      ~B() {cout << "~B()" << endl;}
18  };
19
20  class Test {
21      unique_ptr<A> pA{};
22      B b{};      // calls destructor when exception
```

```
23      unique_ptr<int> pInt{};
24      string pStr{};
25      vector<int> pArr{};
26  public:
27      Test() {
28      pA.reset(new A);
29      cout << "Test(): Acquire resources" << endl;
30      throw runtime_error("Failed to initialize");
31      };
32      ~Test() {
33      cout << "~Test(): Release resources" << endl;
34      };
35  };
36
37  int main() {
38      try {
39      Test t;
40      }
41      catch (runtime_error &ex) {
42      cout << ex.what();
43      }
44      return 0;
45  }
```

```
B()
A()
Test(): Acquire resources
~B()
~A()
Failed to initialize
```



Destructor

Destructor should not throw an exception. Exception within destructor may lead to stack unwinding. If a destructor needs to throw an exception, than the exception shall be handled within the destructor.

10.6 Basic Exception Handling - Part VI (noexcept keyword)

noexcept

- Applied to functions (both in declaration & definition)
- Indicates the function does not throw exceptions
- Compiler can optimize the function code
  - no need to generate stack unwinding code
- An exception from such function terminates the program
  - stack may or may not be unwinded
- Not all functions should use noexcept specifier
  - especially functions calling other functions (in library)
  - such functions will be exception-neutral

11 File Input & Output

11.1 Raw String Literals

Listing 187: Raw String Literals

```
#include <iostream>
#include <string>

using namespace std;

int main() {

    string filename{ R"(~/Daten/01-tutorial/Modern-CPP/chapters/11-file-io.org)" };
    cout << filename << endl;
    return 0;
}
```

```
~/Daten/01-tutorial/Modern-CPP/chapters/11-file-io.org
```

Listing 188: Raw String Literals with Delemiter

```
#include <iostream>
#include <string>

using namespace std;

int main() {

    string message { R"MSG(C++ introduced filesystem API in "C++17")MSG" };
    cout << message << endl;
    return 0;
}
```

```
C++ introduced filesystem API in "C++17"
```

allows to include ” and ’ within the string without terminating it.

11.2 Introduction to Filesytem Library

Listing 189: Directory Iterator

```
#include <iostream>
#include <string>
#include <filesystem>

using namespace std;
using namespace filesystem;

int main() {

    path p { R"(/home/christian/Daten/01-tutorial/Modern-CPP/chapters/11-file-io.org)" };
    if(p.has_filename()) {
        cout << p.filename() << endl;
        cout << p.remove_filename() << endl;
    }
    cout << endl;
    for(const auto &x : p) {  //usage of iterators (begin/end)
        cout << x << endl;
```

```
18      }
19      cout << endl;
20
21      p = p.remove_filename();
22      directory_iterator beg {p};
23      directory_iterator end { };
24      if(p.is_absolute()) {
25          while(beg != end) {
26              cout << *beg << endl;
27              ++beg;
28          }
29      }
30      return 0;
31  }
```

```
"11-file-io.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/"

"/"
"home"
"christian"
"Daten"
"01-tutorial"
"Modern-CPP"
"chapters"
""

"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/.#11-file-io.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/c-document-setup-improvements.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/04-class-objects.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/b-org-mode-simulate-keyboard-input.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/08-more-cpp-goodies.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/01-introduction.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/06-operator-overloading.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/09-object-oriented-programming.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/07-memory-management.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/11-file-io.org~"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/11-file-io.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/05-new-move-sementic.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/03-memory-management.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/05-move-semantic.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/a-used-compiler-flags.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/02-basic-language-facilities.org"
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters/10-exception-handling.org"
```

11.3 File I/O - Part I (Basics)

- C++ provides support for file I/O through following classes
  - *ofstream* - wite to an output stream
  - *ifstream* - read from an input stream
  - *fstream* - write/read stream
- Include *fstream* header
- All classes can be used for both text & binary I/O
- Additionally, they support modes that decide how the file is opened and operated

  - *is_open()* to check if a stream is open

Stream Class Typedefs

- **ios:** contains the state that is common fo all the stream classes such as
  - File open modes
  - formatting flags
- **istream & ostream:** Used for console input-output
- **ifstream & ofstream:** Used for File input-output, whereas the **fstream** class is a convenient class for read-write at the same time
- **istringstream & ostringsream:** Used for reading from and writing to a buffer, whereas the **stringtream** class is a convenient class for read-write at the same time.

File Open Modes

- the following modes can be used while opening a file
- These are constants defined in std::ios_base class
- Some modes can be compined e.g. *std::ios::in | std::ios::out* etc, for read write at the same time
  - *app* seek to the end before each write operation
  - *binary* open in binary mode
  - *in* open for reading (default)
  - *out* open for writing
  - *trunc* discard file contents before openeing
  - *ate* seek to end after open

Listing 190: Basic Input Output

```
#include <iostream>
#include <fstream>
#include <string>
void Write() {
    std::ofstream out;     // {"data.txt"};
    out.open("data");       // open instead of using the constructor
    out << "Hello world" << std::endl;
    out << 10 << std::endl;
    out.close();
}
void Read() {
    std::ifstream input;     // { "data.txt" };
    input.open("data.txt");  // open instead of using the constructor
    std::string message;
    std::getline(input, message);
    int value{};
    input >> value;
    input.close();
    std::cout << message << ":" << value << std::endl;
}
int main() {
    Write();
    Read();
    return 0;
}
```

```
Hello world:10
```

11.4 File I/O - Part II (Error Handling)

Listing 191: File open test

```cpp
#include <iostream>
#include <fstream>
#include <string>
void Write() {
    std::ofstream out;     // {"data.txt"};
    out.open("data");       // open instead of using the constructor
    out << "Hello world" << std::endl;
    out << 10 << std::endl;
    out.close();
}
void Read() {
    std::ifstream input;     // { "data.txt" };
    input.open("data.txt");  // open instead of using the constructor
    if (!input.is_open()) {
        std::cout << "Could not open the file" << std::endl;
        return;
    }
    std::string message;
    std::getline(input, message);
    int value{};
    input >> value;
    input.close();
    std::cout << message << ":" << value << std::endl;
}
int main() {
    Write();
    Read();
    return 0;
}
```

Stream State Flags

- The exlamation operator ! is overloaded to check for the *failbit*
  - *goodbit* no error *boo good()*
  - *badbit* irrecoverable stream *boo bad()*
  - *failbit* I/O operation failed *boo fail() [operator !]*
  - *eofbit* end of file reached during input *bool eof()*

Listing 192: Error Handling

```cpp
#include <iostream>
#include <fstream>
#include <string>
void Write() {
    std::ofstream out;     // {"data.txt"};
    out.open("data");       // open instead of using the constructor
    out << "Hello world" << std::endl;
    out << 10 << std::endl;
    out.close();
}
void Read() {
    std::ifstream input;     // { "data.txt" };
    input.open("data.txt");  // open instead of using the constructor
    if (input.fail()) {
        std::cout << "Could not open the file" << std::endl;
        return;
```

```
    }
    std::string message;
    std::getline(input, message);
    int value{};
    input >> value;
    input >> value;
    if (input.eof()) {
        std::cout << "End of file encountered" << std::endl;
    }
    if (input.good()) { // test with a none existing file
        std::cout << "I/O operations are successful" << std::endl;
    }
    else {
        std::cout << "Some I/O operations failed" << std::endl;
    }
    input.close();
    input.clear();                      // sets the good bit and resets all other states
    input.setstate(std::ios::failbit);  // sets failbit
    std::cout << message << ":" << value << std::endl;
}
int main() {
    Write();
    Read();
    return 0;
}
```

```
End of file encountered
Some I/O operations failed
Hello world:10
```

11.5 File I/O - Part III (Copy Utility)

Listing 193: Copy Text File

```
#include <iostream>
#include <fstream>
#include <filesystem>
#include <string>

using namespace std;
using namespace filesystem;

int main() {
    // set curren_path to project path
    path project(current_path());
    project /= "../projects/section-11/cputil";
    current_path(project);

    path source(current_path());
    source /= "Source.cpp";

    path dest(current_path());
    dest /= "Copy.cpp";

    ifstream input{ source };
    if(!input) {
        cout << "Source file not found" << endl;
    }

    ofstream output { dest };
```

```
    string line;
    while(!getline(input, line).eof()) {
        output << line << endl;
    }
    input.close();
    output.close();

    cout << "all file operations done..." << endl;
    return 0;
}
```

```
all file operations done...
```

11.6 File I/O - Part IV (Character IO & Seeking)

Listing 194: Reading and Writing Characters

```
#include <iostream>
#include <fstream>
#include <string>
#include <filesystem>

using namespace std;
using namespace filesystem;

void Write(const filesystem::path& project) {
    current_path(project);

    std::ofstream out {"test.txt"};
    if(!out) {
        out << "Could not open file for writing" << endl;
        return;
    }
    string message{"C++ was invented by Bjarne"};

    // write character per character.
    for(char ch : message) {
        out.put(ch);
    }

}
void Read(const filesystem::path& project) {
    current_path(project);

    std::ifstream input {"test.txt"};
    if (!input.is_open()) {
        cout << "Could not open the file reading" << endl;
        return;
    }

    char ch{};
    while (input.get(ch)) {
        cout << ch;
    }

}
int main() {
    // set current path
    path project(current_path());
    project /= "../projects/section-11/cputil";
    current_path(project);
```

```
45
46      Write(project);
47      Read(project);
48
49      return 0;
50  }
```

```
C++ was invented by Bjarne
```

The input and output streams have a pointer internally, the pointer represents the position at which the operation will be performed in the file.

- **Output streams:** this pointer is known as the **put pointer**.
- **Input streams:** this pointer is known as the **get pointer**.

Listing 195: Placing put- and get pointer

```
1   #include <iostream>
2   #include <fstream>
3   #include <string>
4   #include <filesystem>
5
6   using namespace std;
7   using namespace filesystem;
8
9   void Write(const filesystem::path& project) {
10      current_path(project);
11
12      std::ofstream out {"test.txt"};
13      if(!out) {
14          out << "Could not open file for writing" << endl;
15          return;
16      }
17      string message{"C++ was invented by Bjarne"};
18
19      // write character per character.
20      for(char ch : message) {
21          out.put(ch);
22      }
23      out.seekp(20, ios::beg);
24      out.put('#');
25  }
26  void Read(const filesystem::path& project) {
27      current_path(project);
28
29      std::ifstream input {"test.txt"};
30      if (!input.is_open()) {
31          cout << "Could not open the file reading" << endl;
32          return;
33      }
34      input.seekg(10, ios::beg);
35      cout << "Current position is: " << input.tellg() << endl;
36      char ch{};
37      while (input.get(ch)) {
38          cout << ch;
39      }
40  }
41  int main() {
42      path project(current_path());
43      project /= "../projects/section-11/cputil";
```

```
    current_path(project);

    Write(project);
    Read(project);
    return 0;
}
```

```
Current position is: 10
vented by #jarne
```

fstream does not create a file if it does not exist.

Listing 196: Using fstream classx

```
#include <iostream>
#include <fstream>
#include <string>
#include <filesystem>

using namespace std;
using namespace filesystem;

void UsingFstream(const filesystem::path& project) {
    current_path(project);
    fstream stream("file.txt");
    if(!stream) {
        cout << "FILE DOES NOT EXIST, CREATING ONE..." << endl;
        ofstream out("file.txt");
        out.close();
        stream.open("file.txt");
    }
    stream << "Hello World" << endl;;
    stream.seekg(0);   // set get pointer to 0
    string line;
    getline(stream, line);
    cout << line << endl;
}
int main() {
    path project(current_path());
    project /= "../projects/section-11/cputil";
    current_path(project);

    UsingFstream(project);
    project /= "file.txt";
    // remove(project);

    return 0;
}
```

```
Hello World
```

11.7 File I/O - Part V (Binary IO)

A binary file stores data in it's equivalent binary form. If you open a binary file in a text editor, it will contain non-human readable characters.

Listing 197: Binary Files

```
#include <iostream>
#include <fstream>
```

```
#include <string>
#include <filesystem>

using namespace std;
using namespace filesystem;

int main() {
    // set current path
    path project(current_path());
    project /= "../projects/section-11/cputil";
    current_path(project);

    ofstream textstream {"data"};
    textstream << 12345678;

    ofstream binstream("binary", ios::binary | ios::out);
    int num{12345678};
    binstream.write((const char*)&num, sizeof(num));

    cout << "data and binary filed to ../projects/section-11/cputil" << endl;
    cout << "binary is stored as little endian 0xBC614E becomes 0x4E61BC" << endl;

    num = 0;
    binstream.close();
    ifstream input{ "binary", ios::binary | ios::in};
    input.read((char*)&num, sizeof(num));
    cout << num << endl;

    return 0;
}
```

```
data and binary filed to ../projects/section-11/cputil
binary is stored as little endian 0xBC614E becomes 0x4E61BC
12345678
```

Listing 198: Record

```
#include <iostream>
#include <fstream>
#include <string>
#include <filesystem>

using namespace std;
using namespace filesystem;

struct Record {
    int id;
    char name[10];
};

void WriteRecord(const filesystem::path &project, Record *p) {
    current_path(project);

    ofstream binstream{"records", ios::binary | ios::out };
    binstream.write((const char*)p, sizeof(Record));
}

Record ReadRecord(const filesystem::path &project) {
    current_path(project);

```

```
    ifstream input {"records", ios::binary | ios::in };
    Record r;
    input.read((char*)&r, sizeof(Record));
    return r;
}

int main() {
    // set current path
    path project(current_path());
    project /= "../projects/section-11/cputil";
    current_path(project);

    Record r1 {1963, "Christian"};
    WriteRecord(project, &r1);
    Record r2 = ReadRecord(project);

    cout << "id: " << r2.id << ", name: " << r2.name << endl;

    return 0;
}
```

```
id: 1963, name: Christian
```

Store Multiple Records

- Save to File: Store the total number or records stored in the file
- Read back: Firs read how many records are stored and then read the records accordingly.



Binary Files

Do not have enf of file character in them. Therfore the sturcture of the file is important to know.

12 Templates

12.1 Introduction to Templates

- Generalizes software components
- Such components can be reused in different situations
- Operate of any kind of data
- High performance algorithm & classes
- Compile time; no run time costs are involved
- Libraries such as ATL, WTL, Boost, POCO, ACE, etc. use templates for implementation

Listing 199: Find Maximum of two numbers

```
#include <iostream>
#include <string>

using namespace std;

int Max(int x, int y) {
    return x > y ? x : y;
}
float Max(float x, float y) {
    return x > y ? x : y;
}

int main() {

    auto num1 = Max(3,8);
    auto num2 = Max(3.1f,8.1f);
    cout << "Integer:\t" << num1 << endl;
    cout << "Float:\t"   << num2 << endl;
    return 0;
}
```

```
Integer: 8
Float: 8.1
```

Listing 200: Template Find Maximum of two numbers

```
#include <iostream>
#include <string>

using namespace std;

template<typename T>
T Max(T x, T y) {
    return x > y ? x : y;
}

int main() {

    auto num1 = Max(3,8);
    auto num2 = Max(3.1f,8.1f);
    cout << "Integer:\t" << num1 << endl;
    cout << "Float:\t"   << num2 << endl;
    return 0;
}
```

```
Integer: 8
Float: 8.1
```

typename vs. class

The differnces between the keyword *class* and *typename* are apparent only when doing advanced template programming.

Template Instantiation

The process by which the function is generated is called **instantiation**:
The compiler will examine the arguments of the function template and it'll accordingly, deduce the type of T. Once the type of T is deduced, the function is **instantiated** for this particular type.

12.2 Assignment I

12.2.1 ADD Template: Return the sum of two elements

int Add(int x,int y)

Listing 201: Return the sum of two elements

```
#include <iostream>
#include <string>

using namespace std;

template<typename T>
T ADD(T x, T y) {
    return x + y;
}

int main() {

    auto num1 = ADD(3,8);
    auto num2 = ADD(3.1f,8.1f);
    cout << "Integer:\t" << num1 << endl;
    cout << "Float:\t\t"   << num2 << endl;
    return 0;
}
```

```
Integer: 11
Float:  11.2
```

12.2.2 Array Sum Template : Returns the sum of array elements

*int ArraySum(const int *pArr, size_t arrSize);*

Listing 202: Returns the sum of array elements

```
#include <iostream>
#include <string>

using namespace std;

template<typename T>
T ArraySum(T *x, size_t arrSize) {
```

```
    T sum {};
    for(size_t sz = 0; sz < arrSize; ++sz) {
^^Isum += x[sz];
    }
    return sum;
}

int main() {

    size_t sz = 5;
    float myArray[5] {3.8f,3.8f,3.8f,3.8f,3.8f};

    auto num = ArraySum(myArray, sz);
        cout << "ArraySum:\t" << num << endl;

    return 0;
}
```

```
ArraySum: 19
```

12.2.3 Max Template : Return the largest element in the array

*int Max(const int *pArr, size_t arrSize)*

Listing 203: Returns the largest element in the array

```
#include <iostream>
#include <string>

using namespace std;

template<typename T>
T Max(T *x, size_t arrSize) {
    T max = x[0];
    for(size_t sz = 1; sz < arrSize; ++sz) {
^^Imax = max < x[sz] ? x[sz] : max;
    }
    return max;
}

int main() {

    size_t sz = 5;
    float myArray[5] {1.8f,5.8f,7.8f,4.8f,3.8f};

    auto num = Max(myArray, sz);
        cout << "Max:\t" << num << endl;

    return 0;
}
```

```
Max: 7.8
```

12.2.4 MinMax Template: Return the smallest and largest element in a pair

*std::pair<int,int> MinMax(const int *pArr, size_t arrSize)*

Listing 204: Return the smallest and largest element in a pair

```cpp
#include <iostream>
#include <string>

using namespace std;

template<typename T>
pair<T,T> MinMax(T *x, size_t arrSize) {
    T max = x[0];
    for(size_t sz = 1; sz < arrSize; ++sz) {
^^Imax = max < x[sz] ? x[sz] : max;
    }
    T min = x[0];
    for(size_t sz = 1; sz < arrSize; ++sz) {
^^Imin = min > x[sz] ? x[sz] : min;
    }
    return pair<T,T>(min,max);
}

int main() {

    size_t sz = 5;
    float myArray[5] {1.8f,5.8f,7.8f,4.8f,3.8f};

    auto num = MinMax(myArray, sz);
        cout << "Min:" << num.first << ", Max:" << num.second << endl;

    return 0;
}
```

```
Min:1.8, Max:7.8
```

12.3 Template Argument Deduction & Instantiation

Template Argument Deduction:

- Process of decucing the types
- Each function argument is examined
- The corresponding type argument is deduced from the argument
- The type argument deduction should lead to same type
- Type conversion are not performed
- After deduction, the templae is instantiated
- Override deduction by specifying types in template argument list *Max<int>(3,5);*

Template Instantiation:

- A template function of class only acts as a **blueprint**
- The compiler generates code from the blueprint at **compile time**
- Know as template instantiation
- Occurs implicitly when
  - a function template is invoked
  - taking address of a function template
  - using explicit instantiation

  - creating explicit specialization

- Full definition of template shoulb be available
- Define in **header-file**

Listing 205: Instantiation: Explicite & by Address

```
#include <iostream>
#include <string>

using namespace std;

template<typename T>
T Max(T x, T y) {
    std::cout << typeid(T).name() << std::endl;
    return x > y ? x : y;
}

template char Max(char x, char y);     // Option d) explicite Instantiation for char types

int main() {

    Max(3, 8);
    // Max(3.1,8.1f); Error can not deduce Type T one time double and one time float

    Max(static_cast<float>(3.1), 8.1f);   // Option a) type cast

    Max<float>(3.1, 8.1f);                // Option b) override deduction process
                                          //           by explicit specification

    int (*pfn)(int, int) = Max;           // instantiate by taking address
    pfn(3,2);

    return 0;
}
```

```
i
f
f
i
```

12.4 Explicite Specialization

- Template specialiced for a particular type
- Provides correct semantic for some datatypes
- Or implement an algorithm more optimally for a specific type
- Explicte specialication must be defined in a **cpp file**
- Primary template definition should occur before specialization

Listing 206: Explicite Specialization

```
#include <iostream>
#include <string>
#include <cstring>

using namespace std;
```

```
6
7   template<typename T>
8   T Max(T x, T y) {
9       //std::cout << typeid(T).name() << std::endl;
10      return x > y ? x : y;
11  }
12
13  // Explicite Instantiation
14  template char Max(char x, char y);      // Option d) explicite Instantiation for char types
15
16  // Explicite Spezialisation
17  // Optional: Specifie the type for specialisation Max<const char *x>()
18  template<> const char * Max(const char *x, const char *y) {
19  cout << "Max<const *char>()" << endl;
20      return strcmp(x,y) > 0 ? x : y;
21  }
22
23  int main() {
24
25      const char *b{"B"};
26      const char *a{"Hallo World"};
27
28      auto m = Max(a,b);
29      cout << m << endl;
30      return 0;
31  }
```

```
Max<const *char>()
Hallo World
```

12.5 Non-type Template Arguments

- Expression that is computed at compile time within a template argument list
- Must be a constant expression (addresses,references,integrals,nullptr,enums)
- Part of the template library
- Used to implement std::begin & std::end functions

Listing 207: Non-type Template Arguments

```
1   #include <iostream>
2   #include <string>
3   #include <cstring>
4
5   using namespace std;
6
7   template<typename T>
8   T Max(T x, T y) {
9       //std::cout << typeid(T).name() << std::endl;
10      return x > y ? x : y;
11  }
12
13  // Explicite Instantiation
14  template char Max(char x, char y);      // Option d) explicite Instantiation for char types
15
16  // Explicite Spezialisation
17  // Optional: Specifie the type for specialisation Max<const char *x>()
18  template<> const char * Max(const char *x, const char *y) {
```

```
19      cout << "Max<const *char>()" << endl;
20      return strcmp(x,y) > 0 ? x : y;
21  }
22
23  template<int size>
24  void Print() {
25      // ++size; // not allowdes size needs to be a constant expression
26      cout << size << endl;
27  }
28
29  template<typename T>
30  T Sum1(T *pArr, int size) {
31      T sum {};
32      for(int i=0; i<size; ++i) {
33          sum += pArr[i];
34      }
35      return sum;
36  }
37
38  template<typename T, int size>
39  T Sum2(T (&pArr)[size]) {  // pass a reference to array
40      T sum {};
41      for(int i=0; i<size; ++i) {
42          sum += pArr[i];
43      }
44      return sum;
45  }
46
47
48  int main() {
49
50      Print<3>(); // argument needs to be constant
51      int arr[] {3,4,5,6,7};
52      auto sum1 = Sum1(arr, 5);
53      //int (&ref)[5] = arr; // reference to array - size specification needed!
54      auto sum2 = Sum2(arr);
55      cout << sum1 << ", " << sum2 << endl;
56
57      return 0;
58  }
```

```
3
25, 25
```



Reference to Array and None-type Arguments

We can pass an array into a function without explicitely specifying the size.
This is used by the standard template library to implement the global functions std::begin and std::end for arrays.

12.6 Assignment III

For the functions in questions 2,3 & 4 in Assignment I:

- Modify to accept array by reference.
- Specialize for an array of strings (const char *). Observe, the limitations.
- Specialize for an array of std::strings. Compare with the previous implementation.

- Define the specialized functions in a .h file and include it in multiple source (.cpp) files. Observe what happens.

12.6.1 Array Sum Template : Returns the sum of array elements

*int ArraySum(const int *pArr, size_t arrSize);*

Listing 208: Returns the sum of array elements

```
#include <iostream>
#include <string>

using namespace std;

template<typename T, int arrSize>
T ArraySum(T (&x)[arrSize]) {
    T sum {};
    for(size_t sz = 0; sz < arrSize; ++sz) {
        sum += x[sz];
    }
    return sum;
}

// Explicite Spezialisation
// Optional: Specifie the type for specialisation ArraySum<const char *x>()
template<> const char * ArraySum(char (&x)[arrSize]) {
    cout << "ArraySum<char (&x)[arrSize]>()" << endl;

    char sum {};
    for(size_t sz = 0; sz < arrSize; ++sz) {
        sum += x[sz];
    }
    return sum;
}


int main() {

    float myArray[5] {3.8f,3.8f,3.8f,3.8f,3.8f};
    auto num = ArraySum(myArray);
    cout << "ArraySum:\t" << num << endl;

    char myCharArray[] {"AB"};
    auto charNum = ArraySum(myCharArray);
    cout << "ArraySum:\t" << charNum << endl;

    return 0;
}
```

12.6.2 Max Template : Return the largest element in the array

*int Max(const int *pArr, size_t arrSize)*

Listing 209: Returns the largest element in the array

```
#include <iostream>
#include <string>

using namespace std;

template<typename T>
T Max(T *x, size_t arrSize) {
```

```
    T max = x[0];
    for(size_t sz = 1; sz < arrSize; ++sz) {
        max = max < x[sz] ? x[sz] : max;
    }
    return max;
}

int main() {

    size_t sz = 5;
    float myArray[5] {1.8f,5.8f,7.8f,4.8f,3.8f};

    auto num = Max(myArray, sz);
    cout << "Max:\t" << num << endl;

    return 0;
}
```

12.6.3 MinMax Template: Return the smallest and largest element in a pair

*std::pair<int,int> MinMax(const int *pArr, size_t arrSize)*

Listing 210: Return the smallest and largest element in a pair

```
#include <iostream>
#include <string>

using namespace std;

template<typename T>
pair<T,T> MinMax(T *x, size_t arrSize) {
    T max = x[0];
    for(size_t sz = 1; sz < arrSize; ++sz) {
        max = max < x[sz] ? x[sz] : max;
    }
    T min = x[0];
    for(size_t sz = 1; sz < arrSize; ++sz) {
        min = min > x[sz] ? x[sz] : min;
    }
    return pair<T,T>(min,max);
}

int main() {

    size_t sz = 5;
    float myArray[5] {1.8f,5.8f,7.8f,4.8f,3.8f};

    auto num = MinMax(myArray, sz);
    cout << "Min:" << num.first << ", Max:" << num.second << endl;

    return 0;
}
```

12.7 Integer Class

Listing 211: Integer Class Declaration

```
#include <iostream>
class Integer {
    int *m_pInt;
public:
```

```
    Integer();                                  // Default constructor
    Integer(int value);                         // Parameterized constructor
    Integer(const Integer &obj);                // Copy constructor
    Integer(Integer &&obj);                     // Move constructor
    int GetValue()const;
    void SetValue(int value);
    ~Integer();
    Integer & operator ++();                    // pre-increment operator
    Integer operator ++(int);                   // post-increment operator
    bool operator ==(const Integer &a)const;    // comparison
    Integer & operator =(const Integer &a);     // Copy assignment
    Integer & operator =(Integer &&a);          // Move assignment
    Integer operator +(const Integer & a)const; // + operator

    void operator ()();                         // Function Call Operator
    explicit operator int();                    // Type Conversion Operator

    // operator >> is decalred as friend of Integer Class
    friend std::istream & operator >> (std::istream &input, Integer &a);
};
Integer operator +(int x, const Integer &y);
std::ostream & operator <<(std::ostream & out, const Integer &a);
std::istream & operator >> (std::istream &input, Integer &a);
```

Listing 212: Integer Class Definition: Constructors

```
#include "integer.h"
#include <iostream>

using namespace std;

Integer::Integer() {                            // standard constructor
    cout << "Integer()" << endl;
    m_pInt = new int(0);
}

Integer::Integer(int value) {                   // paramterizied constructor
    cout << "Integer(int)" << endl;
    m_pInt = new int(value);
}

Integer::Integer(const Integer & obj) {         // copy constructor
    cout << "Integer(const Integer&)" << endl;
    m_pInt = new int(*obj.m_pInt);
}

Integer::Integer(Integer && obj) {              // move constructor
    cout << "Integer(int&&)" << endl;
    m_pInt = obj.m_pInt;
    obj.m_pInt = nullptr;
}

Integer::~Integer() {                          // destructor
    cout << "~Integer()" << endl;
    delete m_pInt;
}
```

Listing 213: Integer Class Definition: Setter and Getter

```

  int Integer::GetValue() const {
```

```
    return *m_pInt;
  }

  void Integer::SetValue(int value) {
    *m_pInt = value;
  }
```

Listing 214: Integer Class Definition: Operator Overlaoding member functiosn

```
Integer & Integer::operator++() { // ++a means a.operator++()
    cout << "operator++()" << endl;
    ++(*m_pInt);
    return *this;  // the value a the address of this
}

Integer Integer::operator++(int) { // a++ means a.operator++(int)
    cout << "operator++(int)" << endl;
    Integer temp(*this);
    ++(*m_pInt);
    return temp;
}

bool Integer::operator==(const Integer & a) const { // c==a means c.operator==(a)
    cout << "operator==(const Integer &) const" << endl;
    return *m_pInt == *a.m_pInt;
}

Integer & Integer::operator=(const Integer & a) { // c=a means c.operator=(a)
    cout << "operator=(const Integer &) const" << endl;
    if (this != &a) {            // check for self assignemnt
    delete m_pInt;
    m_pInt = new int(*a.m_pInt);
    }
    return *this;
}

Integer & Integer::operator=(Integer && a) { // c=&&a means c.operator=(&&a)
    cout << "operator=(Integer &&) const" << endl;
    if (this != &a) {
    delete m_pInt;
    m_pInt = a.m_pInt;
    a.m_pInt = nullptr;
    }
    return *this;
}

Integer Integer::operator+(const Integer & a) const { // c+a means c.operator+(a)
    cout << "operator+(const Integer &) const" << endl;
    Integer temp;
    *temp.m_pInt = *m_pInt + *a.m_pInt; // *this->m_pInt
    return temp;
}

void Integer::operator()() {
    // cout << "operator()()" << endl;
    cout << *m_pInt << endl;
}
```

Listing 215: Integer Class Definition: Global Operator overloading

```
Integer operator +(int x, const Integer &y) {
```

```
    // cout << "operator+(int, const Integer)" << endl;
    Integer temp;
    temp.SetValue(x + y.GetValue());
    return temp;
}
ostream & operator <<(ostream & out, const Integer &a) {
    // cout << "operator <<(ostream &, const Integer &)" << endl;
    out << a.GetValue();
    return out;
}
istream & operator >> (istream &input, Integer &a) {
    // cout << "operator >>(istream &, Integer &)" << endl;
    int x;
    input >> x;
    // a.SetValue(x);  // SetValue() allows copy by value, no friend required
    *a.m_pInt = x;     // access to private member of Integer Class.
    return input;
}
```

12.8 Perfect Forwarding - Part I (C++11)

Listing 216: Perfect Forwarding

```
#include <integer.h>
#include <iostream>
#include <string>

using namespace std;

class Employee {
    string m_Name;
    Integer m_Id;

public:
    Employee(const string &name, const Integer &id) :
        m_Name { name },
        m_Id { id } {
        cout << "Employee(const string &, const Integer &)" << endl;
    }
    Employee(string &&name, Integer &&id) :
        m_Name { name },
        m_Id { move(id) } {
        cout << "Employee(string &&, Integer &&)" << endl;
    }
};

int main() {

    Employee emp1 { "christian", 63 };

    int val {100};
    Employee emp2 { "christian", val };

    string name {"christian"};
    Employee emp3 { name, 63 };

    Employee emp4 { name, val };


    // we needed to implement 4 constructors !!!

```

```
39      return 0;
40  }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Integer(int)
Integer(const Integer&)
Employee(const string &, const Integer &)
~Integer()
~Integer()
```

Listing 217: Perfect Forwarding - Member Template

```
1   #include <integer.h>
2   #include <iostream>
3   #include <string>
4
5   using namespace std;
6
7   class Employee {
8       string m_Name;
9       Integer m_Id;
10
11  public:
12      template<typename T1, typename T2>
13      Employee(T1 &&name, T2 &&id) :
14          m_Name { name },
15          m_Id { id } {
16          cout << "Employee(T1 &&, T2 &&)" << endl;
17      }
18  };
19
20  int main() {
21
22      cout << "Constructor Calls:" << endl;
23      cout << "---------- emp 1 -------------" << endl;
24      Employee emp1 { "christian", 63 };
25      cout << "---------- emp 3 -------------" << endl;
26      int val {100};
27      Employee emp2 { "christian", val };
28      cout << "---------- emp 3 -------------" << endl;
29      string name {"christian"};
30      Employee emp3 { name, 63 };
31      cout << "---------- emp 4 -------------" << endl;
32      Employee emp4 { name, val };
33
34      cout << endl;
35      cout << "Value Type Presevation Required:" << endl;
36      cout << "---------- emp 5 -------------" << endl;
37
38      cout << endl;
39      cout << "-------- Destructor ----------" << endl;
40
41      return 0;
42  }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
---------- emp 1 -------------
```

```
Integer(int)
Employee(T1 &&, T2 &&)
---------- emp 3 -------------
Integer(int)
Employee(T1 &&, T2 &&)
---------- emp 3 -------------
Integer(int)
Employee(T1 &&, T2 &&)
---------- emp 4 -------------
Integer(int)
Employee(T1 &&, T2 &&)
---------- emp 5 -------------
Integer(int)
Integer(const Integer&)
Employee(T1 &&, T2 &&)
~Integer()
-------- Destructor ----------
~Integer()
~Integer()
~Integer()
~Integer()
~Integer()
```

12.9 Perfect Forwarding - Part II (C++11)

- *foreward<T>() Function>* is defined in the *<utility>* header

Listing 218: Perfect Forwarding - Member Template

```
#include <integer.h>
#include <iostream>
#include <string>

using namespace std;

class Employee {
    string m_Name;
    Integer m_Id;

public:
    template<typename T1, typename T2>
    Employee(T1 &&name, T2 &&id) :
        m_Name { forward<T1>(name) },     // preserve value type T1
        m_Id   { forward<T2>(id)   } {    // prseerve value type T2
        cout << "Employee(T1 &&, T2 &&)" << endl;
    }
};

template<typename T1, typename T2>
Employee *Create(T1 &&a, T2 &&b) {
    return new Employee(forward<T1>(a), forward<T2>(b));
}

int main() {

    int val {100};
    string name {"christian"};

    cout << "Value Type Presevation Required:" << endl;
```

```
31      cout << "---------- emp 5 -------------" << endl;
32      Employee emp5 { "christian", Integer {63} };
33      cout << "---------- emp 6 -------------" << endl;
34      Employee emp6 { string {"christian"}, val };
35
36      cout << endl;
37      cout << "Factroy Class:" << endl;
38      cout << "---------- emp 7 -------------" << endl;
39      auto emp7 = Create( "christian", Integer {63} );
40
41      cout << endl;
42      cout << "-------- Destructor ----------" << endl;
43
44      (void)emp7;
45      return 0;
46  }
```

```
g++ -std=c++17 -Wall -Wextra -fno-elide-constructors -I../inc src/main.cpp ../src/integer.cpp -o app
./app
Value Type Presevation Required:
---------- emp 5 -------------
Integer(int)
Integer(int&&)
Employee(T1 &&, T2 &&)
~Integer()
---------- emp 6 -------------
Integer(int)
Employee(T1 &&, T2 &&)

Factroy Class:
---------- emp 7 -------------
Integer(int)
Integer(int&&)
Employee(T1 &&, T2 &&)
~Integer()

-------- Destructor ----------
~Integer()
~Integer()
```

12.10 Variadic Templates - Part I (C++11)

Listing 219: Arguments all of same type

```
1   #include <iostream>
2
3   using namespace std;
4
5   template<typename T>
6   void Print(initializer_list<T> args) {
7       for (const auto &x : args) {
8   ^^Icout << x << " " ;
9       }
10  }
11
12  int main() {
13
14      Print({ 1,2,3,4,5,6 });
15
```

```
16      return 0;
17  }
```

```
1 2 3 4 5 6
```

Listing 220: Variadic Template: Various Argument Types

```
1   #include <iostream>
2
3   using namespace std;
4
5   void Print() {                          // Base Case Function
6   }
7
8   template<typename T, typename...Params>    // Template parameter pack
9   void Print(T a, Params... args) {          // Function parameter pack
10      Print(args...);                        // recursive call
11
12  }
13
14  int main() {
15
16      Print( 1, 2.5, 3, "4" );
17
18      return 0;
19  }
20
21  /* Recursive Expansion
22     1. Print(1, 2.5, 3, "4") a=1,    args= 2.5, 3, "4"
23     2. Print(2.5, 3, "4")    a=2.5, args=3, "4"
24     3. Print(3, "4")         a=3,    args="4"
25     4. Print("4")            a="4", args=""
26     5. Print()               a="",  args=""
27  */
```

12.11 Variadic Templates - Part II (C++11)

Listing 221: Variadic Template: Various Argument Types

```
1   #include <iostream>
2
3   using namespace std;
4
5   void Print() {                          // Base Case Function
6       cout << endl;
7   }
8
9   template<typename T, typename...Params>    // Template parameter pack
10  void Print(T a, Params... args) {          // Function parameter pack
11      // cout << sizeof...(args) << ", " << sizeof...(Params) << endl;
12      cout << a;
13      if(sizeof...(args) != 0) {
14  ^^Icout << ", ";
15      }
16      Print(args...);                        // recursive call
17
18  }
19
20  int main() {
21
```

```
    Print( 1, 2.5, 3, "4" );

    return 0;
}
```

```
1, 2.5, 3, 4
```

Listing 222: Variadic Template: Pass by reference Type

```
#include <iostream>

using namespace std;

void Print() {                                      // Base Case Function
    cout << endl;
}

template<typename T, typename...Params>             // Template parameter pack
void Print(const T &a, const Params&... args) {     // Function parameter pack
    // cout << sizeof...(args) << ", " << sizeof...(Params) << endl;
    cout << a;
    if(sizeof...(args) != 0) {
        cout << ", ";
    }
    Print(args...);                         // still pass by value

}

int main() {

    Print( 1, 2.5, 3, "4" );

    return 0;
}
```

Listing 223: Variadic Template: Perfect Forewarding

```
#include <integer.h>
#include <iostream>

using namespace std;

void Print() {                              // Base Case Function
    cout << endl;                           // recursion terminates here
}

template<typename T, typename...Params>     // Template parameter pack
void Print(T &&a, Params&&... args) {       // Function parameter pack
    // cout << sizeof...(args) << ", " << sizeof...(Params) << endl;
    cout << a;
    if(sizeof...(args) != 0) {
        cout << ", ";
    }
    Print(forward<Params>(args)...);        // perfect forewarding

}

int main() {

    Integer val {1};
```

```
24      Print( 0, val, Integer {2} );
25
26      return 0;
27  }
```

```
./app
Integer(int)
Integer(int)
0, 1, 2
~Integer()
~Integer()
```

12.12 Assignment IV

12.13 Class Templates

Listing 224: Class Template: Stack for Int's

```
1   #include <iostream>
2
3   using namespace std;
4
5   class Stack {
6
7       int m_Buffer[512];
8       int m_Top {-1};
9   public:
10      void Push(int elem) {
11          m_Buffer[++m_Top] = elem;
12      }
13      void Pop() {
14          --m_Top;
15      }
16      int Top() const {
17          return m_Buffer[m_Top];
18      }
19      bool IsEmpty() {
20          return m_Top == -1;
21      }
22  };
23
24  int main() {
25
26      Stack s;
27      s.Push(3);
28      s.Push(1);
29      s.Push(6);
30      s.Push(9);
31
32      while (!s.IsEmpty()) {
33          cout << s.Top() << " ";
34          s.Pop();
35      }
36      return 0;
37  }
```

```
9 6 1 3
```

Listing 225: Class Template: Stack Template

```cpp
#include <iostream>

using namespace std;

template<typename T, int size> // not template type argumetn
class Stack {

    T m_Buffer[size];
    int m_Top {-1};
public:
    Stack() = default;
    Stack(const Stack &obj) {
        m_Top = obj.m_Top;
        for(int i=0; i <= m_Top; ++i) {
            m_Buffer[i] = obj.m_Buffer[i];
        }
    }
    void Push(const T &elem) {
        m_Buffer[++m_Top] = elem;
    }
    void Pop();

    const T& Top() const {
        return m_Buffer[m_Top];
    }
    bool IsEmpty() {
        return m_Top == -1;
    }
    static Stack Create();
};

// Template parameters are part of the type of the function
template<typename T, int size>
void Stack<T, size>::Pop() {
    --m_Top;
}

template<typename T, int size>
Stack<T, size> Stack<T, size>::Create() {
    return Stack<T, size>();
}




int main() {
    // Template parameters are part of the type of the class
    Stack<float, 10> s = Stack<float, 10>::Create();
    // Class instantion only for type which are invoked
    s.Push(3);
    s.Push(1);
    s.Push(6);
    s.Push(9);

    auto s2(s);
    while (!s2.IsEmpty()) {
        cout << s2.Top() << " ";
        s2.Pop();
    }
    return 0;
}
```

```
9 6 1 3
```

12.14 Class Template Explicit Specialization - Part I

- all of the template are specialised

Listing 226: Class Template: Pretty Printer

```
#include <iostream>

using namespace std;

template<typename T>
class PrettyPrinter {
    T * m_pData;
public:
    PrettyPrinter(T *data) : m_pData(data) {
    }
    void Print() {
        cout << "{" << *m_pData << "}" << endl;
    }

    T * GetData() {
        return m_pData;
    }
};

int main() {

    int data = 5;
    float f = 8.2f;

    PrettyPrinter<int> p1(&data);
    p1.Print();
    PrettyPrinter<float> p2(&f);
    p2.Print();

    // Prints the first char only
    const char *p{"Hello World"};
    PrettyPrinter<const char> p3(p);
    p3.Print();
    const char *pData = p3.GetData();
    return 0;
}
```

```
{5}
{8.2}
{H}
```

Listing 227: Class Template: Pretty Printer Explicite Spezialisation for char*

```
#include <iostream>

using namespace std;

template<typename T>
class PrettyPrinter {
    T * m_pData;
public:
```

```
    PrettyPrinter(T *data) : m_pData(data) {
    }
    void Print() {
        cout << "{" << *m_pData << "}" << endl;
    }

    T * GetData() {
        return m_pData;
    }
};

// Explicite Specialisation for const char
template<>
class PrettyPrinter<const char*> {
    const char * m_pData;
public:
    PrettyPrinter(const char * data) : m_pData(data) {
    }
    void Print() {
        cout << "{" << m_pData << "}" << endl;
    }

    const char* GetData() {
        return m_pData;
    }
};

int main() {

    const char *p{"Hello World"};
    PrettyPrinter<const char*> p3(p);
    p3.Print();
    const char *pData = p3.GetData();
    return 0;
}
```

```
{Hello World}
```

12.15 Class Template Explicit Specialization - Part II

Listing 228: Class Template: Pretty Printer with vector

```
#include <iostream>
#include <vector>

using namespace std;

template<typename T>
class PrettyPrinter {
    T * m_pData;
public:
    PrettyPrinter(T *data) : m_pData(data) {
    }
    void Print() {
        cout << "{" << *m_pData << "}" << endl;
    }

    T * GetData() {
        return m_pData;
    }
};
```

```
20
21  // Explicite Specialisation for const char
22  template<>
23  class PrettyPrinter<const char*> {
24      const char * m_pData;
25  public:
26      PrettyPrinter(const char * data) : m_pData(data) {
27      }
28      void Print() {
29          cout << "{" << m_pData << "}" << endl;
30      }
31
32      const char* GetData() {
33          return m_pData;
34      }
35  };
36
37  // Explicite Specialisation for vector
38  template<>
39  class PrettyPrinter<vector<int>> {
40      vector<int> *m_pData;
41  public:
42      PrettyPrinter(vector<int> * data) : m_pData(data) {
43      }
44      void Print() {
45          cout << "{";
46          for(const auto &x : * m_pData) {
47              cout << x << ", ";
48          }
49          cout << "}" << endl;
50      }
51
52      vector<int>* GetData() {
53          return m_pData;
54      }
55  };
56
57
58  int main() {
59
60      int data = 5;
61      float f = 8.2f;
62
63      PrettyPrinter<int> p1(&data);
64      p1.Print();
65      PrettyPrinter<float> p2(&f);
66      p2.Print();
67
68      const char *p{"Hello World"};
69      PrettyPrinter<const char*> p3(p);
70      p3.Print();
71      const char *pData = p3.GetData();
72
73      vector<int> v{1,2,3};
74      PrettyPrinter<vector<int> > p4(&v);
75      p4.Print();
76      return 0;
77  }
```

```
{5}
{8.2}
```

```
{Hello World}
{1, 2, 3, }
```

Listing 229: Class Template: Pretty Printer with Specialisation of Print Function

```
#include <iostream>
#include <vector>

using namespace std;

template<typename T>
class PrettyPrinter {
    T * m_pData;
public:
    PrettyPrinter(T *data) : m_pData(data) {
    }
    void Print() {
        cout << "{" << *m_pData << "}" << endl;
    }

    T * GetData() {
        return m_pData;
    }
};

// Explicite Specialisation for const char
template<>
class PrettyPrinter<const char*> {
    const char * m_pData;
public:
    PrettyPrinter(const char * data) : m_pData(data) {
    }
    void Print() {
        cout << "{" << m_pData << "}" << endl;
    }

    const char* GetData() {
        return m_pData;
    }
};

// Print() Function Explicite Specialisatio
template<>
void PrettyPrinter<vector<int> >::Print() {
    cout << "{";
    for(const auto &x : * m_pData) {
        cout << x << ", ";
    }
    cout << "}" << endl;
}
int main() {

    int data = 5;
    float f = 8.2f;

    PrettyPrinter<int> p1(&data);
    p1.Print();
    PrettyPrinter<float> p2(&f);
    p2.Print();

    const char *p{"Hello World"};
```

```
57      PrettyPrinter<const char*> p3(p);
58      p3.Print();
59      const char *pData = p3.GetData();
60
61      vector<int> v1{1,2,3};
62      PrettyPrinter<vector<int> > p4(&v1);
63      p4.Print();
64      return 0;
65  }
```

```
{5}
{8.2}
{Hello World}
{1, 2, 3, }
```

12.16 Assignment

Listing 230: Specialisation for vector<vector<int»

```
1   #include <iostream>
2   #include <vector>
3
4   using namespace std;
5
6   template<typename T>
7   class PrettyPrinter {
8       T * m_pData;
9   public:
10      PrettyPrinter(T *data) : m_pData(data) {
11      }
12      void Print() {
13          cout << "{" << *m_pData << "}" << endl;
14      }
15
16      T * GetData() {
17          return m_pData;
18      }
19  };
20
21
22  // Explicite Specialisation for vector<vector<int>>
23  template<>
24  class PrettyPrinter<vector<vector<int>>> {
25      vector<vector<int>> *m_pData;
26  public:
27      PrettyPrinter(vector<vector<int>> * data) : m_pData(data) {
28      }
29      void Print() {
30
31          for(const auto& row : * m_pData) {
32              cout << "{";
33              for(const auto& x : row) {
34                  cout << x << ", ";
35              }
36              cout << "}" << endl;
37          }
38          cout << endl;
39      }
40
41      vector<vector<int>>* GetData() {
```

```
        return m_pData;
    }
};


int main() {

    vector<vector<int>> v2{{1,2,3}, {4,5,6}};

    PrettyPrinter<vector<vector<int>>> p5(&v2);
    p5.Print();
    return 0;
}
```

```
{1, 2, 3, }
{4, 5, 6, }
```

Listing 231: Function Print() Explicite Specialisation for vector<vector<int»

```
#include <iostream>
#include <vector>

using namespace std;

template<typename T>
class PrettyPrinter {
    T * m_pData;
public:
    PrettyPrinter(T *data) : m_pData(data) {
    }
    void Print() {
        cout << "{" << *m_pData << "}" << endl;
    }

    T * GetData() {
        return m_pData;
    }
};

// Function Print Explicite Specialisation for vector<vector<int>>
template<>
void PrettyPrinter<vector<vector<int>>>::Print() {
    for(const auto& row : * m_pData) {
        cout << "{";
        for(const auto& x : row) {
            cout << x << ", ";
        }
        cout << "}" << endl;
    }
    cout << endl;
}
int main() {

    vector<vector<int>> v2{{1,2,3}, {4,5,6}};

    PrettyPrinter<vector<vector<int>>> p5(&v2);
    p5.Print();
    return 0;
}
```

```
{1, 2, 3, }
{4, 5, 6, }
```

12.17 Class Template Partial Specializuation

- only some of the template parameter are specialized

Listing 232: Class Template: Pretty Printer

```
#include <iostream>

using namespace std;

template<typename T, int columns>
class PrettyPrinter {
    T * m_pData;
public:
    PrettyPrinter(T *data) : m_pData(data) {
    }
    void Print() {
        cout << "Columns: " << columns << endl;
        cout << "{" << *m_pData << "}" << endl;
    }

    T * GetData() {
        return m_pData;
    }
};

template<typename T>
class PrettyPrinter<T, 80> {
    T * m_pData;
public:
    PrettyPrinter(T *data) : m_pData(data) {
    }
    void Print() {
        cout << "Using 80 Columns: " << endl;
        cout << "{" << *m_pData << "}" << endl;
    }

    T * GetData() {
        return m_pData;
    }
};

int main() {

    int data = 800;

    PrettyPrinter<int, 80> p{&data};
    p.Print();

    return 0;
}
```

```
Using 80 Columns:
{800}
```

Listing 233: Class Template: Pretty Printer

```cpp
#include <iostream>

using namespace std;

template<typename T, int columns>
class PrettyPrinter {
    T * m_pData;
public:
    PrettyPrinter(T *data) : m_pData(data) {
    }
    void Print() {
        cout << "Columns: " << columns << endl;
        cout << "{" << *m_pData << "}" << endl;
    }

    T * GetData() {
        return m_pData;
    }
};

template<typename T>
class PrettyPrinter<T, 80> {
    T * m_pData;
public:
    PrettyPrinter(T *data) : m_pData(data) {
    }
    void Print() {
        cout << "Using 80 Columns: " << endl;
        cout << "{" << *m_pData << "}" << endl;
    }

    T * GetData() {
        return m_pData;
    }
};

template<typename T>
class SmartPointer {
    T *m_ptr;
public:
    SmartPointer(T *ptr) : m_ptr(ptr) {
    }
    T * operator ->() {
        return m_ptr;
    }

    T & operator *() {
        return *m_ptr;
    }
    ~SmartPointer() {
        delete m_ptr;
    }
};

template<typename T>
class SmartPointer<T[]> {
    T *m_ptr;
public:
    SmartPointer(T *ptr) : m_ptr(ptr) {
    }
    T & operator [](int index) {
        return m_ptr[index];
```

```
63      }
64      ~SmartPointer() {
65          delete[] m_ptr;
66      }
67  };
68
69
70
71  int main() {
72
73      int data = 800;
74
75      PrettyPrinter<int, 80> p{&data};
76      p.Print();
77      SmartPointer<int[]> s1 {new int[5] };
78      s1[0] = 5;
79      cout << s1[0] << endl;
80      return 0;
81  }
```

```
Using 80 Columns:
{800}
5
```

12.18 TODO Typedef, Type Alias & Alias Templates (C++11)

12.19 Type Traits (C++11)

- C++ does not have any support for introspection at runtime, C++11 introduced tpye traits
- Tpye traits give the ability to introspect
  - find characteristics of types at compile time
  - transform the properties of the type
- Useful in template metaprogramming
- Will either return a boolean or a type when inspecting types
- Provides template-based interface and defined in header *<type_traits>*
- Some traits require support from the compiler
  - compiler provides intrinsics for such traits

12.19.1 Type Traits

is_void	C++11	checks if a type is void
is_null_pointer	C++14	checks if a type is std::nullptr
is_integral	C++11	checks if a type is an integral
is floating_point	C++11	checks if a type is floating-point type
is_array	C++11	checks if a type is an array type
is_enum	C++11	checks if a type is emumeration type
is_union	C++11	checks if a type is union type
is_class	C++11	checks if a type is a non-union class type
is_function	C++11	checks if a type is a function
is_pointer	C++11	checks if a type is pointer type

Listing 234: Traits Example - Runtime Checks

```cpp
#include <iostream>
#include <type_traits>

using namespace std;

template<typename T>
T Divide(T a, T b) {
    if(is_floating_point<T>::value == false) {
^^Icout << "Use floating point types only! ";
^^Ireturn 0;
    }
    return a/b;
}

template<typename T>
void Check(T&&) {
    cout << boolalpha;
    cout << "Is reference? " << is_reference<T>::value << endl;

    cout
^^I<< "After removing: "
^^I<< is_reference<typename remove_reference<T>::type>::value << endl;
}

int main() {

   cout
       << std::boolalpha
       << "Is Integer? " << is_integral<int>::value << endl;

   cout << Divide(5.0f,2.0f) << endl;

   Check(5);    // r-value
   int val {};  // l-value
   Check(val);

   return 0;
}
```

```
Is Integer? true
2.5
Is reference? false
After removing: false
Is reference? true
After removing: false
```

12.20 static_assert (C++11)

- *static_assert* is a new kind of declaration for evaluating expressions
- Accepts an expression and a user defined message
- Used for testing an expression at **compile time**
- If the expression returns false; the compiler displays the specified message and compilation fails.

Listing 235: static-assert - Check Expression at Compile

```cpp
#include <iostream>
#include <type_traits>
```

```
3
4  using namespace std;
5
6  int main() {
7
8      // static_assert( sizeof(void*)==4, "Compile in 32-bit mode only");
9      static_assert( sizeof(void*)==8, "Compile in 64-bit mode only");
10
11     cout << "Code has been compiled successfully" << endl;
12    return 0;
13 }
```

```
Code has been compiled successfully
```

Listing 236: Traits Static Asserts - Compile Time Checks

```
1  #include <iostream>
2  #include <type_traits>
3
4  using namespace std;
5
6  template<typename T>
7  T Divide(T a, T b) {
8
9      static_assert(is_floating_point<T>::value, "Only Floating Point Types supported");
10
11     return a/b;
12 }
13
14 int main() {
15
16    cout << Divide(5.0f,2.0f) << endl;
17
18    return 0;
19 }
```

```
2.5
```

13 Lambdas

13.1 Callbacks Revisted - Function Pointer

limitation of this solution:

- sort only in ascending order
- C-Strings cannot be sorted (compare addresses)
- The swap happens by value, move semenatic where more efficient

Listing 237: Sort Algorithm Template

```
#include <iostream>
#include <string>

using namespace std;

template<typename T, int size>
void Sort(T(&arr)[size]) {
    for(int i = 0; i < size; ++i) {
        for(int j = 0; j < (size -1); ++j) {
            if(arr[j] > arr[j+1]) {     // will become a predicate
                T temp = arr[j];        // swap by value
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}


int main() {

    int arr[] {10, 7, 3, 128, 64, 77, 99 };

    cout << "unsorted values:\t";
    for(const auto& a : arr)
        cout << a << ", ";
    cout << endl;

    cout << "sorted values:\t";
    Sort(arr);
    for(const auto& a : arr)
        cout << a << ", ";
    cout << endl;

    return 0;
}
```

```
unsorted values: 10, 7, 3, 128, 64, 77, 99,
sorted values: 3, 7, 10, 64, 77, 99, 128,
```

Function Pointer

With the help of a function pointer simplifes greatly the usage of the Sort Algorithm. We can control how the sorting should be performed. The function pointer is a Callback Function.

Listing 238: Sort Algorithm Template wiht Function Pointer

```
#include <iostream>
#include <iomanip>
#include <string>

using namespace std;

// Function Prototype:
using Comperator = bool(*)(int,int);

template<typename T, int size>
void Sort(T(&arr)[size], Comperator comp) {
    for(int i = 0; i < size; ++i) {
        for(int j = 0; j < (size -1); ++j) {
            if( comp(arr[j], arr[j+1]) )    {
                T temp   = move(arr[j]);
                arr[j]   = move(arr[j+1]);
                arr[j+1] = move(temp);
            }
        }
    }
}

// Prädicate: returns bool
bool Comp1(int x, int y) {
    return x > y;               // ascedning order
}
bool Comp2(int x, int y) {
    return x < y;               // descending order
}

// C-String: Implement a predicate which usses strcmp.


int main() {

    int arr[] {10, 7, 3, 128, 64, 77, 99 };

    cout << "unsorted values:\t";
    for(const auto& a : arr)
        cout << setw(5) << a;
    cout << endl;

    cout << "ascending order:\t";
    Sort(arr, Comp1);
    for(const auto& a : arr)
        cout << setw(5) << a;;
    cout << endl;

    cout << "descending order:\t";
    Sort(arr, Comp2);
    for(const auto& a : arr)
        cout << setw(5) << a;
    cout << endl;

    return 0;
}
```

```
unsorted values:    10    7    3  128   64   77   99
```

```
ascending order:     3    7   10   64   77   99  128
descending order:  128   99   77   64   10    7    3
```

Limitiation

- Sort is a function template and will be instantiated at compile time
- But comp is invocation of a function through its addess, that is this function cannot be **inlined**.
- Function pointers cannot be optimized by the compiler
- Function pointers point to global functions and global functions can not have a state.
- **Comperator** is therefore a callback implemented as functionn pointer.

13.2 Callbacks - Function Objects



Function Object

Is an object of a class or a structure that has the function operator **operator()** overloaded.

Listing 239: Sort Algorithm Template wiht Function Object

```
1  #include <iostream>
2  #include <iomanip>
3
4  using namespace std;
5
6  // Function Prototype:
7  // using Comperator = bool(*)(int,int);
8
9  template<typename T, int size, typename Comparator> // The type of Comparator is unknown until it is
   ↪  invoked
10 void Sort(T(&arr)[size], Comparator comp) {
11     for(int i = 0; i < size; ++i) {
12         for(int j = 0; j < (size -1); ++j) {
13             if( comp(arr[j], arr[j+1]) )    {
14                 T temp   = move(arr[j]);
15                 arr[j]   = move(arr[j+1]);
16                 arr[j+1] = move(temp);
17             }
18         }
19     }
20 }
21
22 // Prädicate: returns bool
23 bool Comp1(int x, int y) {
24     return x > y;              // ascedning order
25 }
26 bool Comp2(int x, int y) {
27     return x < y;              // descending order
28 }
29
30 // C-String: Implement a predicate which usses strcmp.
31
32 struct Comp3 {
33     bool operator()(int x, int y) {
34         return x > y;
35     }
36 };
37
```

```
38 struct Comp4 {
39     bool operator()(int x, int y) {
40         return x < y;
41     }
42 };
43
44
45
46 int main() {
47
48     int arr[] {10, 7, 3, 128, 64, 77, 99 };
49
50     cout << "unsorted values:\t";
51     for(const auto& a : arr)
52         cout << setw(5) << a;
53     cout << endl;
54
55     // Global Function Call
56     auto val1 = Comp1(3,5);
57     cout << boolalpha << val1 << endl;
58
59     // Local Function call
60     Comp3 comp3;
61     auto val3 = comp3(5,3);  // comp.operator(){3,5};
62     cout << boolalpha << val3 << endl;
63
64     cout << endl;
65     cout << "Usage with Function Pointer:" << endl;
66     cout << "ascending order:\t";
67     Sort(arr, Comp1);
68     for(const auto& a : arr)
69         cout << setw(5) << a;;
70     cout << endl;
71
72     cout << "descending order:\t";
73     Sort(arr, Comp2);
74     for(const auto& a : arr)
75         cout << setw(5) << a;
76     cout << endl;
77
78
79     cout << endl;
80     cout << "Usage with Function Object:" << endl;
81     cout << "ascending order:\t";
82     Sort(arr, comp3);
83     for(const auto& a : arr)
84         cout << setw(5) << a;;
85     cout << endl;
86
87     Comp4 comp4;
88     cout << "descending order:\t";
89     Sort(arr, comp4);
90     for(const auto& a : arr)
91         cout << setw(5) << a;
92     cout << endl;
93
94
95
96     return 0;
97 }
```

```
unsorted values:    10    7    3  128   64   77   99
```

```
false
true

Usage with Function Pointer:
ascending order:     3    7   10   64   77   99  128
descending order:  128   99   77   64   10    7    3

Usage with Function Object:
ascending order:     3    7   10   64   77   99  128
descending order:  128   99   77   64   10    7    3
```

13.2.1 Function Object

- Object with overloaded function call operator it is also know as the **functor**
- Call to overloaded function call operator resembles a local function call
- Can be used as a **callback** instead of function pointers
- More efficient than function pointers
- Usually implemented as structs

13.2.2 Function Pointer vs. Function Objects

Function Pointer	Function Object
Invoked through a pointer	Invoked through an object
Dynamic in nature	Static in nature
Can be specified at runtime	Must be specified at compile time
Difficult to optimize	Easy to optimize (Compiler)
Slow	Fast
Cannot store state	Can store state

13.3 Lambda Expressions

- Defines an anounymous function object
- Syntactic shortcut for a function object
- Can be passed as an argument
- Can accpet parameters and return values
- Typically, encapsulates a few lines of code
- Behaves like a normal function when invoked
- Replaces function objects
- Internally, implemented as a nameless function object
- Use auto to provide an explicite name

Listing 240: Lambda Expressions

```
1 #include <iostream>
2 #include <iomanip>
3
4 using namespace std;
5
6 int main() {
```

```
7
8       []() {
9           cout  << "Welcome to Lambda Expressions 1" << endl;
10      }();  // <- Use the functor to invokde the overloaded function call operator.
11
12      auto fn = []() {  // <- give a name to the anounymous function
13          cout  << "Welcome to Lambda Expressions 2" << endl;
14      };
15      fn();
16
17      cout << typeid(fn).name() << endl;
18
19      return 0;
20  }
```

```
Welcome to Lambda Expressions 1
Welcome to Lambda Expressions 2
Z4mainEUlvE0_
```

13.4 Lambda Expressions - Internals

Listing 241: Lambda Expressions Return Type / Internals

```
1   #include <iostream>
2   #include <iomanip>
3
4   using namespace std;
5
6   template<typename T1, typename T2>
7   struct Unnamed {
8       T2 operator()(T1 x, T2 y) const {
9           return x+y;
10      }
11  };
12
13  int main() {
14
15      auto sum1 = [](int x, int y) {              // return type is optional (same as arguments)
16          return x + y;
17      };
18      cout << sum1(3,4) << endl;
19
20      auto sum2 = [](auto x, auto y)noexcept{
21          return x + y;
22      };
23      cout << sum2(4, 4.3) << endl;
24
25      // same as
26      Unnamed<int,double> uname;
27      cout << uname(4, 4.3) << endl;
28
29
30      return 0;
31  }
```

```
7
8.3
8.3
```

13.5 Lambda Expressions Capture List - Part I

Listing 242: A possible implementation of a Lampda Experession

```
#include <iostream>
#include <iomanip>

using namespace std;

template<typename T>
struct __Unnamed {
    int offset;
    __Unnamed(int off) : offset(off) {
    }

    // To modifiy outher variable, the Functor must not be const wh
    void operator()(T &x) /*const*/ {
        x += offset;
        ++offset;
    }
};

int main() {

    __Unnamed<int> n(3);  // Offset = 3
    int x = 5;
    n(x);
    cout << x << endl;

    return 0;
}
```

```
8
```

Listing 243: Lambda Expressions Capture List

```
#include <iostream>
#include <iomanip>

using namespace std;

template<typename T, int size, typename Callback>
void ForEach(T(&arr)[size], Callback operation) {
    for(int i=0; i<size; ++i) {
        operation(arr[i]);
    }
}

int main() {

    int arr[] {10, 7, 3, 128, 64, 77, 99 };
    cout << "Input Array:\t";
    ForEach(arr, [](auto x) { cout << setw(5) << x << " ";});

    int offset = 5;
    cout << endl;
    cout << "With Offset 5:\t";
    ForEach(arr, [offset](auto &x) {x += offset;});   // offset is captured by value
    ForEach(arr, [](auto x) { cout << setw(5) << x << " ";});

    // MUTABLE REMOVES THE CONST FROM THE FUNCTOR
```

```
    cout << endl;
    cout << "Mutable Offset:\t";
    ForEach(arr, [offset](auto &x) {x -= offset;});   // set to original values
    ForEach(arr, [offset](auto &x) mutable { x += offset; ++offset; });
    ForEach(arr, [](auto x) { cout << setw(5) << x << " ";});

    return 0;
}
```

```
Input Array:    10     7     3   128    64    77    99
With Offset 5:    15    12     8   133    69    82   104
Mutable Offset:    15    13    10   136    73    87   110
```

- *mutable* removes the *const* expression of the constant function call operator

13.6 Lambda Expression Capture List - Part II

Listing 244: Capture List Modes

```
#include <iostream>
#include <iomanip>

using namespace std;

template<typename T, int size, typename Callback>
void ForEach(T(&arr)[size], Callback operation) {
    for(int i=0; i<size; ++i) {
        operation(arr[i]);
    }
}

int main() {

    int arr[] {10, 7, 3, 128, 64, 77, 99 };
    cout << "Input Array:\t";
    ForEach(arr, [](auto x) { cout << setw(5) << x << " ";});

    cout << "\nArray Sum:\t";
    int sum {};
    ForEach(arr, [&sum](auto &x) {sum += x;});   // sum is captured by reference
    cout <<setw(5) << sum << endl;

    return 0;
}
```

```
Input Array:    10     7     3   128    64    77    99
Array Sum:   388
```

13.6.1 Capture List Modes

Capture Mode	Explanation
[var]	Capture *var* by value
[=]	Capture all enclosing scoope variables by value
[&var]	Capture all enclosing scoope variables by reference
[&,var]	Capture *var* by value & other variables by reference
[=, &var]	Capture *var* by reference & other variables by value
[this]	Capture *this* (captures all member variables)

13.7 Lambda Expression Capture List - Part III

- Capture List is only required for local variables.
- Global and Static variables are automatically captured

Listing 245: Capture Mode this-Pointer

```
#include <iostream>
#include <iomanip>
#include <string>

using namespace std;

template<typename T, int size, typename Callback>
void ForEach(T(&arr)[size], Callback operation) {
    for(int i=0; i<size; ++i) {
        operation(arr[i]);
    }
}

class Product {

    string name;
    float price;
public:
    Product(const string &n, float p) : name(n), price(p) {

    }
    void AssignFinalPrice() {
        float taxes[] {12, 5, 5};
        float basePrice{ price };
        ForEach(taxes, [basePrice, this](float tax) {  // capture all the member variables by value
            float taxedPrice = basePrice * tax /100;
            price += taxedPrice;
        });
    }
    float GetPrice() {
        return price;
    }
    const string& GetName() {
        return name;
    }

};
int main() {

    Product p{"Watch", 500};
    p.AssignFinalPrice();
    cout << p.GetName() << " " << p.GetPrice() << endl;

    return 0;
}
```

```
Watch 610
```

Listing 246: Lambda Expression within Lambda Expression

```
#include <iostream>
#include <iomanip>
```

```
#include <string>

using namespace std;

int main() {

    [](int x) {
        x *= 2;
        [](int x) {
            cout << x << endl;
        }(x);
    }(5);

    return 0;
}
```

```
10
```

Callback in C-Functions

If a lambda expresson does not specify a capture clause, then it automatically decompose to a function pointer.

Therefore lambda expression can be used as **callbacks in C functions**.

Listing 247: Lambda Expression as C-Function Callback

```
#include <iostream>

using namespace std;

int main() {

    // int atexit( void (*func)(void) );
    atexit([]() { cout << "Program is exitting.." << endl;});

    return 0;
}
```

```
Program is exitting..
```

13.8 Generalized Capture (C++14)

- This feature allows creation of new variables in the capture clause
- The type of these variables is deduced from the type produced by the expression
- Consequently, these variables must always be initialized
- If the initializer expression is a variable, the new variable can have the same or a different name
- To create a reference, you have to use *operator &* before the variable name
  - *[var=expression](args)*
  - *[&var=expression(args)*

Listing 248: Generalized Capture List

```cpp
#include <iostream>

using namespace std;

int main() {

    int x {5};
    auto f = [y=x](int arg) {
        return y + arg;
    };

    cout << f(5) << endl;

    return 0;
}
```

```
10
```

Listing 249: Generalized Capture List

```cpp
#include <iostream>
#include <fstream>

using namespace std;

int main() {

    ofstream out{"../projects/file.txt"};
    // out shall be used only within the lambda expression
    auto write = [out = move(out)] (int x) mutable {

        out << x;
    };

    write(200);

    return 0;
}
```

13.9 Assignement

Listing 250: T MAX(T,T); // Return largest Element

```cpp
#include <iomanip>
#include <iostream>

using namespace std;

template<typename T, int size, typename Callback>
void ForEach(T(&arr)[size], Callback operation) {
    for(int i=0; i<size; ++i) {
        operation(arr[i]);
    }
}



int main() {

    int arr[] {10, 7, 3, 128, 64, 77, 99 };
```

```
18      cout << "Input Array:\t";
19      ForEach(arr, [](auto x) { cout << setw(5) << x << " ";});
20
21      int Max {};
22      ForEach(arr, [&Max](auto x){ Max = Max > x ? Max : x; });
23
24      cout << endl;
25      cout << "Max Value is:\t" << setw(5) << Max;
26
27      return 0;
28  }
```

```
Input Array:    10     7     3   128    64    77    99
Max Value is:   128
```

Listing 251: Greater

```
1   #include <iomanip>
2   #include <iostream>
3
4   using namespace std;
5
6   template<typename T, typename Callback>
7   bool Greater(T a, T b, Callback operation) {
8       return operation(a, b);
9   }
10
11
12  int main() {
13
14      auto i = Greater<int>(7, 8, [](int a, int b){ return ( a > b ? true : false); });
15      cout << boolalpha << i << endl;
16
17      auto c = Greater<char>('8', '1', [](const char a, const char b){ return ( atoi(&a) > atoi(&b) ?
        ↪  true : false); });
18      cout << boolalpha << c << endl;
19
20      auto G = [](int a, int b) {return (a > b ? true : false);}(8,7);
21      cout << boolalpha << G << endl;
22      return 0;
23  }
```

```
false
true
true
```

14 Function Rappers (C++11)

14.1 std::function - I

- A general-purpose function wrapper
- Can **store** any entity thac can be **invoke** like a function. and **target** that can be invoked with
- We can store **function pointers** & **function objects** inside *std::function*.
- Stored object is callted the **target**
- This target can be invoked with the same syntax through *std::function* object
- Invocation without a target throws *std::bad_function_call* exception

std::function Usage . std::function is a class template: *std::function<function signature> object = {callable};*

Listing 252: std::function Examples

```
#include <functional>
int Square (int);
int *Alloc(size_t_bytes);

std::function<int(int)>        f1 = Square;                        f1(3);
std::function<int*(size_t)>    f2 = Alloc;                         f3(1024);
std::function<bool(int)>       f3 = [](int x) {return x%2 == 0;});    f3(3);
```

14.2 std::function - II

Listing 253: std::function Solve Problems

```
#include <iostream>
#include <functional>

using namespace std;

int Square(int x) {
    return x*x;
}

int Add(int x, int y) {
    return x + y;
}

struct Max {
    int operator()(int x, int y) const {
        return x > y ? x : y;
    }
};


int main() {

    try {
        function<int(int)> f1 /* = Square */;  // f1 can hold any address that matches the signature
        if(f1) {                               // check for target
            cout << f1(3) << endl;
        } else {
            cout << "No target to invoke!" << endl;
        }

```

```
        Max m {};
        function<int(int,int)> f2 = m;
        cout << f2(7, 8) << endl;

        function<bool(int)> f3 = [](int x) {return x%2 == 0;};
        cout << "is even: " << boolalpha << f3(8) << endl;
    }
    catch (const std::exception& ex) {
        cout << "Exception -> " << ex.what() << endl;
    }
    return 0;
}
```

```
No target to invoke!
8
is even: true
```

Listing 254: std::function Usage

```
#include <iostream>
#include <functional>

using namespace std;

int Square(int x) {
    return x*x;
}

int Add(int x, int y) {
    return x + y;
}

int Subtract(int x, int y) {
    return x - y;
}


struct Max {
    int operator()(int x, int y) const {
        return x > y ? x : y;
    }
};

int Operation(int x, int y, function<int(int,int)>  callback) {
    return callback(x,y);
}

int main() {

    try {
        function<int(int, int)> f1 = Add;
        cout << Operation(3,5, f1) << endl;
        f1 = Subtract;
        cout << Operation(3,5, f1) << endl;
        cout << Operation(3,5, [](int x, int y) {return x*y;}) << endl;
    }
    catch (const std::exception& ex) {
        cout << "Exception -> " << ex.what() << endl;
    }
    return 0;
}
```

```
8
-2
15
```

14.3 std::bind - I

- Wrapper that can bind some arguments to a given function
- The arguments to the target function are represented using placeholders or directly bound to specific values.
- Placeholders are represented using constant objects of unspecified type in *std::placehlders* namespace
  - named _1, _2,...,_N
- Can be used to invoke a function that accepts n arguments as a function with n-m arguments
- Returns a function object through which the target is invoked

std::function Usage . std::bind is a class template: *std::bind<callable, arg1, arg2,...argN>;*

Listing 255: std::bind Examples

```
#include <functional>
int Square (int);
int Print(const char *);

auto f1 = std::bind(Square 3);     f1();
auto f2 = std::bind(Print, _1);    f2("Hello");
```

14.4 std::bind - II

Listing 256: std::bind Usage

```
#include <iostream>
#include <functional>

using namespace std;

int Square(int x) {
    return x*x;
}

int Add(int x, int y) {
    return x + y;
}

int Subtract(int x, int y) {
    return x - y;
}


struct Max {
    int operator()(int x, int y) const {
        return x > y ? x : y;
    }
};

int Operation(int x, int y, function<int(int,int)>  callback) {
    return callback(x,y);
}

```

```
29 int main() {
30
31     std::function<int(int,int)> f1 = bind(Subtract, std::placeholders::_1, std::placeholders::_2);
32     cout << f1(13,25) << endl;  // Subtract(12,25)
33
34     std::function<int(int,int)> f2 = bind(Subtract, std::placeholders::_2, std::placeholders::_1);
35     cout << f2(13,25) << endl;  // Subtract(25,12)
36
37
38     return 0;
39 }
```

```
-12
12
```

Listing 257: std::bind Usage - Syntax Aid

```
1  #include <iostream>
2  #include <functional>
3
4  using namespace std;
5  using namespace placeholders;
6
7  int Square(int x) {
8      return x*x;
9  }
10
11 int Add(int x, int y) {
12     return x + y;
13 }
14
15 int Subtract(int x, int y) {
16     return x - y;
17 }
18
19 void Print(const char * p) {
20     cout << p << endl;
21 }
22
23 struct GreaterThan {
24     bool operator()(int x, int y) const {
25         return x > y;
26     }
27 };
28
29 struct Max {
30     int operator()(int x, int y) const {
31         return x > y ? x : y;
32     }
33 };
34
35 int Operation(int x, int y, function<int(int,int)>  callback) {
36     return callback(x,y);
37 }
38
39 int Accumulate(int x, int y, int z) {
40     return x + y + z;
41 }
42
43 int main() {
44
```

```
45      auto f1 = bind(Subtract, _1, _2);
46      cout << f1(13,25) << endl;  // Subtract(12,25)
47
48      auto f2 = bind(Subtract, _2, _1);
49      cout << f2(13,25) << endl;  // Subtract(25,12)
50
51      auto f3 = bind(Subtract, 23, 11);
52      cout << f3() << endl;  // Subtract(23,11)
53
54      auto f4 = bind(Subtract, _1, 5);
55      cout << f4(100) << endl;  // Subtract(100,5)
56
57      auto gt = bind(GreaterThan{}, -1,-2);
58      cout << boolalpha << gt(5,7) << endl;
59
60      auto acc = bind(Accumulate, _1, _2, 0);
61      cout << Operation(8, 5, acc) << endl;
62
63      return 0;
64  }
```

```
-12
12
12
95
true
13
```

14.5 std::bind - III

Listing 258: std::bind Replacement by lambda function

```
1   #include <iostream>
2   #include <functional>
3
4   using namespace std;
5   using namespace placeholders;
6
7   int Square(int x) {
8       return x*x;
9   }
10
11  int Add(int x, int y) {
12      return x + y;
13  }
14
15  int Subtract(int x, int y) {
16      return x - y;
17  }
18
19  void Print(const char * p) {
20      cout << p << endl;
21  }
22
23  struct GreaterThan {
24      bool operator()(int x, int y) const {
25          return x > y;
26      }
27  };
28
```

```
struct Max {
    int operator()(int x, int y) const {
        return x > y ? x : y;
    }
};

int Operation(int x, int y, function<int(int,int)>  callback) {
    return callback(x,y);
}

int Accumulate(int x, int y, int z) {
    return x + y + z;
}

int main() {

    // auto f4 = bind(Subtract, _1, 5);
    auto f4 = [](int x) {return Subtract(x,5);};
    cout << f4(100) << endl;  // Subtract(100,5)

    // auto gt = bind(GreaterThan{}, -2,-1);
    auto gt = [](int x, int y){return GreaterThan{}(y,x); };
    cout << boolalpha << gt(5,7) << endl;

    //auto acc = bind(Accumulate, _1, _2, 0);
    auto acc = [](int x, int y){return Accumulate(x,y,0);};
    cout << Operation(8, 5, acc) << endl;

    return 0;
}
```

```
95
true
13
```

Lambda Expression

Binder are superseeded by lambda expressions since they are mutch easier to use.

Listing 259: std::bind Replacement by lambda function and correct type forewarding also see C++20

```
#include <iostream>
#include <functional>

using namespace std;
using namespace placeholders;

int Square(int x) {
    return x*x;
}

int Add(int x, int y) {
    return x + y;
}

int Subtract(int x, int y) {
    return x - y;
}

```

```
19 void Print(const char * p) {
20     cout << p << endl;
21 }
22
23 struct GreaterThan {
24     bool operator()(int x, int y) const {
25         return x > y;
26     }
27 };
28
29 struct Max {
30     int operator()(int x, int y) const {
31         return x > y ? x : y;
32     }
33 };
34
35 int Operation(int x, int y, function<int(int,int)>  callback) {
36     return callback(x,y);
37 }
38
39 int Accumulate(int x, int y, int z) {
40     return x + y + z;
41 }
42
43 int main() {
44
45     // Type correctness in C++17
46     auto f4 = [](auto && x) {return Subtract(forward<decltype(x)>(x),5);};
47     cout << f4(100) << endl;  // Subtract(100,5)
48
49     // C++20 template type parameters in lambda
50     auto gt = []<typename T>(T && x, T && y){return GreaterThan{}(forward<T>(y),forward<T>(x)); };
51     cout << boolalpha << gt(5,7) << endl;
52
53     // C++20 template type parameters in lambda
54     auto acc = []<typename T>(T && x, T && y){return Accumulate(forward<T>(x),forward<T>(y),0);};
55     cout << Operation(8, 5, acc) << endl;
56
57     return 0;
58 }
```

```
95
true
13
```

15 Standard Template Library



Remark

These videos have all been watched but the details are better looked up in cppreference.com or TC++PL as to reproduce everything here.

15.1 Introduction

- Heart of the standard library for C++
- Developed by Alexander Stepnov & Meng Lee at Hewlett Packard
- Provides fas & reusable **containers** and **algorithms**
- Relies heavily on templates; almost every component in the STL is a template.

Core Components

- **Container** classes, **algorithms** & **iterators** form the core components of the STL
- Container classes represent data & algorithms represent operations on the data
- Iterators serve as the **glue** between containers and algorithms
- Also include classes for **concurrency**, **random numbers**, **regular expressions**, **utilities**, etc.

Why use the STL

- Quicker development
- Reliable
- Portable
- Efficient & fast
- Accurate
- Readable code
- Lower maintenance cost

Container Types

Sequence	Header
array	<array
vector	<vector>
list	<list>
deque	<deque>
forward_list	<forward_list

Associative	Header
set, multiset	<set>
map, multimap	<map>

Unsorted	Header
unordered_set; unordered_multiset	<unordered_set>
unordered_mp; unordered_multimap	<unordered_map>

Common Functions

- default constructor
- uniform initialization constructor
- copy constructor
- iterator constructor
- size()
- clear()
- begin()
- end()
- default allocator

15.2 std::array

Listing 260: std::function Solve Problems

```
#include <iostream>
#include <functional>

using namespace std;

int main() {

    return 0;
}
```

```
No target to invoke!
8
is even: true
```

15.3 std::vector
15.4 std::deque
15.5 std::list std::forward_list (C++11)
15.6 std::set & std::multiset
15.7 std::map & std::multimap
15.8 Unordered Containers (C++11) - I
15.9 Unordered Containers (C++11) - II
15.10 std::hash (C++11)
15.11 Big O Notation & Performance of Conatainers
15.12 Algorithm - Part I
15.13 Algorithm - Part II
15.14 Container Changes in C++11 - I
15.15 Container Changes in C++11 - II
15.16 Container Changes in C++11 - III
15.17 Container Changes in C++11 - IV
15.18 Container Changes in C++11 - V

16 C++ Concurrency

16.1 Concurrency Basics

- Large application have multiple components
- Some components may have to execute concurrently
- This allows efficient usage of the CPU
- This can be achievd by creating multiple threads
- C++11 added support for concurrency
- Threding library includes utilities for starting and managing threads

Listing 261: Single Thread Application

```cpp
#include <iostream>
#include <list>

std::list<int> g_Data;
const int SIZE = 5000000;

void Download() {
    std::cout << "[Downloader]Started download of file :" << std::endl;
    for (int i = 0; i < SIZE; ++i) {
        g_Data.push_back(i);
    }
    std::cout << "[Downloader]Finished download" << std::endl;
}
int main() {
    std::cout << "[main]User started an operation" << std::endl;
    Download();
    std::cout << "[main]User started another operation" << std::endl;

    return 0;
}
```

```
[main]User started an operation
[Downloader]Started download of file :
[Downloader]Finished download
[main]User started another operation
```

16.2 std::thread

- Accepts a callable as constructor argument
- The callable is executed in a seperate thread
- The constructor does not wait for the thread to start it returns immediately.
- Resides in <thread> header

Listing 262: Joinable Thread Application

```cpp
#include <iostream>
#include <list>
#include <thread>

```

```
std::list<int> g_Data;
const int SIZE = 5000000;

void Download() {
    std::cout << "[Downloader]Started download of file :" << std::endl;
    for (int i = 0; i < SIZE; ++i) {
        g_Data.push_back(i);
    }
    std::cout << "[Downloader]Finished download" << std::endl;
}
int main() {
    std::cout << "[main]User started an operation" << std::endl;
    std::thread theDownloader(Download);
    std::cout << "[main]User started another operation" << std::endl;

    theDownloader.join();
    return 0;
}
```

```
[main]User started an operation
[main]User started another operation
[Downloader]Started download of file :
[Downloader]Finished download
```

Listing 263: Detached Thread Application

```
#include <iostream>
#include <list>
#include <thread>

std::list<int> g_Data;
const int SIZE = 5000000;

void Download() {
    std::cout << "[Downloader]Started download of file :" << std::endl;
    for (int i = 0; i < SIZE; ++i) {
        g_Data.push_back(i);
    }
    std::cout << "[Downloader]Finished download" << std::endl;
}
int main() {
    std::cout << "[main]User started an operation" << std::endl;
    std::thread theDownloader(Download);
    theDownloader.detach();

    std::cout << "[main]User started another operation" << std::endl;

    //theDownloader.join();
    // std::cout << "Press Enter...\n";
    // std::cin.get();
    return 0;
}
```

```
[main]User started an operation
[main]User started another operation
[Downloader]Started download of file :
```

16.3 std::thread Passing Arguments

Listing 264: Passing Arguments by Value

```
#include <iostream>
#include <list>
#include <thread>
#include <string>

using namespace std;

list<int> g_Data;
const int SIZE = 5000000;


void Download(string file) {
    cout << "[Downloader]Started download of file :" << file << endl;
    for (int i = 0; i < SIZE; ++i) {
        g_Data.push_back(i);
    }
    cout << "[Downloader]Finished download" << endl;
}
int main() {
    string file {"cppcast.mp4"};
    cout << "[main]User started an operation" << endl;

    //use ref and cref to pass arguments as reference & const reference, respectively
    thread thDownloader(Download, file );
    //thDownloader.detach();

    cout << "[main]User started another operation" << endl;

    //Always check before joining a thread
    if (thDownloader.joinable()) {
        thDownloader.join() ;
    }
    //system("Pause");
    return 0;
}
```

```
[main]User started an operation
[main]User started another operation
[Downloader]Started download of file :cppcast.mp4
[Downloader]Finished download
```

Listing 265: Passing Arguments by Reference

```
#include <iostream>
#include <list>
#include <thread>
#include <string>
std::list<int> g_Data;
const int SIZE = 5000000;
class String {
public:
    String() {
        std::cout << "String()" << std::endl;
    }
    String(const String &) {
        std::cout << "String(const String&)" << std::endl;
    }
    String & operator=(const String&) {
        std::cout << "operator=(const String&)" << std::endl;
```

```
17
18          return *this;
19      }
20      ~String() {
21          std::cout << "~String()" << std::endl;
22      }
23  };
24  void Download(const String &file) {
25      (void)file;
26      //std::cout << "[Downloader]Started download of file :" << file << std::endl;
27      for (int i = 0; i < SIZE; ++i) {
28          g_Data.push_back(i);
29      }
30      std::cout << "[Downloader]Finished download" << std::endl;
31  }
32  int main() {
33      String file;
34      std::cout << "[main]User started an operation" << std::endl;
35      //use std::ref and std::cref to pass arguments as reference & const reference, respectively
36      std::thread thDownloader(Download,std::cref(file) );
37      //thDownloader.detach();
38
39      std::cout << "[main]User started another operation" << std::endl;
40      //Always check before joining a thread
41      if (thDownloader.joinable()) {
42          thDownloader.join() ;
43      }
44      //system("Pause");
45      return 0;
46  }
```

```
String()
[main]User started an operation
[main]User started another operation
[Downloader]Finished download
~String()
```

16.4 std::thread Returning Values

Listing 266: packaged-task return value

```
1   #include <iostream>
2   #include <vector>
3   #include <thread>
4   #include <future>
5
6   using namespace std;
7
8   int Add(int x, int y) {
9       return x + y;
10  }
11
12  int Square(int x) {
13      return x * x;
14  }
15
16  int Compute(const vector<int> &data) {
17
18      int sum {};
19      for(auto e : data) {
```

```
20          sum += e;
21          this_thread::sleep_for(1s);
22          cout << ".";
23      }
24      return sum;
25  }
26
27  int main() {
28
29      packaged_task<int(const vector<int> &)> tCompute {Compute};
30      auto fCompute = tCompute.get_future();
31      vector<int> data {1,2,3,4,5};
32      tCompute(data);                         // blocks main function!!!
33      cout << fCompute.get() << endl;
34      cout << "End fo main()" << endl;
35
36      return  0;
37  }
```

```
8
25
.....15
End fo main()
```



packaged task

Packaged tasks blocks the main thread

Listing 267: packaged-task<> run in seperate thread

```
1   #include <iostream>
2   #include <vector>
3   #include <thread>
4   #include <future>
5
6   using namespace std;
7
8   int Add(int x, int y) {
9       return x + y;
10  }
11
12  int Square(int x) {
13      return x * x;
14  }
15
16  int Compute(const vector<int> &data) {
17
18      int sum {};
19      for(auto e : data) {
20          sum += e;
21          this_thread::sleep_for(1s);
22          cout << ".";
23      }
24      return sum;
25  }
26
27  int main() {
28
29      packaged_task<int(const vector<int> &)> taskCompute {Compute};
```

```
30      auto futureCompute = taskCompute.get_future();
31      vector<int> data {1,2,3,4,5};
32
33      thread threadCompute{ move(taskCompute), data};
34      cout << "Thread has started computing" << endl;
35      cout << futureCompute.get() << endl;                // blocks main
36
37      cout << "End fo main()" << endl;
38      threadCompute.join();
39      return  0;
40  }
```

```
Thread has started computing
.....15
End fo main()
```

16.5 std::thread Synchronization std:mutex

Listing 268: Synchronization

```
1   #include <iostream>
2   #include <list>
3   #include <thread>
4   #include <mutex>
5
6   using namespace std;
7
8   list<int> g_Data;
9   const int SIZE = 10000;
10  mutex mux_data;
11
12  void Download() {
13      for (int i = 0; i < SIZE; ++i) {
14          mux_data.lock();
15          g_Data.push_back(i);
16          // if(i == 500) {               // generates a deadlock
17          //     return;
18          // }
19          mux_data.unlock();
20      }
21  }
22
23  void Download2() {
24      for (int i = 0; i < SIZE; ++i) {
25          mux_data.lock();
26          g_Data.push_back(i);
27          mux_data.unlock();
28      }
29  }
30
31  int main() {
32      thread theDownloader(Download);
33      thread theDownloader2(Download2);
34
35      theDownloader.join();
36      theDownloader2.join();
37
38      cout << g_Data.size() << endl;
39
40      return 0;
41  }
```

20000

16.6 std::thread Synchronization std::lock_guard

Listing 269: Synchronization lock-guard

```
#include <iostream>
#include <list>
#include <thread>
#include <mutex>

using namespace std;

list<int> g_Data;
const int SIZE = 10000;
mutex mux_data {};

void Download() {
    for (int i = 0; i < SIZE; ++i) {
        lock_guard<mutex> mtx(mux_data);   // RAII mux_data is destryed when mtx exits.
        if(i == 500)
            return;
    }
}

void Download2() {
    for (int i = 0; i < SIZE; ++i) {
        lock_guard<mutex> mtx(mux_data);
        g_Data.push_back(i);
    }
}

int main() {
    thread theDownloader(Download);
    thread theDownloader2(Download2);

    theDownloader.join();
    theDownloader2.join();

    cout << g_Data.size() << endl;

    return 0;
}
```

10000



CPU utilsation

As long as the resource is locked, the thread is sleeping thus the CPU utilisation might not be efficient.

A better solution might be not to share the data.

16.7 std:Sthread Functions & std::this Namespace

Listing 270: More Thread Features

```
#include <iostream>
#include <list>
```

```cpp
#include <vector>
#include <thread>
#include <mutex>
#include <pthread.h>

using namespace std;

void Process() {

    pthread_setname_np(pthread_self(), "Worker-01");
    this_thread::sleep_for(chrono::seconds(2));
}

int main() {
    thread t1(Process);

    auto id1 = t1.get_id();
    cout << id1 << endl;

    auto handle = t1.native_handle();
    // pthread_setname_np(handle, "Worker-011");

    vector<char> buffer(16);  // the kernel may hold only 15 characters
    pthread_getname_np(handle, buffer.data(), buffer.size());
    std::cout << "Name: " << buffer.data() << '\n';

    int cores = thread::hardware_concurrency();
    cout << "Cores: " << cores << endl;
    t1.join();

    return 0;
}
```

```
140586819524288
Name: Worker-01
Cores: 22
```

16.8 Task Based Concurrency - Part I

Listing 271: High Level Concurrency

```cpp
#include <iostream>
#include <future>
#include <thread>

using namespace std;
using namespace chrono_literals;

void Downloader() {
    for (int i = 0; i < 10; ++i) {
        cout << '.' ;
        this_thread::sleep_for(300ms);
    }
}

int main() {

    future<void> result = async(Downloader);
    cout << "main() threading continues execution..";

```

```
20      result.get();   // simiöar to jon
21
22      return 0;
23  }
```

```
main() threading continues execution.....
```

16.9 Task Based Concurrency - Part II

16.9.1 std::async

- Part of high-level concurrency
- Executes a *callable* object of a function in a seperate thread
- Returns a *std::future* objec that provides access to the result of the callable
- Include the header *<future>*
  future<return_type> async (Callable, args) future<return_type> async(launch policy, Callable, args)

16.9.2 Launch Policy

- *std::launch::deferred* - the task is executed synchronously
- *std::launch::async* - the task is executed asynchronoulsy
- Call to *std::async* without a launch policy may not always create a new thread
  - depends on the compiler implementation
  - To force *std:async* to execute the task asynchronoulsy, use the async lauch policy explicittly
- If new thread cannot be created with async launch policy, *std::system_error* exception is thrown

16.9.3 Task Arguments & Return Value

- Arguments can be specified in *std::async* for the task
- By default, the arguments are passed by value
- To pass by reference, use the reference wrapper, *std::ref*
  - for constant reference, use *std::cref*
- To get a value from a task, simply return it from the corresponding function
- This value is than availble throuhg the future object

16.9.4 std::future

- Used for communication between threads
- Has a **shared state** that can be accessed in a different thread
- Create through a *std::promise* object
  - created by *std::async*, that directly returns a future object
  - *std::promise* is an **input** channel
  - *std::future* is the **output** channel
- The thread that reads the shared state will wait until the future is ready (with the shared state)
- Promise/future pair allow safe data sharing between threads

16.10 Launch Policies

Listing 272: Launch Policies

```
#include <iostream>
#include <future>
#include <thread>

using namespace std;
using namespace chrono_literals;

int Operation(int count) {
    int sum {};
    for (int i = 0; i < count; ++i) {
        sum += i;
        cout << '.';
        this_thread::sleep_for(300ms);
    }
    return sum;
}

int main() {

    future<int> result = async(launch::async, Operation, 10);
    // future<int> result = async(launch::deferred, Operation, 10);
    this_thread::sleep_for(1s);
    cout << "main() thread continues execution..";
    if(result.valid()) {
        auto sum = result.get();
        cout << "Sum: " << sum << endl;   // simiöar to jon
    }
    return 0;
}
```

```
....main() thread continues execution........Sum: 45
```

get()

get() will make the main thread wait.

The shared state is destroyed and cannot get it again and the future becomes invalid.

deferred

With deferred launch policy the Task gets invoked when you try to get the shared state of the task. And we get the shared state of the task when calling get() on the future.

16.11 std::future Wait Functions

Listing 273: Wait

```
#include <iostream>
#include <future>
#include <thread>

using namespace std;
using namespace chrono_literals;
```

```

int Operation(int count) {
    int sum {};
    for (int i = 0; i < count; ++i) {
        sum += i;
        cout << '.';
        this_thread::sleep_for(300ms);
    }
    return sum;
}

int main() {

    future<int> result = async(launch::async, Operation, 10);
    // future<int> result = async(launch::deferred, Operation, 10);
    this_thread::sleep_for(1s);
    cout << "main() thread continues execution..";
    if(result.valid()) {
        result.wait();
        auto sum = result.get();
        cout << "Sum: " << sum << endl;   // simiöar to jon
    }
    return 0;
}
```

```
....main() thread continues execution........Sum: 45
```

Listing 274: Wait For

```
#include <iostream>
#include <future>
#include <thread>

using namespace std;
using namespace chrono_literals;

int Operation(int count) {
    int sum {};
    for (int i = 0; i < count; ++i) {
        sum += i;
        cout << '.';
        this_thread::sleep_for(300ms);
    }
    return sum;
}

int main() {

    future<int> result = async(launch::async, Operation, 10);
    // future<int> result = async(launch::deferred, Operation, 10);
    this_thread::sleep_for(1s);
    cout << "main() thread continues execution..";
    if(result.valid()) {
        auto status = result.wait_for(4s);   // wait for certain amount of time

        switch(status) {
        case future_status::deferred: // wait not possible
            cout << "Task is synchronous\n";
            break;
        case future_status::ready:  // the result is ready
            cout << "Result is Ready\n";
```

```
            break;
        case future_status::timeout:
            cout << "Task is still running\n";
            break;
        }
        auto sum = result.get();
        cout << "Sum: " << sum << endl;   // simiöar to jon
    }
    return 0;
}
```

```
....main() thread continues execution........Result is Ready
Sum: 45
```

Listing 275: Wait Until

```
#include <iostream>
#include <future>
#include <thread>

using namespace std;
using namespace chrono_literals;

int Operation(int count) {
    int sum {};
    for (int i = 0; i < count; ++i) {
        sum += i;
        cout << '.';
        this_thread::sleep_for(300ms);
    }
    return sum;
}

int main() {

    future<int> result = async(launch::async, Operation, 10);
    // future<int> result = async(launch::deferred, Operation, 10);
    this_thread::sleep_for(1s);
    cout << "main() thread continues execution..";
    if(result.valid()) {

        auto timepoint = std::chrono::system_clock::now();
        timepoint += 1s;
        auto status = result.wait_until(timepoint);   // wait until timepoint is reached
        switch(status) {
        case future_status::deferred: // wati not possible
            cout << "Task is synchronous\n";
            break;
        case future_status::ready:  // the result is ready
            cout << "Result is Ready\n";
            break;
        case future_status::timeout:
            cout << "Task is still running\n";
            break;
        }
        auto sum = result.get();
        cout << "Sum: " << sum << endl;   // simiöar to jon
    }
    return 0;
}
```

```
....main() thread continues execution.....Task is still running
...Sum: 45
```

16.12 std::promise

- Provides a way to store a value or an exception
- This is called the **shared state**
- This state can be accessed later from another thread through a *future* object
- *promise/future* are two *endpoints of a communication channel
- One operation stores a value in a *promise* and the other operation will retrieve it through a *future* **asynchronoulsy**
- These operations are **synchronized**, therefore **thread-safe**
- *promise* object can be used only once

Listing 276: Promise - Future

```
#include <iostream>
#include <future>
#include <thread>

using namespace std;
using namespace chrono_literals;

int Operation(std::promise<int>& data) {
    auto f = data.get_future();                 // get future
    cout <<"[task] waiting for count" << endl;
    auto count = f.get();                       // get data
    int sum {};
    for (int i = 0; i < count; ++i) {
        sum += i;
        cout << '.';
        this_thread::sleep_for(300ms);
    }
    return sum;
}

int main() {

    for(int i = 0; i < 3; ++i) {
        promise<int> data;
        future<int> result = async(launch::async, Operation, std::ref(data));
        this_thread::sleep_for(100ms);
        cout << "[main] Setting the data in promise: ";
        data.set_value(10+i);           // set data
        if(result.valid()) {

            auto sum = result.get();
            cout << "Sum: " << sum << endl;   // simiöar to jon
        }
        cout << '\n';
    }
    return 0;
}
```

```
[task] waiting for count
[main] Setting the data in promise: ..........Sum: 45

[task] waiting for count
[main] Setting the data in promise: ...........Sum: 55

[task] waiting for count
[main] Setting the data in promise: ............Sum: 66
```

Listing 277: Promise - Future

```
#include <iostream>
#include <future>
#include <thread>

using namespace std;
using namespace chrono_literals;

int Operation(std::future<int>& f) {
    cout <<"[task] waiting for count" << endl;
    auto count = f.get();                       // get data (blocking)
    int sum {};
    for (int i = 0; i < count; ++i) {
        sum += i;
        cout << '.';
        this_thread::sleep_for(300ms);
    }
    return sum;
}

int main() {

    for(int i = 0; i < 3; ++i) {
        promise<int> data;
        auto f = data.get_future();
        future<int> result = async(launch::async, Operation, std::ref(f));
        this_thread::sleep_for(100ms);
        cout << "[main] Setting the data in promise: ";
        data.set_value(10+i  );           // set data
        if(result.valid()) {

            auto sum = result.get();
            cout << "Sum: " << sum << endl;   // simiöar to jon
        }
        cout << '\n';
    }
    return 0;
}
```

```
[task] waiting for count
[main] Setting the data in promise: ..........Sum: 45

[task] waiting for count
[main] Setting the data in promise: ...........Sum: 55

[task] waiting for count
[main] Setting the data in promise: ............Sum: 66
```



High Level Concurrency

So, this is how you can share data between the threads without having to do manual thread synchronization.

Pomise and future can also be used with low-level concurrency.

16.13 Handling Exceptions

Listing 278: Propaget an exception from main thread (promise) to a task (future)

```
#include <iostream>
#include <future>
#include <thread>

using namespace std;
using namespace chrono_literals;

int Operation(std::promise<int>& data) {
    auto f = data.get_future();                     // get future
    try {
        throw runtime_error{"Counter failed"};
        cout << "[task] waiting for count" << endl;
        auto count = f.get();                        // get data
        int sum {};
        for (int i = 0; i < count; ++i) {
            sum += i;
            cout << '.' ;
            this_thread::sleep_for(300ms);
        }
        return sum;
    }
    catch(exception &ex) {
        cout << "[task] Exception: " << ex.what() << endl;
        throw;
    }
}

int main() {

    promise<int> data;
    future<int> result = async(launch::async, Operation, std::ref(data));
    this_thread::sleep_for(100ms);
    cout << "[main] Setting the data in promise: " << endl;;
    try {
        throw runtime_error{"Data not available"};
        data.set_value(10);           // set data
        if(result.valid()) {
            auto sum = result.get();
            cout << "Sum: " << sum << endl;   // simiöar to jon
        }
        cout << '\n';
    }catch(exception &ex) {
        data.set_exception(make_exception_ptr(ex)); // Exception Pointer
    }
    return 0;
}
```

```
[task] Exception: Counter failed
```

```
[main] Setting the data in promise:
```

Listing 279: Propaget an exception the task (future) to the from main thread (promise)

```
#include <iostream>
#include <future>
#include <thread>

using namespace std;
using namespace chrono_literals;

int Operation(std::promise<int>& data) {
    auto f = data.get_future();                     // get future
    try {
        throw runtime_error{"Counter failed"};
        cout << "[task] waiting for count" << endl;
        auto count = f.get();                        // get data
        int sum {};
        for (int i = 0; i < count; ++i) {
            sum += i;
            cout << '.' ;
            this_thread::sleep_for(300ms);
        }
        return sum;
    }
    catch(exception &ex) {
        // no exception pointer needed to propaget from task to main
        cout << "[task] Exception: " << ex.what() << endl;
        return -1;
    }
}

int main() {

    promise<int> data;
    future<int> result = async(launch::async, Operation, std::ref(data));
    this_thread::sleep_for(100ms);
    cout << "[main] Setting the data in promise: " << endl;;
    try {
        throw runtime_error{"Data not available"};
        data.set_value(10);           // set data
        if(result.valid()) {
            auto sum = result.get();
            cout << "Sum: " << sum << endl;   // simiöar to jon
        }
        cout << '\n';
    }catch(exception &ex) {
        data.set_exception(make_exception_ptr(ex)); // Exception Pointer
        cout << "[main] Exception: " << ex.what();
    }
    return 0;
}
```

```
[task] Exception: Counter failed
[main] Setting the data in promise:
[main] Exception: Data not available
```



Promise

Proimse is useful for sharing data between teh threads without having to manually perform, thread synchronisation.

17 C++17 Core Language Features

17.1 Deprecated & Removed Features

17.1.1 Removed Features

- registe keyword
- Exception specification
- operator++(boo)
- Trigraphs
- auto_ptr
- Library Functions
  - std::random_shuffle
  - std::unary_function
  - std::ptr_fun
  - std::bind1st…

17.2 Attributes

- Attributes used to be compiler-specific commands that give additional information to the compiler
- That might be utilised for optimization
- Before C++11, different compiler vendors used keywords for attributes e.g. MSVC uses ___declspec, GCC uses ___attribute, etc
- C++11 introduced some standardized attributes that work with all standard-compilatn compilers
- C++14 & C++17 introduced more attributes that you can use in your code
- You cannot create custom attributes

17.2.1 Common Attributes

noreturn	Indicates the function does not return (C++11)	Function
deprecated	The entity is deprecated but can be used (C++14)	variables, classes, typefs…
nodiscard	The return value should not be discarded (C++17)	function, class enum,…
maybe_unused	Prevents compiler for issuing warnings (++17)	class, variables, enum decl…
fall_through	Indicated deliberate fall through in caase statemen (C++17)	switch-case

Listing 280: Attributes

```
1  #include <cstdlib>
2  #ifdef __has_include
3  #if __has_include(<filesystem>)
4  #define FS 1
5  #include <filesystem>
6  #endif
7  #endif
8
9
10 [[deprecated("Don't use")]]
11 void Foo() {
12
```

```
}

class [[deprecated("Not allowed")]] A{

} ;

namespace [[deprecated("Not allowed")]] Old{
}

#pragma message("DO NOT USE THIS FILE")
class [[nodiscard]] Test{
};
[[nodiscard]]
int &Square(int x) {
    int *p = new int{x} ;
    return *p;
}
Test Create() {
    return Test{} ;
}


[[deprecated("Use the template version instead")]]
[[nodiscard]]
int * CreateIntArray(size_t size) {
    return new int[size] ;
}

template<typename T>
[[nodiscard]] T * CreateArray(size_t size) {
    return new T[size] ;
}

int main() {
    // Foo();
    // A a;
    // Test t;
    // CreateIntArray(5) ;
    // CreateArray<int>(3) ;
    // auto ret = CreateArray<int>(3) ;

    // (void)ret;
}
```

17.3 Feature Test Macros

Link to cppreference.com

Listing 281: Feature Test Macros

```
#include <iostream>
#ifdef __has_include
#       if __has_include(<unistd.h>)
#               include <filesystem>
namespace fs = std::filesystem ;
#       else
#               include <experimental/filesystem>
#               namespace fs = std::experimental::filesystem ;
#       endif
#endif

int main() {
```

```
    fs::path p{"C:"} ;

    std::cout << __cpp_inline_variables << std::endl;
    std::cout << __cpp_capture_star_this << std::endl;
}
```

```
201606
201603
```

Language Features

Each macro expands to an integer literal corresponding to the year and month when the corresponding feature has been included in the working draft. When a feature changes significantly, the macro will be updated accordingly.

17.4 If & switch with Initialization

Listing 282: If & switch with Initialization

```
#include <iostream>
#include <fstream>
#include <filesystem>

using namespace std;

void Alloc() {
    int* p = (int*) malloc(sizeof(int)); // p is unesceserray visible outside of is statement
    if (p != nullptr) {
        *p = 50;
        free(p);
    }
    //if (initialization; condition) {
    if (int* p = (int*) malloc(sizeof(int)); p != nullptr) {   // here not
        *p = 50;
        free(p);
    }
    else {
        //Here p will be nullptr
    }
}
void Write(string data) {
    ofstream out{ "file1.txt" };   // out is unesceserray visible outside of is statement
    if (out) {
        if (!data.empty()) {
            cout << "Write data: " << data << endl;
            out << data << endl;
        }
        else {
            cout << "No data to write " << endl;
            cout << data << endl;
        }
    }
    if (ofstream out{ "file.txt" }; !data.empty()) {  // here not
        cout << "Write data: " << data << endl;
        out << data << endl;
    }
    else {
        cout << "no data to write \n";
```

```
40      }
41  }
42
43  void UsingSwitch(int errorCode) {
44      switch (ofstream out{ "file2.txt" }; errorCode) { // out is only visible within the scope of the
        ↪  switch statement
45      case 0:
46          cout << "Write data: Invalid parameter" << endl;
47          out << "Invalid parameter";
48          break;
49      case 1:
50          cout << "Write data: Could not read from file" << endl;
51          out << "Could not read from file";
52          break;
53      }
54  }
55  int main(){
56      Write("hello World");
57      UsingSwitch(0x1) ;
58  }
```

```
Write data: hello World
Write data: hello World
Write data: Could not read from file
```

If and Switch with initialization

We don't have to expose unnecessary variables to the outer scope

17.5 Inline Variables

17.5.1 Declaration in header files leads to issues

```
1  int global;
```

Listing 283: vars.h

```
1  #include "vars.h"
```

Listing 284: a.cpp

```
1  #include "vars.h"
```

Listing 285: b.cpp

```
g++ -Iinc -std=c++17 -Wall -Wextra -fno-elide-constructors src/main.cpp src/a.cpp src/b.cpp src/main.cp
```

```
1  / usr/lib64/gcc/x86_64-suse-linux/15/../../../../x86_64-suse-linux/bin/ld:
   ↪  /tmp/ccIByloA.o:(.bss+0x0): multiple definition of `global'; /tmp/ccskRSob.o:(.bss+0x0): first
   ↪  defined here
```

```cpp
#include <iostream>
#include "vars.h"

using namespace std;

int main() {

    return 0;
}
```

Listing 286: main.cpp

```
/ usr/lib64/gcc/x86_64-suse-linux/15/../../../../x86_64-suse-linux/bin/ld:
↪  /tmp/ccqvHxyt.o:(.bss+0x0): multiple definition of `global'; /tmp/ccskRSob.o:(.bss+0x0): first
↪  defined here
/ usr/lib64/gcc/x86_64-suse-linux/15/../../../../x86_64-suse-linux/bin/ld:
↪  /tmp/cc2SAHiX.o:(.bss+0x0): multiple definition of `global'; /tmp/ccskRSob.o:(.bss+0x0): first
↪  defined here
/ usr/lib64/gcc/x86_64-suse-linux/15/../../../../x86_64-suse-linux/bin/ld: /tmp/cc2SAHiX.o: in
↪  function `main':
  main.cpp:(.text+0x0): multiple definition of `main'; /tmp/ccskRSob.o:main.cpp:(.text+0x0): first
  ↪  defined here
/  usr/lib64/gcc/x86_64-suse-linux/15/../../../../x86_64-suse-linux/bin/ld:
↪  /tmp/cc7OVPEK.o:(.bss+0x0): multiple definition of `global'; /tmp/ccskRSob.o:(.bss+0x0): first
↪  defined here
/  usr/lib64/gcc/x86_64-suse-linux/15/../../../../x86_64-suse-linux/bin/ld:
↪  /tmp/cc5Bd2nF.o:(.bss+0x0): multiple definition of `global'; /tmp/ccskRSob.o:(.bss+0x0): first
↪  defined here
  collect2: error: ld returned 1 exit status
  make: *** [Makefile:19: app] Error 1

```

The One Definition Rule

The global name such a function or a variable can be defined exactly only once throughout the program.
see p425 TC++PL

17.5.2 Declaration as extern in H-File

Before C++17

Declare the variable/function in a header file with EXTERN and define it just one time in a CPP File

```cpp
// DECLARATION
extern int global;
extern void setGlobal(int);
extern int getGlobal(void);
```

Listing 287: vars.h

```
g++ -Iinc -std=c++17 -Wall -Wextra -fno-elide-constructors src/main.cpp src/a.cpp src/b.cpp -o app
./app
The value of global is: 100
```

```cpp
#include "vars.h"
int global = 42;                        // DEFINITION!!! - Default Value
void setGlobal(int x) { global = x;}    // DEFINITION of setter

```

Listing 288: a.cpp

```cpp
#include "vars.h"
int getGlobal() { return global; }       // DEFINITION of getter
```

Listing 289: b.cpp

17.5.3 C++17 with Inline

```
g++ -Iinc -std=c++17 -Wall -Wextra -fno-elide-constructors src/main.cpp src/a.cpp src/b.cpp -o app
./app
Test::m_Data: 50Global: 5
```

17.6 Nested Namespaces

Listing 295: Nested Namespaces

```cpp
#include <iostream>

namespace A {
    namespace B {
^^Inamespace C {
^^I}
    }
}

namespace A::B::C {

}

int main(){
    A::B::CC::Foo();
    using A::B::CC::Foo;
    using namespace A::B::C;
    foo();

    return 0;
}
```

```cpp
#include <iostream>
#include "vars.h"

using namespace std;

int main() {
    setGlobal(100);
    cout << "The value of global is: " << getGlobal() << endl;
    return 0;
}
```

Listing 290: main.cpp

```
#pragma once
inline int global ;
class Test {
    public:
^^Iinline static int m_Data = 50 ;
^^Iconstexpr static int PATHSIZE=  255 ;
^^Iint x = 5 ;
};
```

Listing 291: vars.h

```
#include "vars.h"
```

Listing 292: a.cpp

```
#include "vars.h"
```

Listing 293: b.cpp

```
#include <iostream>
#include "vars.h"

using namespace std;

int main() {
    global = 5;
    Test t;
    cout << "Test::m_Data: "<< t.m_Data << "Global: "<< global << endl;
}
```

Listing 294: main.cpp

17.7 noexcept

Listing 296: Exception Specification C++11

```
#include <iostream>

// Exception Specification C++98
// Foo thows int and float only.
void Foo() noexcept {

}

void Bar() {

}


int main(){

    void(*p)() noexcept;

    p = Foo;
    p = Bar;

    p(); // p can throw an excepeption -> the exception specification does not hold
    return 0;
}
```

Listing 297: Exception Specification C++17

```
#include <iostream>

// Exception Specification C++98
// Foo thows int and float only.
void Foo() noexcept {

}

void Bar() {

}


int main(){

    void(*p)() noexcept;

    p = Foo;
    p = Bar;

    p(); // C++17 gives an error
    return 0;
}
```

17.8 constexpr Lambda & Capture by *this

Listing 298: constexpr Lambda & Capture by *this

```
#pragma region constexpr lambda
#include <iostream>
#include <sstream>
```

```
4
5   using namespace std;
6
7   ////////////// constexpr lambda
8   template<typename T, int size, typename Callback>
9   void ForEach(T(&arr)[size], Callback operation) {
10      for (int i = 0; i < size - 1; ++i) {
11          operation(arr[i]);
12      }
13  }
14
15  class Product {
16      string name;
17      float price;
18  public:
19      Product(const string &n, float p) :name(n), price(p) {
20
21      }
22      void AssignFinalPrice() {
23          float taxes[]{ 12, 5, 5 };
24          float basePrice{ price };
25          //Capture this
26          ForEach(taxes, [basePrice, this](float tax) {
27              float taxedPrice = basePrice * tax / 100;
28              price += taxedPrice;
29          });
30      }
31      float GetPrice()const {
32          return price;
33      }
34
35      // copies object by value
36      auto GetDescription() {
37          return [*this](const string& header) {
38              ostringstream ost;
39              ost << header << endl;
40              ost << "Name : " << name << endl;
41              ost << "Price : " << price << endl;
42              return ost.str();
43          };
44
45      }
46  };
47
48  void UsingLambda() {
49      Product *p = new Product { "Watch", 500 };
50      p->AssignFinalPrice();
51      // cout << p->GetPrice() << endl;
52      auto desc = p->GetDescription() ;
53      delete p ;                                  // deletes the pointer to member of the clas
54      cout << desc("###############") << endl;
55  }
56  void UsingConstExprLambda() {
57      auto f = [](int x,int y) {                    // the lambda becomes a constexpr function
58          return x + y ;                            // as soon as the criteria for constexpr are
            ↪ fulfilled
59      };
60      constexpr auto sum = f(3,5) ;                  // computed at compile time
61      printf("%d", sum) ;
62  }
63
64
```

```
65
66  int main() {
67
68      UsingLambda();
69      UsingConstExprLambda();
70
71      return 0;
72  }
73
74
75  #pragma endregion
```

```
##############
Name : Watch
Price : 585
```

17.9 Structured Bindings

- Allows initializing of multiple variables with the elements or members of an object
- The object could be object of a class/struct or an array
- For objects of class, the members should be **public**
- The number of variable should match with the number of elements in the object
  auto [variables] = object; <cv qualifiers> auto &[variables] = object;
- Creates an **anonymus entity** which is a copy of the object
- This anonymous entity cannot be accessed directly
- We can only access the members through the **structured binding**
- Useful when you wan to read the data of each member to separate variables (or assign)
- Makes the code more readable by binding the value directly to a name that conveys the real purpose of the data

Listing 299: Structured Bindings

```
1   #include <iostream>
2   #include <map>
3
4   struct Person {
5       std::string m_Name ;
6       int m_Age ;
7       Person() = default ;
8       Person(const std::string &name, int age):m_Name{name}, m_Age{age}{}
9   };
10
11  struct S1 {
12      int arr1[8] ;
13      char ch1[256] ;
14  };
15
16  int main() {
17
18      Person p{"Ayaan", 14} ;
19      /*auto name = p.m_Name ;
20        auto age = p.m_Age ;*/
```

```
21
22      const auto &[name, age] = p ;       // note the reference (by value also possible)
23      // age = 10 ;                       // changes age of p.m_Age as long as not const
24      std::cout << p.m_Age << std::endl;
25
26      std::pair<int,int> p1{3,5} ;
27      auto [key, value] = p ;
28
29      std::map<int, std::string> errorInfo{ {1, "Not available"}, {2, "Port is in use"}} ;
30
31      // a bit tedisous because because using pair syntax
32      for(auto err : errorInfo) {
33  ^^Istd::cout << err.first << ":" << err.second << std::endl;
34      }
35
36      // access with structured binding
37      for(auto [key, value] : errorInfo) {
38  ^^Istd::cout << key << ":" << value << std::endl;
39      }
40
41      int arr[] = {1,2,3} ;
42      auto [a, b, c] = arr ;
43
44      auto arr2 = arr ;               // arr2 is a pointer not an array
45
46      S1 s ;
47      auto [s1, s2] = s ;            // but s1 and s2 are still array type
48
49  }
```

```
14
1:Not available
2:Port is in use
```

17.10 Expression Evaluation Order

- In previous standars, the evaluation order of sub-expressions in function parameters was unpredictable
- IN the following statement, it is unspecified in which order the functions will be invoked
  using namespace std; cout « a() « b() « c() ;
  cout.operator«(a()).operator«(b()).operator«(c());
  In total 6 functions calls: a(), b(), c() and 3 times operator()

Listing 300: Evaluation Order C++98

```
1  #include <iostream>
2
3  class EvaluationOrder {
4  public:
5      EvaluationOrder& First(int) {
6          std::cout << __PRETTY_FUNCTION__ << std::endl;
7          return *this;
8      }
9      EvaluationOrder& Second(int) {
10         std::cout << __PRETTY_FUNCTION__ << std::endl;
11         return *this;
12     }
13 };
```

```
int FirstSubExpression(int) {
    std::cout << __PRETTY_FUNCTION__ << std::endl;
    return 0 ;

}
int SecondSubExpression(int) {
    std::cout << __PRETTY_FUNCTION__ << std::endl;
    return 0 ;

}
int main() {
    EvaluationOrder eo ;
    eo.First(FirstSubExpression(0)).Second(SecondSubExpression(0)) ;
}
```

```
int FirstSubExpression(int)
EvaluationOrder& EvaluationOrder::First(int)
int SecondSubExpression(int)
EvaluationOrder& EvaluationOrder::Second(int)
```

Listing 301: Evaluation Order C++17

```
#include <iostream>

class EvaluationOrder {
public:
    EvaluationOrder& First(int) {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        return *this;
    }
    EvaluationOrder& Second(int) {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        return *this;
    }
};
int FirstSubExpression(int) {
    std::cout << __PRETTY_FUNCTION__ << std::endl;
    return 0 ;

}
int SecondSubExpression(int) {
    std::cout << __PRETTY_FUNCTION__ << std::endl;
    return 0 ;

}
int main() {
    EvaluationOrder eo ;
    eo.First(FirstSubExpression(0)).Second(SecondSubExpression(0)) ;
}
```

```
int FirstSubExpression(int)
EvaluationOrder& EvaluationOrder::First(int)
int SecondSubExpression(int)
EvaluationOrder& EvaluationOrder::Second(int)
```

GNU Compiler - no Interleaving of evaluation

g++ already evaluates sub expressions without interleaving since C++98.

17.11 Mandatory Copy Elision - I

Listing 302: C++14 Optional Copy Elision

```
#include <iostream>

class Number {
public:
    Number(int value) {
        std::cout << "Parameterized ctor\n" ;
    }
    Number(const Number &) {
        std::cout << "Copy ctor\n" ;
    }
    Number(Number &&){
        std::cout << "Move ctor\n" ;
    }
};

void Foo(Number n) {

}

Number Create() {
    // Number n{0} ;
    // return n ;               // copy elision - named return value optimization
                              // move operator applied

    return Number{0} ;    // return value optimization
}
// template<typename T, typename...Args>
// T Create(Args&&...args) {
//     return T{args...} ;
// }
int main() {

    // copy elision is applied in all these cases

    Number n1 = Number{3} ;
    Foo(Number {3} ) ;

    auto n2 = Create() ;

    // auto n3 = Create<Number>(0) ;
}
```

```
Parameterized ctor
Move ctor
Parameterized ctor
Move ctor
Parameterized ctor
Move ctor
Move ctor
```

Listing 303: C++14 Optional Copy Elison: Copy- and Move Constructors Deleted

```cpp
#include <iostream>
#define DELETE_COPY_MOVE
class Number {
public:
    Number(int value) {
        std::cout << "Parameterized ctor\n" ;
    }
#ifdef DELETE_COPY_MOVE
    Number(const Number &) = delete ;
    Number(Number &&) = delete ;
#else
    Number(const Number &) {
        std::cout << "Copy ctor\n" ;
    }
    Number(Number &&){
        std::cout << "Move ctor\n" ;
    }
#endif
};

void Foo(Number n) {

}

Number Create() {
    //Number n{0} ;
    //return n ;

    return Number{0} ;
}

int main() {

    // copy elision is applied in all these cases
    Number n1 = Number{3} ;
    Foo(Number {3} ) ;

    auto n2 = Create() ;

    // auto n3 = Create<Number>(0) ;
}
```

17.12 Mandatory Copy Elision - II

Listing 304: C++17 Mandatory Copy Elision - w/o Copy and Move Constructors

```cpp
#include <iostream>
#define DELETE_COPY_MOVE
class Number {
public:
    Number(int value) {
        std::cout << "Parameterized ctor\n" ;
    }
#ifdef DELETE_COPY_MOVE
    Number(const Number &) = delete ;
    Number(Number &&) = delete ;
#else
    Number(const Number &) {
        std::cout << "Copy ctor\n" ;
    }
    Number(Number &&){
```

```
16          std::cout << "Move ctor\n" ;
17      }
18  #endif
19  };
20
21  void Foo(Number n) {
22
23  }
24
25  Number Create() {
26      //Number n{0} ;
27      //return n ;
28
29      return Number{0} ;
30  }
31  template<typename T, typename...Args>
32  T Create(Args&&...args) {
33      return T{args...} ;
34  }
35  int main() {
36      Number n1 = Number{3} ;
37      Foo(Number{3}) ;
38
39      auto n2 = Create() ;
40
41      auto n3 = Create<Number>(0) ;
42  }
```

```
Parameterized ctor
Parameterized ctor
Parameterized ctor
Parameterized ctor
```



Mandatory Copy Elison

It is mandatory for expressions where you'are initializing an L-value with an R-value and those expressions would be when a function returns a temporary by value, when an object is initialized by R-value and when a function that accepts an L-value is invoked with R-value argument.

18 C++17 Template Features

18.1 Class Template Argument Deduction (CTAD)

18.2 Class Template Argument Deduction (CTAD)

- Template argument deduction is a process by which a compiler deduces the types
- Until C++14, this worked only with functions and not classes
- Consequently, to create instances of classe with multiple template arguments, we had to resort to make_functions e.g. *make_pair*
- With C++17, the decuction works with class templates as well
- This feature is called **Class Template Argument Deductio*n or *CTAD**
- There are two types
  - Compiler generated
  - User defined
- Works with direct or copy initialization of objects or objects on heap

Listing 305: CTAD Class Template Argument Deduction

```
#include <iostream>
#include <mutex>
#include <vector>

template<typename T>
class Data {
public:
    Data(const T &t) {

    }
};

// Deduction Specification
Data(const char *) -> Data<std::string> ;
Data(int)->Data<long> ;

int main() {

    Data d1{5} ;
    Data d2{8.2f} ;

    Data d3 = d1 ;
    auto d4 = new Data{"Hello"} ;

    Data d5{"Hello"} ;

    Data d6{5} ;

    std::pair<int,int> p1{2,5} ;
    auto p2 = std::make_pair(2,5) ;

    std::pair p3{2,5} ;
    std::vector v1{1,2,3,4} ;

    std::mutex m ;
```

```
36      std::lock_guard lck{m} ;
37
38  }
```

18.3 Folding Basics

- It is a process that applies a binary operator to a list of values recursively
- The results are combined recursively, taht builds up the final result
- This process is calles as folding
- In C++11 Variadic templates can perform folds over a template parameter pack
- However, it requires overloads with recursion
  - needs to unpack the parameter for processing

Listing 306: C++11 Folding with Variadic Template

```
1   #include <iostream>
2
3   auto Sum() {
4       return 0 ;
5   }
6   template<typename T, typename ...Args>
7   auto Sum(T a, Args...args) {
8       return a + Sum(args...) ;
9   }
10  /*
11   * Sum(1,2,3,4,5) ;
12   * return 1 + Sum(2,3,4,5) ;
13   * return 2 + Sum(3,4,5) ;
14   * return 3 + Sum(4,5) ;
15   * return 4 + Sum(5) ;
16   * return 5 + Sum() ;
17   * return 5 + 0 ;
18   *
19   *  1 + (2 + (3 + (4 + (5 + 0))))
20   *  1 + (2 + (3 + (4 + 5)))
21   *  1 + (2 + (3 + 9))
22   *  1 + (2 + 12)
23   *  1 + 14
24   *  15
25   *
26   */
27
28
29  int main() {
30      auto result = Sum(1,2,3,4,5) ;
31      std::cout << result << std::endl;
32  }
```

```
15
```

18.4 Fold Expressions - Unary Folds

- C++17 offers a new way to unpack variadic parameters with binary operators
- This is called as **fold expression**

- A fold expression reduces (folds) a parameter pack over a binary operator
- This is a compact syntaxl for applying binary operations to the elements of a parameter pack
- Simplifies implementation of variadic templates that have to apply binary operators on a parameter pack

Fold Expression	
(pack op …)	unary right fold
(… pack op)	unary left fold
(pack op … op init)	binary left fold
(init op … pack op)	binary right fold

Listing 307: Folding Expressions - Unary Folds

```
#include <iostream>

template<typename...Args>
auto Sum2(Args...args) {
    //return (args + ...) ;        //Unary right fold
    return (... + args) ;          //Unary left fold
}

/*
 * Unary Right Fold
 * Sum2(1,2,3,4,5)
 * (1 +(2 + (3 + (4 + 5))))
 *
 * Unary Left Fold
 * ((((1 + 2) + 3) + 4) + 5)
 */

int main() {
    auto result = Sum2(1,2,3,4,5) ;
    std::cout << result << std::endl;

}
```

```
15
```

18.5 Fold Expressions - Binary Folds

Listing 308: Folding Expressions - Binary Folds

```
#include <iostream>

template<typename...Args>
auto Sum3(Args...args) {
    //return (0 + ... + args) ;   //Binary left fold
    return (args + ... + 0) ;     //Binary right fold
}
/*
 * Binary Right fold
 * 1 + (2 + (3 + (4 + (5 + 0))))
 *
 * Binary Left Fold
 * (((((0+1)+2) + 3) + 4) + 5)
 */

/*
  + - * / % ^ & | = < > << >> += -
```

```
  = *= /= %= ^= &= |= <<= >>= == != <= >= && || , .* ->*

  &&          - true
  ||          - false
  ,           - void()
  Others      - ill-formed
*/

template<typename...Args>
bool AnyOf(Args...args) {
    return (... || (args % 2 == 0)) ;
}
template<typename...Args>
bool AllOf(Args...args) {
    return (... && (args % 2 == 0)) ;
}

template<typename...Args, typename Predicate>
bool AnyOf2(Predicate p, Args...args) {
    return (... || p(args)) ;
}

int main() {
    auto result = Sum3() ;
    // auto result = Sum3(1,2,3,4,5) ;
    std::cout << result << std::endl;

    std::cout << std::boolalpha ;
    std::cout << "Any even?" << AnyOf(10,3,5) << std::endl;
    std::cout << "All even?" << AllOf(10,30,52) << std::endl;

    std::cout << "Any even?" << AnyOf2([](int x){return x % 2 == 0;},10,3,5) << std::endl;



}
```

```
0
```

Listing 309: Folding Expressions - Binary Fold Operators

```
#include <iostream>

/*
  + - * / % ^ & | = < > << >> += -
  = *= /= %= ^= &= |= <<= >>= == != <= >= && || , .* ->*

  &&          - true
  ||          - false
  ,           - void()
  Others      - ill-formed
*/

template<typename...Args>
bool AnyOf(Args...args) {
    return (... || (args % 2 == 0)) ;
}
template<typename...Args>
bool AllOf(Args...args) {
    return (... && (args % 2 == 0)) ;
}
```

```
21
22 template<typename...Args, typename Predicate>
23 bool AnyOf2(Predicate p, Args...args) {
24     return (... || p(args)) ;
25 }
26
27 int main() {
28
29     std::cout << std::boolalpha ;
30     std::cout << "Any even?" << AnyOf(10,3,5) << std::endl;
31     std::cout << "All even?" << AllOf(10,30,52) << std::endl;
32     std::cout << "Any even?" << AnyOf2([](int x){return x % 2 == 0;},10,3,5) << std::endl;
33 }
```

```
Any even?true
All even?true
Any even?true
```

18.6 Fold Expressions - Recap

Analyzing C++ with CPPINSIGHTS.IO

18.7 Type Traits Suffixes

- New Type Traits
  - is_reference_v
  - remove_reference_t
  - is_floating_point_v
  - is_default_constructible_v

Listing 310: Type Traits Suffixes

```
1  #include <iostream>
2  #include <type_traits>
3  template<typename T>
4  T Divide(T a, T b) {
5      if(std::is_floating_point_v<T> == false) {
6          std::cout << "Use floating point types only\n" ;
7          return 0 ;
8      }
9      return a / b ;
10 }
11 template<typename T>
12 void Check(T &&) {
13     std::cout << std::boolalpha ;
14     std::cout << "Is reference?" << std::is_reference_v<T> << std::endl ;
15     std::cout << "After removing:" <<
16         std::is_reference_v<typename std::remove_reference_t<T>> << std::endl;
17 }
18 class Number {
19     int m_Number ;
20 public:
21     Number()=default ;
22     Number(int x):m_Number{x} {
23
24     }
25 };
```

```
26 int main() {
27     static_assert(std::is_default_constructible_v<Number>, "Only objects with default constructor can
       ↪  be deserialized") ;
28     Check(5) ;
29     int value{} ;
30     Check(value) ;
31 }
```

```
Is reference?false
After removing:false
Is reference?true
After removing:false
```

18.8 If constexpr - I

Compile-Time if

- This feature allows the condition of an if statement to be evaluated at compile time
- It also discards branches of an *if* statement at **compile-time**
  if constexpr(condition)
- The expression condition must be a constant condition and it should be possible to be evaluate it at compile-time.
- Can be used in template / non-template functions
- Only the block that follows the true condition is evaluated; the else blocks become discarded statements
- Note that the discarded statements must be still valid statements
- It also work with initialization inside if constexpr and runtime if statements
- if constexpr can be used only inside functions, not outside

Listing 311: If constexpr I

```
1  #include <iostream>
2  #include <string>
3  #include <type_traits>
4
5  template<typename T>
6  void Print(const T& value) {
7      if constexpr (std::is_pointer_v<T>) {
8          std::cout << *value << std::endl;
9      }
10     else if constexpr(std::is_array_v<T>) {
11         for (auto v : value) {
12             std::cout << v << ' ';
13         }
14     }
15     else {
16         std::cout << value << std::endl;
17     }
18 }
19
20
21 int main() {
22     int value {5};
23     Print(&value);
24     int arr[] = { 1,2,3,4,5 };
25     Print(arr);
26 }
```

18.9 If constexpr - II

Listing 312: If constexpr II

```
#include <iostream>
#include <string>
#include <type_traits>

template<typename T>
void Print(const T& value) {
    if constexpr (std::is_pointer_v<T>) {
        std::cout << *value << std::endl;
    }
    else if constexpr(std::is_array_v<T>) {
        for (auto v : value) {
            std::cout << v << ' ';
        }
    }
    else {
        std::cout << value << std::endl;
    }
}

template<typename T>
std::string ToString(T value) {
    if constexpr (std::is_arithmetic_v<T>){
        return std::to_string(value) ;
    }else {
        return std::string{value} ;
    }
}
void CheckMode() {
    if constexpr (sizeof(void *) == 4) {
        std::cout << "32-bit\n" ;
    }else if constexpr (sizeof(void *) == 8) {  // Evaluates to true at compile time
        std::cout << "64-bit\n" ;
    }else {
        std::cout << "Unknown mode\n" ;
    }
}
int main() {
    CheckMode() ;

    int value{ 5 };
    auto s = ToString(value) ;
    Print(s) ;
}
```

```
64-bit
5
```

19 C++17 Standard Library Component

19.1 std::optional -I

- A new library **type** that can be used when a function may or may not return a value
- If it does not return a value, a common way is to compare with a predefined value such a 0, nullptr, true/false, etc
- This leads to different kinds of checks for different types
- *std::optional<T>* can represent a type that may or may not contain a value
  - often called a **nullable type**

Properties

- No value is represented by *std::nullopt*
- *std::optional* is a value type - so, it can be copied through deep copy
- Does not need to allocate any memory on heap
- You cannot store references inside *std::optional*
- Provides several overloaded operators and functions to access the value inside safely
- May throw *std::bad_optional_access*

Listing 313: std::optional

```
#include <iostream>
#include <optional>
#include <cstring>

const char * GetErrorString(int errorNo) {
    switch (errorNo) {
    case 0:
        return "Invalid input" ;
    case 1:
        return "Connection not established yet\n" ;
    default:
        return "" ;
    }
}

int main() {

// C++11: return no value
    auto message = GetErrorString(1) ;
    if(std::strlen(message) != 0) {
        std::cout << message << std::endl;
    }
    else {
        std::cout << "Unknown error number\n" ;
    }

// C++17: Handling std::optional type
    // std::optional<int> value = 5;         // Copy initialization
    std::optional value {5};                 // Direct initialization
    // auto value = std::make_optional(5);   // Initialization from a factory function

    if(value.has_value()) {
```

```
33          std::cout << value.value() << std::endl ;  // safe way of reading the value of optional
34      }
35      if(value) {
36          std::cout << *value << std::endl;
37      }
38
39      value = 1 ;
40      value.reset() ;
41  }
```

```
Connection not established yet

5
5
```

19.2 std::optional - II

Listing 314: std::optional - II

```
1   #include <iostream>
2   #include <optional>
3   #include <cstring>
4
5   std::optional<const char *> GetErrorStringOp(int errorNo) {
6       switch (errorNo) {
7       case 0:
8           return "Invalid input" ;
9       case 1:
10          return "Connection not established yet" ;
11      default:
12          return std::nullopt ;
13      }
14  }
15
16  int main() {
17
18      try{
19          auto errorMessage = GetErrorStringOp(4) ;
20          std::cout << errorMessage.value_or("Default") << std::endl ;     // option 1: C++17, use
            ↪ value_or
21
22          errorMessage = GetErrorStringOp(1) ;                             // option 2: C++17, throws
            ↪ exception
23          std::cout << errorMessage.value() << std::endl ;
24
25          errorMessage = GetErrorStringOp(4) ;
26          if(errorMessage.has_value()) {                                   // option 3, to check for
            ↪ value
27              std::cout << errorMessage.value() << std::endl ;
28          }else {
29              std::cout << "Unknown error number\n" ;
30          }
31
32          errorMessage = GetErrorStringOp(4) ;
33          if(errorMessage) {                                               // option 4, to check for
            ↪ value
34              //Undefined behavior when no value exists in optional
35              std::cout << *errorMessage << std::endl ;
36          }else {
37              std::cout << "Unknown error number\n" ;
```

```
        }

    }catch(std::exception &ex) {
        std::cout << "Exception:"<< ex.what() << std::endl ;
    }

    return 0 ;
}
```

```
Default
Connection not established yet
Unknown error number
Unknown error number
```

19.3 std::optional - III

Listing 315: std::optional as class members

```
#include <iostream>
#include <string>
#include <optional>


class ContactName {
    std::string first ;
    std::optional<std::string> middle ;
    std::string last ;

public:

    ContactName(const std::string& f, const std::optional<std::string>& m, const std::string& l)
        : first{f},
          middle{m},
          last{l} {
    }

    void Print() {
        std::cout << first << ' ' ;
        std::cout << middle.value_or("")<< ' ' ;
        std::cout << last << ' ' ;
        std::cout << '\n' ;
    }
};
int main() {

    ContactName n1{"Ernst", "Christian", "Bollinger"} ;
    ContactName n2{"Christian", std::nullopt, "Bollinger"} ;  // nullopt argument avoids the creation
    ↪  of optiona with empty string in constructor

    n1.Print() ;
    n2.Print() ;

    return 0 ;
}
```

```
Ernst Christian Bollinger
Christian  Bollinger
```

19.4 std::variant - I

- *std::variant* is a **type-safe** replacement for **union** type
- Just like union, it uses the storage of the largest member
- The first memeber is always **default initialized** if variant is default constructed
- Alternatively, you can initialize any member of the variant during construction
- Members are destroyed properly (in contrast to union)
- Throws *bad_variant_access* on invalid acces

Listing 316: std::variant I - Usage

```
#include <iostream>
#include <variant>
#include <string>

using namespace std;

int main() {
    using namespace std::string_literals ;

    try {
        std::variant<std::string, int> v{ 7 };
        auto val = std::get<int>(v);
        // auto val = std::get<string>(v);         // throws an exception
        cout << val << endl;

        auto activeIndex = v.index();
        cout << activeIndex << endl;
        cout << get<1>(v) << endl;
        // cout << get<activeIndex>(v) << endl;   // activeIndex is not a compile time constant

        auto p = get_if<string>(&v);                // get a pointer to the active index/element
        if(p == nullptr) {
            cout << "Not Active" << endl;
        }
        else {
            cout << *p << endl;
        }
    }
    catch (std::exception& ex) {
        std::cout << "Exception:" << ex.what() << std::endl;
    }
}
```

```
7
1
7
Not Active
```

19.5 std::variant - II

Listing 317: std::variant II - with Number Class

```
#include <iostream>
#include <variant>
#include <string>
```

```
4
5   using namespace std;
6
7   struct Number {
8       int m_Num{} ;
9       Number(int num): m_Num{num} {
10          std::cout << __PRETTY_FUNCTION__ << std::endl;
11      }
12
13      Number() {
14          std::cout << __PRETTY_FUNCTION__ << std::endl;
15      }
16      ~Number() {
17          std::cout << __PRETTY_FUNCTION__ << std::endl;
18      }
19
20      Number(const Number& other) {
21          m_Num = other.m_Num ;
22
23          std::cout << __PRETTY_FUNCTION__ << std::endl;
24      }
25
26      Number(Number&& other) noexcept {
27          m_Num = other.m_Num ;
28          other.m_Num = 0 ;
29          std::cout << __PRETTY_FUNCTION__ << std::endl;
30      }
31
32      Number& operator=(const Number& other) {
33          std::cout << __PRETTY_FUNCTION__ << std::endl;
34          if (this == &other){
35              return *this;
36          }
37          m_Num = other.m_Num ;
38
39          return *this;
40      }
41
42      Number& operator=(Number&& other) noexcept {
43          std::cout << __PRETTY_FUNCTION__ << std::endl;
44          if (this == &other)
45              return *this;
46          m_Num = other.m_Num ;
47          other.m_Num = 0 ;
48
49          return *this;
50      }
51      friend std::ostream& operator <<(std::ostream &out, const Number &n) {
52          return out << n.m_Num ;
53      }
54  };
55
56  int main() {
57      using namespace std::string_literals ;
58      try {
59          std::variant<std::string, int, Number> v{int{7}};
60
61          v = Number{1} ;
62          v.emplace<Number>(100) ;
63          std::get<Number>(v) = 100 ;
64          auto val = std::get<Number>(v);
65  ^^Ival = std::get<2>(v);
```

```
66
67  ^^Iauto activeIndex = v.index();
68  ^^I(void)activeIndex;
69
70  ^^I// auto p = std::get_if<std::string>(&v);
71  ^^Iauto p = std::get_if<Number>(&v);
72  ^^Iif (p == nullptr) {
73  ^^I    std::cout << "Not active\n";
74  ^^I}
75  ^^Ielse {
76  ^^I    std::cout << *p << std::endl;
77  ^^I}
78  ^^I
79      }
80      catch (std::exception& ex) {
81          std::cout << "Exception:" << ex.what() << std::endl;
82      }
83  }
```

```
Number::Number(int)
Number::Number(Number&&)
Number::~Number()
Number::~Number()
Number::Number(int)
Number::Number(int)
Number& Number::operator=(Number&&)
Number::~Number()
Number::Number(const Number&)
Number& Number::operator=(const Number&)
100
Number::~Number()
Number::~Number()
```

19.6 std::variant - III std::visit

Listing 318: std::variant II std::visit Visito

```
1  #include <iostream>
2  #include <variant>
3  #include <string>
4  struct Number {
5      int m_Num{} ;
6      Number(int num): m_Num{num} {
7          std::cout << __PRETTY_FUNCTION__ << std::endl;
8      }
9
10     Number() {
11         std::cout << __PRETTY_FUNCTION__ << std::endl;
12     }
13     ~Number() {
14         std::cout << __PRETTY_FUNCTION__ << std::endl;
15     }
16
17     Number(const Number& other) {
18         m_Num = other.m_Num ;
19
20         std::cout << __PRETTY_FUNCTION__ << std::endl;
21     }
22
23     Number(Number&& other) noexcept {
```

```
        m_Num = other.m_Num ;
        other.m_Num = 0 ;
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number& operator=(const Number& other) {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        if (this == &other){
            return *this;
        }
        m_Num = other.m_Num ;

        return *this;
    }

    Number& operator=(Number&& other) noexcept {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        if (this == &other)
            return *this;
        m_Num = other.m_Num ;
        other.m_Num = 0 ;

        return *this;
    }
    friend std::ostream& operator <<(std::ostream &out, const Number &n) {
        return out << n.m_Num ;
    }
};

struct Visitor {
    void operator()(const std::string &s)const {
        std::cout << "string:" <<  s << std::endl;

    }
    void operator()(int x)const {
        std::cout << "int:" << x << std::endl ;
    }
    void operator()(const Number &n)const {
        std::cout << "Number:" << n << std::endl;

    }
};

int main() {
    using namespace std::string_literals ;
    try {
        std::variant<std::string, int, Number> v{7 }; // active member: integer
^^Istd::visit(Visitor{}, v) ;

^^Iv = "C++" ;
^^Istd::visit(Visitor{}, v) ;  // active member: string

^^Iv.emplace<Number>(100) ;
^^Istd::visit(Visitor{}, v) ; // active member: Number

    }
    catch (std::exception& ex) {
        std::cout << "Exception:" << ex.what() << std::endl;
    }
}
```

```
int:7
```

```
string:C++
Number::Number(int)
Number:100
Number::~Number()
```

Listing 319: std::variant II std::visit Modifier

```
#include <iostream>
#include <variant>
#include <string>
struct Number {
    int m_Num{} ;
    Number(int num): m_Num{num} {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number() {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }
    ~Number() {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number(const Number& other) {
        m_Num = other.m_Num ;

        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number(Number&& other) noexcept {
        m_Num = other.m_Num ;
        other.m_Num = 0 ;
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number& operator=(const Number& other) {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        if (this == &other){
            return *this;
        }
        m_Num = other.m_Num ;

        return *this;
    }

    Number& operator=(Number&& other) noexcept {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        if (this == &other)
            return *this;
        m_Num = other.m_Num ;
        other.m_Num = 0 ;

        return *this;
    }
    friend std::ostream& operator <<(std::ostream &out, const Number &n) {
        return out << n.m_Num ;
    }
};

struct Visitor {
    void operator()(const std::string &s)const {
```

```
        std::cout << "string:" <<  s << std::endl;

    }
    void operator()(int x)const {
        std::cout << "int:" << x << std::endl ;
    }
    void operator()(const Number &n)const {
        std::cout << "Number:" << n << std::endl;

    }
};
struct Modifier {
    void operator()( std::string &s)const {
        s += " modified string" ;
    }
    void operator()(int& x)const {
        x += 1000 ;
    }
    void operator()( Number &n)const {
        n = 999 ;
    }
};
int main() {
    using namespace std::string_literals ;
    try {
        std::variant<std::string, int, Number> v{7 }; // active member: integer
^^Istd::visit(Modifier{}, v) ;
^^Istd::visit(Visitor{}, v) ;

^^Iv = "C++" ;
^^Istd::visit(Modifier{}, v) ; // active member: string
^^Istd::visit(Visitor{}, v) ;

^^Iv.emplace<Number>(100) ;
^^Istd::visit(Modifier{}, v) ; // active member: Number
^^Istd::visit(Visitor{}, v) ;
^^I
    }
    catch (std::exception& ex) {
        std::cout << "Exception:" << ex.what() << std::endl;
    }
}
```

```
int:1007
string:C++ modified string
Number::Number(int)
Number::Number(int)
Number& Number::operator=(Number&&)
Number::~Number()
Number:999
Number::~Number()
```

Listing 320: std::variant II std::visit Lambda Expression

```
#include <iostream>
#include <variant>
#include <string>
struct Number {
    int m_Num{} ;
    Number(int num): m_Num{num} {
```

```
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number() {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }
    ~Number() {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number(const Number& other) {
        m_Num = other.m_Num ;

        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number(Number&& other) noexcept {
        m_Num = other.m_Num ;
        other.m_Num = 0 ;
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number& operator=(const Number& other) {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        if (this == &other){
            return *this;
        }
        m_Num = other.m_Num ;

        return *this;
    }

    Number& operator=(Number&& other) noexcept {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        if (this == &other)
            return *this;
        m_Num = other.m_Num ;
        other.m_Num = 0 ;

        return *this;
    }
    friend std::ostream& operator <<(std::ostream &out, const Number &n) {
        return out << n.m_Num ;
    }
};

int main() {
    using namespace std::string_literals ;
    try {

^^Istd::variant<std::string, int, Number> v{7 };            // active member: integer
^^I
^^Iauto visitor = [](auto &x) {
            using T = std::decay_t<decltype(x)> ;                // decay the reference to baskic
            ↪  types
            if constexpr (std::is_same_v<T, int>){               // comparing baseic types
                std::cout << "int : " << x << std::endl;
            }else if constexpr (std::is_same_v<T, std::string>){ // comparing baseic types
                std::cout << "string:" << x << std::endl;
            }else if constexpr(std::is_same_v<T, Number>){       // comparing baseic types
                std::cout << "Number:" << x << std::endl ;
            }
```

```
68          } ;
69
70  ^^Iv = "using lambda" ;                                    // active member: string
71          std::visit(visitor,v) ;
72          v = 555 ;                                                // active member: integer
73          std::visit(visitor,v) ;
74          v = Number{3} ;                                          // active member: Number
75          std::visit(visitor,v) ;
76
77      }
78      catch (std::exception& ex) {
79          std::cout << "Exception:" << ex.what() << std::endl;
80      }
81  }
```

```
string:using lambda
int : 555
Number::Number(int)
Number::Number(Number&&)
Number::~Number()
Number:3
Number::~Number()
```

19.7 std::any

Type Safety

- C++ is a strongly typed language
- Object are declared with specific type and that cannot be changed later
- In some cases, we may require an object that should hold values of different types
- This is difficult to achieve in C++, except through *void **
- However, such pointers have a certain disadvantage
  - Not typesafe
  - No way to know the type
  - Cannot access the value in a type-safe way
  - Need to manage the object lifetime

std::any

- C++17 introduced std::any
- A **wrapper** that can hold value of any arbitrary type in a **type-safe** way
- Replacement for *void **
- Contains both the value & its type
- The value is accessed throuhg *any_cast<>*
- May allocate memory on the heap
- Throws exception of type *bad_any_cast* on wrong type access

Listing 321: std::variant II std::visit Visito

```cpp
#include <any>
#include <iostream>
struct Number {
    int m_Num{} ;
    Number(int num): m_Num{num} {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number() {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }
    ~Number() {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number(const Number& other) {
        m_Num = other.m_Num ;

        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number(Number&& other) noexcept {
        m_Num = other.m_Num ;
        other.m_Num = 0 ;
        std::cout << __PRETTY_FUNCTION__ << std::endl;
    }

    Number& operator=(const Number& other) {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        if (this == &other){
            return *this;
        }
        m_Num = other.m_Num ;

        return *this;
    }

    Number& operator=(Number&& other) noexcept {
        std::cout << __PRETTY_FUNCTION__ << std::endl;
        if (this == &other)
            return *this;
        m_Num = other.m_Num ;
        other.m_Num = 0 ;

        return *this;
    }
    friend std::ostream& operator <<(std::ostream &out, const Number &n) {
        return out << n.m_Num ;
    }
};
void Basics() {
    using namespace std::string_literals ;
    std::any v1 = 5;
    // v1 = "Hello"s ;
    if(v1.has_value()) {
        if(v1.type() == typeid(int)) {
            std::cout << std::any_cast<int>(v1) << std::endl;
        }
    }
    v1.reset() ;
    try {
        std::cout << std::any_cast<std::string>(v1) << std::endl;
```

```
63      }catch(std::exception &ex) {
64          std::cout << "Exception:" << ex.what() << std::endl;
65      }
66  }
67  int main() {
68      using namespace std::string_literals ;
69      // Basics();
70
71      // std::any number{Number{5}} ;
72      auto n1 = std::make_any<Number>(5) ;                // instantiation of n1
73      n1.reset() ;
74      n1 = 7 ;
75      std::cout << std::any_cast<int>(n1) << std::endl ;
76
77      auto &n2 = std::any_cast<int&>(n1) ;
78      n2 = 100 ;
79      std::cout << std::any_cast<int>(n1) << std::endl ;   // reference to n1
80
81      auto p = std::any_cast<int>(&n1) ;
82      *p = 200 ;
83      std::cout << *std::any_cast<int*>(p) << std::endl;   // points to n1
84
85  }
```

```
Number::Number(int)
Number::~Number()
7
100
200
```

19.8 std::string_view - I

std::string_view

- Allows us to deal with charachter sequenceis without allocating memory
- Can be considered as a kind of reference to a charachter sequence
- Can be used where ever a **non-modifialbe string** is required
- It simply stores a pointer to the character array along with its length
- It is fast an cheap to copy
- However, it should be used carefully as it can lead to bugs

Properties:

- Underlying sequence is **read-only**
- It can be accessed through *data()* method
  - may return **nullptr**
  - a charachter sequence with or without null terminator
- You can only assign a new value, swap values and shrink it to a subset of character sequence
- Character sequence is **not guaranteed** to be **null-terminated**
  - consequently, **may** not work with C string functions
- No allocator support (because of read only)



:exports results

The code cannot be printed here because it contains literal which is missunterstood by the latex compiler.

19.9 std::string_view - II

When to use std::string_view

- Don't pass in functions that accept a C-style string because it might not have a null terminator
- Do not return sting view to string from a function
- Do not assign temporary strings to string views
- Avoid as class members

Listing 322: std::string-view II

```
#include <iostream>
#include <string_view>
#pragma region Example 1
void PrettyPrint(std::string_view message, char ch) {   // no construction of std:string object
    for(int x = 0 ; x < message.length() ; ++x) {       // it works because string is not modified
        std::cout << ch ;
    }
    std::cout <<'\n' ;
    std::cout << message << '\n' ;
}
#pragma endregion

#pragma region Example 2
enum class Title{MR, MRS, MS};
std::string & CombineName( std::string &name, std::string_view last, Title s) {
    switch (s) {
    case Title::MR:
        name.insert(0, "Mr.") ;
        break ;
    case Title::MRS:
        name.insert(0, "Mrs.") ;
        break ;
    case Title::MS:
        name.insert(0, "Ms.") ;
        break ;
    }
    return name += last ;
}
#pragma endregion

#pragma region Example 3
class Person {
    std::string m_Name;
public:
    Person(std::string name):m_Name{std::move(name)} {

    }
    void Print()const {
        std::cout << m_Name << std::endl;
    }
};
```

```
42
43 Person CreatePerson(std::string_view name) {
44     std::string n{name} ;
45     Person p{n} ;
46     return p ;
47 }
48 void WithClass() {
49     using namespace std::string_literals ;
50     //std::string n{} ;
51     Person p{"Umar"} ;
52
53     p.Print() ;
54 }
55 #pragma endregion
56
57 void BasicUsage() ;
58
59 int main() {
60      // PrettyPrint("Hello world", '#') ;
61     WithClass() ;
62 }
63
```

```
Umar
```

19.10 Filesystem - path

Filesystem

- C++17 added support for working with filesystem components
- Adopted from Boos.Filesystem, it was adjusted to new language standards and made consistent with other parts of the library
- Provides facilities for performing following operations
  - manipulaiton of filesystem path
  - create, move, rename, delete directories
  - list contents of given directory
  - get information about path, file permissions, etc.
- To create, read & write files, you'll still use the stream library class
- The facilities are provided in *<filesystem>* header under *std::filesystem* namespace
  - **path** - allows manipulation of path that represent existing files or directories
  - **directory_entry** - represents a path with additional information such as file **size**, file **times**, etc
  - **directory_iterator** - an iterator that iterates over the contents of a directory
  - functions for working with directories
  - and much more
- Many functions will throw *std::filesystem_error* exception on failure

Listing 323: Filesystem - path

```
1 #include <iostream>
2 #include <filesystem>
3
4 namespace fs = std::filesystem ;
```

```
5
6   int main() {
7       fs::path selectedPath{R"(/home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.org)"} ;
8       std::cout << selectedPath << std::endl;
9       std::cout << selectedPath.string() << std::endl;
10
11      selectedPath.remove_filename() ;
12      selectedPath /= "newfile" ;
13
14      if (selectedPath.has_root_name()) {
15          std::cout << "root name\t = " << selectedPath.root_name().string() << std::endl;
16      }
17      if (selectedPath.has_root_path()) {
18          std::cout << "root path\t = " << selectedPath.root_path().string() << std::endl;
19      }
20      if (selectedPath.has_root_directory()) {
21          std::cout << "root directory\t = " << selectedPath.root_directory().string() << std::endl;
22      }
23      if (selectedPath.has_parent_path()) {
24          std::cout << "parent path\t = " << selectedPath.parent_path().string() << std::endl;
25      }
26      if (selectedPath.has_relative_path()) {
27          std::cout << "relative path\t = " << selectedPath.relative_path().string() << std::endl;
28      }
29      if (selectedPath.has_filename()) {
30          std::cout << "filename\t = " << selectedPath.filename().string() << std::endl;
31      }
32      if (selectedPath.has_stem()) {
33          std::cout << "stem part \t = " << selectedPath.stem().string() << std::endl;
34      }
35      if (selectedPath.has_extension()) {
36          std::cout << "extension\t = " << selectedPath.extension().string() << std::endl;
37      }
38
39      return 0;
40  }
```

```
"/home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.org"
/home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.org
root path  = /
root directory  = /
parent path  = /home/christian/Daten/01-tutorial/Modern-CPP
relative path  = home/christian/Daten/01-tutorial/Modern-CPP/newfile
filename  = newfile
stem part  = newfile
```

19.11 Filesystem - directory_entry

Listing 324: Filesystem - directory-entry

```
1   #include <iostream>
2   #include <filesystem>
3   #include <string_view>
4   #include <vector>
5   #include <algorithm>
6
7   namespace fs = std::filesystem ;
8
9   int main() {
10      std::string_view file = "/home/christian/Daten/01-tutorial/Modern-CPP/" ;
```

```
    fs::path currentPath{file} ;
    fs::directory_iterator begin{currentPath} ;
    fs::directory_iterator end{} ;

    // While loop with iterators
    // while(begin != end) {
    // ^^Iauto de = *begin++ ;
    // ^^Istd::cout << de.path().filename() << std::endl;
    // }
    // std::cout << std::endl;

    std::vector<fs::directory_entry> dir_entries {};
    for(const auto & dir_entry : fs::directory_iterator{currentPath}) {
        dir_entries.push_back(dir_entry) ;
    }
    std::partition(dir_entries.begin(), dir_entries.end(), [](const fs::directory_entry & de) {
        return de.is_directory() ;
    }) ;
    for(const auto &dir_entry : dir_entries) {
        switch(const auto &p = dir_entry.path() ;/*dir_entry.status().type()*/
        ↪  fs::status(p).type()) {
        case fs::file_type::directory:
            std::cout << "[DIR]\t" << p.string() << std::endl;
            break ;
        case fs::file_type::regular:
            std::cout << "\t\t" << p.string() << '\t' << dir_entry.file_size() << std::endl;
            break ;
        }
    }
}
```

```
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/chapters
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/_minted
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/projects
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/fig
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/ .
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/resources
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/uic-gw-evolution
  /home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.org 4569
  /home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.tex 368328
  /home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.org~ 4509
  /home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.pdf 993395
```

19.12 Filesystem - Directory Functions

Listing 325: Filesystem - Directory Functions Create/Remove

```
#include <iostream>
#include <filesystem>
#include <string_view>
#include <vector>
#include <algorithm>

namespace fs = std::filesystem ;

void TraversingDirectory(std::string_view file) {
    fs::path currentPath{file} ;
    fs::directory_iterator begin{currentPath} ;
    fs::directory_iterator end{} ;
```

```
13
14      std::vector<fs::directory_entry> dir_entries {};
15      for(const auto & dir_entry : fs::directory_iterator{currentPath}) {
16          dir_entries.push_back(dir_entry) ;
17      }
18      std::partition(dir_entries.begin(), dir_entries.end(), [](const fs::directory_entry & de) {
19          return de.is_directory() ;
20      }) ;
21      for(const auto &dir_entry : dir_entries) {
22          switch(const auto &p = dir_entry.path() ;/*dir_entry.status().type()*/
            ↪  fs::status(p).type()) {
23          case fs::file_type::directory:
24              std::cout << "[DIR]\t" << p.string() << std::endl;
25              break ;
26          case fs::file_type::regular:
27              std::cout << "\t\t" << p.string() << '\t' << dir_entry.file_size() << std::endl;
28              break ;
29          }
30      }
31  }
32
33  int main() {
34
35      std::string_view file = "/home/christian/Daten/01-tutorial/Modern-CPP/" ;
36      fs::path currentPath{file} ;
37      if(!fs::exists(currentPath)) {
38          std::cout << "Path does not exist = >" << currentPath.string() << std::endl ;
39          return 0;
40      }
41
42      currentPath /= "/home/christian/Daten/01-tutorial/Modern-CPP/test-dir/" ;
43      if(!fs::create_directory(currentPath)) {
44  ^^Istd::cout << "Could not create a directory\n" ;
45      }else {
46  ^^Istd::cout << "Directory created successfully\n" ;
47      }
48
49      TraversingDirectory(file);
50      if(!fs::remove(currentPath)) {
51  ^^Istd::cout << "Could not delete the directory\n" ;
52      }else {
53  ^^Istd::cout << "Directory removed successfully\n" ;
54      }
55
56      return 0;
57  }
```

```
Directory created successfully
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/test-dir
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/chapters
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/_minted
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/projects
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/fig
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/ .
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/resources
[DIR] /home/christian/Daten/01-tutorial/Modern-CPP/uic-gw-evolution
  /home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.org 4569
  /home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.tex 380007
  /home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.org~ 4509
```

```
  /home/christian/Daten/01-tutorial/Modern-CPP/modern-cpp.pdf 1016469
Directory removed successfully
```

Listing 326: Filesystem - Directory Functions Exception Handling Try/Catch

```
#include <iostream>
#include <filesystem>
#include <string_view>
#include <vector>
#include <algorithm>

namespace fs = std::filesystem ;

int main() {

    std::string_view file = "/home/christian/Daten/01-tutorial/Modern-CPP/" ;
    fs::path currentPath{file} ;
    if(!fs::exists(currentPath)) {
        std::cout << "Path does not exist = >" << currentPath.string() << std::endl ;
        return 0;
    }

    currentPath /= "/home/christian/Daten/01-tutorial/Modern-CPP/test-dir/" ;
    // currentPath /= "/home/christian/Daten/01-tutorial/Modern-CPP/projects/" ;
    try {
        std::cout << fs::current_path() << std::endl;
        std::cout << "Changing path\n";
^^Ifs::current_path(currentPath);                    // throws exception
        std::cout << fs::current_path() << std::endl;
    }
    catch (std::exception& ex) {
        std::cout << "Exception:" << ex.what() << std::endl ;
    }
    return 0;
}
```

```
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters"
Changing path
"/home/christian/Daten/01-tutorial/Modern-CPP/projects"
```

Listing 327: Filesystem - Directory Functions Exception Error Code Class

```
#include <iostream>
#include <filesystem>
#include <string_view>
#include <vector>
#include <algorithm>

namespace fs = std::filesystem ;

int main() {

    std::string_view file = "/home/christian/Daten/01-tutorial/Modern-CPP/" ;
    fs::path currentPath{file} ;
    if(!fs::exists(currentPath)) {
        std::cout << "Path does not exist = >" << currentPath.string() << std::endl ;
        return 0;
    }

```

```
18      currentPath /= "/home/christian/Daten/01-tutorial/Modern-CPP/test-dir/" ;
19      std::cout << fs::current_path() << std::endl;
20      std::cout << "Changing path\n";
21
22      std::error_code ec{} ;                          // uses error code class
23      fs::current_path(currentPath, ec);              // call curent path with error code object
24      if(ec) {
25  ^^Istd::cout << "Error:" << ec.message() << std::endl;
26  ^^Ireturn 0;
27      } else {
28  ^^Istd::cout << fs::current_path() << std::endl;
29      }
30      return 0;
31  }
```

```
"/home/christian/Daten/01-tutorial/Modern-CPP/chapters"
Changing path
Error:No such file or directory
```

19.13 Filesystem - Permissions

Listing 328: Filesystem - Permissions

```
1   #include <iostream>
2   #include <filesystem>
3   #include <string_view>
4
5   namespace fs = std::filesystem ;
6
7   // demo_perms() from cpprefence
8   void demo_perms(fs::perms p)
9   {
10      std::cout << ((p & fs::perms::owner_read) != fs::perms::none ? "r" : "-")
11                << ((p & fs::perms::owner_write) != fs::perms::none ? "w" : "-")
12                << ((p & fs::perms::owner_exec) != fs::perms::none ? "x" : "-")
13                << ((p & fs::perms::group_read) != fs::perms::none ? "r" : "-")
14                << ((p & fs::perms::group_write) != fs::perms::none ? "w" : "-")
15                << ((p & fs::perms::group_exec) != fs::perms::none ? "x" : "-")
16                << ((p & fs::perms::others_read) != fs::perms::none ? "r" : "-")
17                << ((p & fs::perms::others_write) != fs::perms::none ? "w" : "-")
18                << ((p & fs::perms::others_exec) != fs::perms::none ? "x" : "-")
19                << '\n';
20  }
21
22  int main() {
23      std::string_view file =
        ↪  "/home/christian/Daten/01-tutorial/Modern-CPP/projects/section-02/func2/math/inc/math.hpp" ;
24      fs::path file_to_modify{file} ;
25      if(!fs::exists(file_to_modify)) {
26          std::cout << "Path does not exist = >" << file_to_modify.string() << std::endl ;
27           return 0;
28      }
29      auto perm = fs::status(file_to_modify).permissions() ;
30      demo_perms(perm) ;
31      std::cout << "Changing permissions\n" ;
32      fs::permissions(file_to_modify,fs::perms::owner_exec, fs::perm_options::add) ;
33      perm = fs::status(file_to_modify).permissions() ;
34      demo_perms(perm) ;
35
36      return 0;
37  }
```

```
rwxr--r--
Changing permissions
rwxr--r--
```

19.14 Parallel Algorithm - I

- May STL algorithms by default execute serially
- C++17 provides overloads to these algorithms that can now execute parallelly
- Algorithms that can be parallelized accept a new template parameter called **execution policy**
  - used to disambiguete the calls to overloads of the algorithm
- This can be used to inform the algorithm if it should execute **serially** or **parallelly**
- The internal implementation details are hidden to the user

Syntax

- Parallel versions of STL algorithms have a simple interface
- They are providedas overloaded funcitons with the first parameter signifying the execution policy
- Execution policy defines how the algorithm should execute
  *std::algorithm(execution_policy, / * begin, end, other args * /);*

Execution Policies

- All execution policies existing in *<execution>* header and in *std::execution* namespace
- Each of the policy is an individual type
- *sequend_policy* - **seq**
  - the algorithm's execution will not be parallelized and will perform operations sequentially element by element
  - same as invoking the algorithm from C++14
- *parallel_policy* - **par**
  - indicates the algorithm should execute parallelly
  - might use thread from a thread pool for execution along with the calling thread
- *parallel_unsiquenced_policy* - **par_unseq**
  - indicates the execution may be parllelized, vectorized or migrated across threads

19.14.1 Parallel Algorithm - II

Listing 329: Parallel Algorithm - Sort

```
1  #include <chrono>
2  #include <iostream>
3  #include <random>
4  #include <string_view>
5  #include <vector>
6  #include <execution>
7  #include <iomanip>
8
9  class Timer {
10     std::chrono::steady_clock::time_point m_start ;
11 public:
12     Timer():m_start{std::chrono::steady_clock::now()} {
13
```

```
    }
    void ShowResult(std::string_view message = "") {
        auto end = std::chrono::steady_clock::now() ;
        auto difference = end - m_start ;
        std::cout << message
                  << ':'
^^I^^I  << std::setw(6)
                  << std::chrono::duration_cast<std::chrono::milliseconds>(difference).count()
                  << '\n' ;
    }
};

constexpr unsigned VEC_SIZE{10000000} ;

std::vector<long> CreateVector() {
    std::vector<long> vec ;
    vec.reserve(VEC_SIZE) ;
    std::default_random_engine engine{std::random_device{}()} ;
    std::uniform_int_distribution<long> dist{0, VEC_SIZE} ;
    for(unsigned i = 0 ; i < VEC_SIZE ; ++i) {
        vec.push_back(dist(engine)) ;
    }
    return vec ;
}

int main() {
    auto dataset = CreateVector() ;
    Timer t1 ;
    std::sort(dataset.begin(), dataset.end()) ;
    t1.ShowResult("Accumulate time t1: ") ;

    Timer t2 ;
    std::sort(std::execution::par, dataset.begin(), dataset.end()) ;
    t2.ShowResult("Accumulate time t2: ") ;
}
```

```
Accumulate time t1: :  1797
Accumulate time t2: :  1054
```

Listing 330: Parallel Algorithm - Accumulate / Reduce

```
#include <chrono>
#include <iostream>
#include <random>
#include <string_view>
#include <vector>
#include <execution>
#include <iomanip>

class Timer {
    std::chrono::steady_clock::time_point m_start ;
public:
    Timer():m_start{std::chrono::steady_clock::now()} {

    }
    void ShowResult(std::string_view message = "") {
        auto end = std::chrono::steady_clock::now() ;
        auto difference = end - m_start ;
        std::cout << message
                  << ':'
^^I^^I  << std::setw(6)
```

```
                  << std::chrono::duration_cast<std::chrono::milliseconds>(difference).count()
                  << '\n' ;
    }
};

constexpr unsigned VEC_SIZE{100000000} ;

std::vector<long> CreateVector() {
    std::vector<long> vec ;
    vec.reserve(VEC_SIZE) ;
    std::default_random_engine engine{std::random_device{}()} ;
    std::uniform_int_distribution<long> dist{0, VEC_SIZE} ;
    for(unsigned i = 0 ; i < VEC_SIZE ; ++i) {
        vec.push_back(dist(engine)) ;
    }
    return vec ;
}

int main() {
    auto dataset = CreateVector() ;
    Timer t1 ;
    auto result1 = std::accumulate(dataset.begin(), dataset.end(),0L) ;
    t1.ShowResult("Accumulate time t1: ") ;

    Timer t2 ;
    auto result2 = std::reduce(dataset.begin(), dataset.end(),0L) ;
    t2.ShowResult("Accumulate time t2: ") ;
    (void)result1; (void)result2;
}
```

```
Accumulate time t1: :   165
Reduce time t2: :   168
```

Accumulate Performs left fold on the container

Reduce Tree approach

19.14.2 Parallel Algorithm - III

Listing 331: Parallel Algorithm III

```
#include <chrono>
#include <iostream>
#include <random>
#include <string_view>
#include <vector>
#include <execution>
#include <iomanip>

class Timer {
    std::chrono::steady_clock::time_point m_start ;
public:
    Timer():m_start{std::chrono::steady_clock::now()} {

    }
    void ShowResult(std::string_view message = "") {
        auto end = std::chrono::steady_clock::now() ;
        auto difference = end - m_start ;
        std::cout << message
                  << ':'
^^I^^I  << std::setw(6)
```

```
                  << std::chrono::duration_cast<std::chrono::nanoseconds>(difference).count()
                  << '\n' ;
    }
};

constexpr unsigned VEC_SIZE{100} ;

std::vector<long> CreateVector() {
    std::vector<long> vec ;
    vec.reserve(VEC_SIZE) ;
    std::default_random_engine engine{std::random_device{}()} ;
    std::uniform_int_distribution<long> dist{0, VEC_SIZE} ;
    for(unsigned i = 0 ; i < VEC_SIZE ; ++i) {
        vec.push_back(dist(engine)) ;
    }
    return vec ;
}

int main() {
    auto dataset = CreateVector() ;
    Timer t1 ;
    std::sort(dataset.begin(), dataset.end()) ;
    t1.ShowResult("Accumulate time t1: ") ;

    Timer t2 ;
    std::sort(std::execution::par, dataset.begin(), dataset.end()) ;
    t2.ShowResult("Accumulate time t2: ") ;
}
```

```
Accumulate time t1: :  8056
Accumulate time t2: :  4424
```

Exception Handling

- If an element access function throws an exception which is not handled, all parallel algorithm call *std::teminate*
- This also applies if sequential execution policy is chosen
- If you want to handle exceptions, use the **standard** algorithms instead
- Parallel algorithms themselves may throw *std::bad_alloc* if they fail to acquire memory during execution

Parallel Algorithms Everywhere?

- Backward compatability
- Can use both input & output iterators
- Synchronization is expensive
- Parallelized algorithms teminate on exceptons, sequenced do not
- Do not give good performance for operations on small number of elements due to **overhead**
-

20 Used Compiler Flags

- **C++ :flags "-std=c++11 -Wall -Wextra -fno-elide-constructors"**
  - **-Wall** Enable the standard, generally safe warnings most people should always use.
  - **-Wextra** Enable the more pedantic / deeper warnings that -Wall leaves out.
  - **-fno-elide-constructors** call copy/move constructors instead of optimizing them away

21 Org Mode Simulate keyboard input

A nice pattern for demonstration purpose C in Org-mode is to keep your program unchanged (still using cin) and simulate input with an istringstream. That way, your examples are reproducible and don't depend on how Babel handles stdin.

Listing 332: Template 1: Simulate keyboard input

```
#include <iostream>
#include <sstream>

int main() {
    std::istringstream cin_sim("63");

    int age{};
    cin_sim >> age;

    std::cout << "Your age is: " << age << '\n';
}
```

```
Your age is: 63
```

Listing 333: Template 2: Multiple Inputs

```
#include <iostream>
#include <sstream>

int main() {
    std::istringstream cin_sim(
        "Alice\n"
        "63\n"
        "1.75\n"
    );

    std::string name;
    int age;
    double height;

    cin_sim >> name >> age >> height;

    std::cout << "Name   : " << name   << '\n';
    std::cout << "Age    : " << age    << '\n';
    std::cout << "Height : " << height << '\n';
}
```

```
Name   : Alice
Age    : 63
Height : 1.75
```

Listing 334: Template 3: Reading a vector

```
#include <iostream>
#include <sstream>
#include <vector>

int main() {
    std::istringstream cin_sim(
        "5\n"
        "10 20 30 40 50\n"
```

```
    );

    int n;
    cin_sim >> n;

    std::vector<int> values(n);

    for (int& x : values)
        cin_sim >> x;

    for (int x : values)
        std::cout << x << ' ';

    std::cout << '\n';
}
```

```
10 20 30 40 50
```

For demonstration purpose, it's often useful to write the program exactly as students would:

```
std::istream& in = cin_sim;
```

Listing 335: Template 4: Keep the code using cin

```
#include <iostream>
#include <sstream>
#include <vector>

int main() {
    std::istringstream cin_sim(
        "3\n"
        "11 22 33\n"
    );

    std::istream& in = cin_sim;

    int n;
    in >> n;

    std::vector<int> v(n);

    for (int& x : v)
        in >> x;

    for (int x : v)
        std::cout << x << ' ';

    std::cout << '\n';
}
```

Later, to make it interactive, change only one line:

```
  std::istream& in = std::cin;
```

Everything else stays the same.

Listing 336: Template 5: Simulate a file

```
#include <iostream>
#include <sstream>
#include <string>
```

```
4
5  int main() {
6      // std::ifstream file("data.txt");
7      std::istringstream file_sim(
8          "first line\n"
9          "second line\n"
10         "third line\n"
11         );
12
13     std::string line;
14
15     while (std::getline(file_sim, line))
16         std::cout << "[" << line << "]\n";
17 }
```

```
[first line]
[second line]
[third line]
```

This behaves like

```
1  std::ifstream file("data.txt");
```

10
20
30
40

Listing 337: Org-mode variable driven input

```
1  #include <iostream>
2  #include <vector>
3
4  int main()
5  {
6      for (const auto& row : values)
7      {
8          std::cout << row[0] << ' ';
9      }
10     std::cout << "\n";
11 }
```

```
10 20 30 40
```

inspect the generated source with:

M-x org-babel-expand-src-block

22 Document Setup & Build Improvements

This appendix documents all structural and build improvements applied to this document. Use it as a checklist when converting other Org-mode LaTeX documents to the same standard.

22.1 Build Speed: Latexmk Configuration

The default latexmk runs up to 5 xelatex passes. With many `minted` code blocks each pass is slow. Two changes reduce this significantly.

22.1.1 Global `~/.latexmkrc`

Managed via `Emacs.org` (`org-babel-tangle`). Limits reruns to 3 and ensures `-shell-escape` (required by minted) is always passed:

```
$xelatex  = 'xelatex  -shell-escape -interaction=nonstopmode %O %S';
$lualatex  = 'lualatex  -shell-escape -interaction=nonstopmode %O %S';
$pdflatex  = 'pdflatex  -shell-escape -interaction=nonstopmode %O %S';
$max_repeat = 3;
$clean_ext = 'xdv synctex.gz';
```

22.1.2 Project-local `.latexmkrc`

Placed in the document root directory. Selects xelatex for this project:

```
# Global settings (shell-escape, max_repeat) come from ~/.latexmkrc
$pdf_mode = 5;   # 5 = xelatex
```

22.1.3 Emacs `org-latex-pdf-process`

```
(setq org-latex-pdf-process
  '("latexmk -xelatex -shell-escape -interaction=nonstopmode -halt-on-error %f"))
```

Changes vs. the original command:

- Removed `-pdf` (redundant — `-xelatex` already sets the engine)
- Removed `-f` (force-on-error causes latexmk to loop through broken states)

22.2 Reducing xelatex Passes: hyperref PageLabels

The combination of `lastpage` + `scrlayer-scrpage` + `hyperref` triggers a *"Rerun to get /PageLabels entry"* warning, adding one unnecessary pass.
Add to the document preamble:
Or, to apply globally to all Org exports, add it to `org-latex-hyperref-template` in `Emacs.org`:

```
(setq org-latex-hyperref-template
  "\\hypersetup{
     ...
     pdfpagelabels=false
  }")
```

22.3 Figure Placement: `[H]` Instead of `[htbp]`

LaTeX's default `[htbp]` lets figures float away from their definition point, which disrupts the reading flow in tutorial documents.
The `float` package's `[H]` specifier forces figures exactly where defined.
Add to preamble:
Replace all `#+ATTR_LATEX` directives:
#+begin_example

22.4 Source Code Formatting: Untabify

Tabs in source blocks cause inconsistent indentation in the PDF output. Convert all tabs to 4 spaces inside `#+begin_src` / `#+end_src` blocks.
In Emacs, on each chapter file:

```
M-x mark-whole-buffer  →  M-x untabify
```

Or use the Python snippet (processes all `.org` files in `chapters/`):

```python
import glob, re

TAB_WIDTH = 4

def untabify(line):
    result, col = [], 0
    for ch in line:
        if ch == '\t':
            spaces = TAB_WIDTH - (col % TAB_WIDTH)
            result.append(' ' * spaces)
            col += spaces
        else:
            result.append(ch)
            col += 1
    return ''.join(result)

for path in glob.glob("chapters/*.org"):
    lines = open(path).readlines()
    out, in_src = [], False
    for line in lines:
        low = line.lower().lstrip()
        if low.startswith("#+begin_src"): in_src = True
        elif low.startswith("#+end_src"):  in_src = False
        out.append(untabify(line) if (in_src and "\t" in line) else line)
    with open(path, 'w') as f:
        f.writelines(out)
```

22.5 Named Box Environments (bclogo)

Instead of repeating the full `#+ATTR_LATEX` style string before every box, define named environments once in the preamble.

22.5.1 Preamble Definitions

Environment	Colour	Logo	Use for
`infobox`	light-cyan	lamp ()	Notes, tips, remarks
`warningbox`	yellow/orange	warning sign	Pitfalls, cautions
`syntaxbox`	light-gray	book ()	Syntax definitions

22.5.2 Usage in Chapters

```
# Before (full style repeated on every box):
#+ATTR_LATEX: :options [couleur=light-cyan,arrondi=0.1,logo=\bclampe,ombre=true]{Title}
#+BEGIN_bclogo
...
#+END_bclogo

# After (only the title changes):
```

```
#+ATTR_LATEX: :options {Title}
#+BEGIN_infobox
...
#+END_infobox
```

22.5.3 Box Type Quick Reference

```
#+ATTR_LATEX: :options {Note Title}
#+BEGIN_infobox
Content here.
#+END_infobox

#+ATTR_LATEX: :options {Warning Title}
#+BEGIN_warningbox
Content here.
#+END_warningbox

#+ATTR_LATEX: :options {Syntax Title}
#+BEGIN_syntaxbox
Content here.
#+END_syntaxbox
```

23 C++ Library Comparision

23.1 Overview

The following libraries are among the most widely used C++ libraries for Windows development, cross-platform applications, networking, and general-purpose programming.

Library	Purpose	Main Features
ATL	Windows COM programming	COM objects, ActiveX, smart pointers, registry support
WTL	Windows GUI framework	Lightweight wrappers around Win32, window classes, dialogs, controls
Boost	General-purpose C++ libraries	Smart pointers, threading, filesystem, regex, networking, containers, m
POCO	Cross-platform application framework	Networking, HTTP, JSON, XML, logging, database access, threading
ACE	Distributed and networked systems	Networking, concurrency, IPC, reactor/proactor patterns

23.2 ATL (Active Template Library)

Developed by Microsoft, ATL simplifies writing COM-based software.

23.2.1 Strengths

- COM and ActiveX support
- Efficient template-based implementation
- Small runtime overhead
- Excellent integration with Windows APIs

23.2.2 Weaknesses

- Windows only
- Primarily useful for COM development
- Less relevant for modern C++ unless maintaining legacy software

23.3 WTL (Windows Template Library)

WTL builds on ATL and provides a lightweight GUI framework.

23.3.1 Strengths

- Thin wrapper around the Win32 API
- Faster and smaller than frameworks like MFC
- Native Windows look and feel
- No large runtime

23.3.2 Weaknesses

- Windows only
- Limited documentation compared to larger frameworks
- No cross-platform support

23.4 Boost

Boost is the largest collection of peer-reviewed C++ libraries and has heavily influenced the C++ standard.

23.4.1 Popular Components

- Smart pointers
- Filesystem
- Threading
- Asio (networking)
- Regex
- Serialization
- Program Options
- Containers
- Graph algorithms

Many Boost libraries eventually become part of the ISO C++ standard, including smart pointers, filesystem, optional, variant, and shared_mutex.

23.4.2 Strengths

- Extremely comprehensive
- High quality
- Cross-platform
- Well tested

23.4.3 Weaknesses

- Large and sometimes complex
- Long compile times
- Template-heavy code can produce difficult compiler diagnostics

23.5 POCO (Portable Components)

POCO provides a cohesive framework for building cross-platform applications, particularly network services.

23.5.1 Main Modules

- HTTP/HTTPS
- TCP/UDP networking
- JSON
- XML
- Logging
- Configuration
- SQLite/MySQL/PostgreSQL connectors

- ZIP compression
- Cryptography
- Multithreading

23.5.2 Strengths

- Clean object-oriented API
- Cross-platform
- Excellent for server applications
- Easier to learn than Boost

23.5.3 Weaknesses

- Smaller ecosystem than Boost
- Less low-level flexibility

23.6 ACE (Adaptive Communication Environment)

ACE is a mature framework focused on high-performance distributed systems.

23.6.1 Main Features

- Socket abstraction
- Multithreading
- Synchronization
- Timers
- IPC
- Event demultiplexing
- Reactor and Proactor design patterns
- Service configurator

23.6.2 Strengths

- Excellent for high-performance networking
- Mature and stable
- Designed for mission-critical systems

23.6.3 Weaknesses

- Steep learning curve
- Older coding style
- Large codebase
- Less commonly used in new projects

23.7 Quick Comparison

Feature	ATL	WTL	Boost	POCO	ACE
Cross-platform	No	No	Yes	Yes	Yes
GUI	Limited	Yes	No	No	No
Networking	Minimal	No	Yes (Asio)	Yes	Yes
Database	No	No	Limited	Yes	Limited
HTTP/REST	No	No	Partial	Yes	Limited
COM support	Yes	Via ATL	No	No	No
Modern C++ style	Moderate	Moderate	Excellent	Good	Older
Learning curve	Medium	Medium	High	Medium	High
Typical domain	COM	Win32 UI	General C++	Servers, IoT	Telecom, Real-time
License	Microsoft	Microsoft	Boost	Boost	DOC (ACE)
Active development	Low	Low	Very active	Active	Maintenance

23.8 Recommendation for Modern Development

- **Boost** is the de facto standard collection of modern C++ libraries. It provides high-quality, portable components covering everything from smart pointers and containers to networking (Asio), serialization, and metaprogramming. Many Boost libraries have been adopted into the ISO C++ standard, making it an excellent foundation for almost any C++ project.
- **POCO** is ideal when developing cross-platform applications that need networking, HTTP/REST services, JSON/XML parsing, logging, configuration management, or database access. Its APIs are more cohesive and object-oriented than Boost's, making it easier to learn for application developers.
- **ATL** remains the preferred choice for developing or maintaining Windows COM components, shell extensions, ActiveX controls, and other technologies tightly coupled to the Windows platform. It is generally not used for new cross-platform projects.
- **WTL** is a lightweight alternative to MFC for native Win32 desktop applications. It produces small, efficient executables while retaining the native Windows look and feel. It is well suited for Windows utilities and engineering tools but is limited to the Windows ecosystem.
- **ACE** is primarily found in long-lived, mission-critical systems such as telecommunications, aerospace, defense, industrial automation, and real-time embedded applications. Although its design patterns (Reactor, Proactor, Acceptor-Connector) remain influential, new projects often prefer Boost.Asio or standalone Asio because they offer a more modern C++ interface and broader community support.

24 Practical Guide

If you are building…	Recommended library
Modern portable C++ software	Boost
Network servers or REST services	POCO (or Boost.Asio)
Native Windows desktop applications	WTL
COM or Windows shell extensions	ATL
Legacy telecom or real-time systems	ACE

Boost and POCO are the libraries most commonly selected for new cross-platform development. ATL and WTL remain highly relevant for Windows-specific software, while ACE is mainly encountered when maintaining or extending existing high-performance distributed systems.